

TRABAJO FINAL DE GRADO

Ingeniería de Sistemas

Facultad de Ciencias Exactas – UNICEN

Herramienta inteligente de formación de equipos para la reducción de conflictos en trabajos colaborativos soportados por computadora.

Autor: Martín Mineo

Directores:

Ing. Franco Berdun

Dr. Marcelo Armentano

Dedicatoria

A mi familia, novia Sole y amigos.

Agradecimientos

A Marcelo y a Franco, por haberme ayudado y guiado para que esto pueda concretarse. A todos aquellos que me acompañaron desde el principio y durante todos estos años. A todos aquellos que contribuyeron en mi crecimiento. A mi familia. A mis amigos. A mi maestro espiritual.

Índice

1. Introducción	9
1.1 Motivación	9
1.2 Propuesta	10
1.3 Conclusión	10
2. Marco Teórico	12
2.1 El modelo IPA de Bales	12
2.2 Minería de Datos (Data Mining)	15
2.2.1 Técnicas de minería de datos	16
2.2.1.a Clasificación	17
2.2.1.a.i Tabla de Decisión	17
2.2.1.a.ii Árboles de Decisión	17
2.2.1.a.iii Reglas de Clasificación	18
2.2.1.a.iv Clasificación Bayesiana	18
2.2.1.a.v Aprendizaje basado en ejemplos	18
2.2.1.a.vi Redes de neuronas	19
2.3 Machine Learning	19
2.4 Modelado de usuarios	20
2.5 Procesamiento de lenguaje natural	21
2.6 EAGLES	22
2.6.1 Atributos/valores recomendados	23
2.7 Plataformas de espacios compartidos	24
2.7.1 Sistema de control de versiones	24
2.7.1.a GIT	24
2.7.2 Servicio de mensajería instantánea	25
2.8 Trabajos relacionados	25
2.9 Conclusión	27
3. Desarrollo de la propuesta	29
3.1. Estructura de la herramienta a implementar	29
3.2. Consideraciones especiales a tener en cuenta	30
3.2.1 Servicio de mensajería instantánea	30
3.2.2 Machine Learning	30
3.2.2.a WEKA	30
3.2.3 Ingreso de datos al sistema	33
3.2.4 Procesamiento de lenguaje natural	34
3.2.4.1 FreeLing	34
3.3. Enfoques de clasificación propuestos	35
3.3.1 Enfoque de clasificación directa	35
3.3.2 Enfoque de clasificación en fases utilizando un clasificador	35

3.3.3 Enfoque de clasificación en fases utilizando múltiples clasificadores	36
3.4 Conclusión	37
4. Diseño e implementación de la herramienta	38
4.1. Requerimientos funcionales y restricciones de diseño	38
4.2. Especificación de requerimientos	39
4.3. Diseño de la solución	41
4.3.1 Model-View-Controller	41
4.3.1.a Modelo	44
4.3.1.b Vista	44
4.3.1.c Controlador	45
4.3.2 Diagrama de clases detallado	45
4.3.3 Diseño del módulo Conversores a ARFF	46
4.3.3.a Google Hangouts JSON	46
4.3.3.b Conversor Microsoft Excel	48
4.3.4 Diseño del módulo WEKA	49
4.3.5 Diseño del módulo FreeLing	52
4.3.5.a Módulo Morfológico (maco)	54
4.3.5.b Módulo Alternativas (alternatives)	54
4.3.5.c Módulo Probabilidades (probabilities)	55
4.3.5.d Módulo Etiquetador (hmmTagger)	55
4.3.5.e Módulo (ner)	55
4.3.5.f Módulo (nec)	55
4.3.5.g Módulo Parseador (chartParser)	55
4.3.6 Diseño del módulo Procesar Dataset	56
4.3.6.a Directo	57
4.3.6.b Fases	58
4.3.6.c Fase1, Fase 2 y Fase 3	59
4.3.7 Diseño del módulo Análisis IPA	61
4.4. Persistencia de los datos	62
4.4.1 Persistencia de resultados	62
4.4.2 Persistencia de datos intermedios	62
4.4.3 Caso especial: Entrenamiento Automático	63
4.5. Atributos y Escenarios de calidad	63
4.5.1 Atributo de calidad: Modificabilidad	63
4.5.2 Atributo de calidad: Performance	64
4.5.3 Atributo de calidad: Usabilidad	65
4.5.4 Atributo de calidad: Portabilidad	68
4.6 Conclusión	68
5. Aplicación de análisis de datos	69
5.1 Detección de conflictos en un grupo	69
5.2 Perfiles de personas	70

5.3 Generación de nuevos grupos de trabajo	71
5.3.1 Minimizar conflictos	71
5.4 Conclusión	73
6. Descripción del experimento	75
6.1. Conjunto de datos	75
6.2. Proceso del experimento	76
6.2.1 Enfoque directo	76
6.2.2 Enfoque en fases	76
6.3 Resultados	79
6.4 Tiempos de ejecución	87
6.4.1 Tiempo de entrenamiento	87
6.4.2 Tiempo de clasificación	89
6.5 Conclusión	89
7. Conclusiones	90
7.1 Limitaciones	91
7.2 Trabajos futuros	91
Apéndice I: Definición de categorías de Bales (resumen)	93
1. Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa	93
2. Muestra liberación de tensión, bromea, ríe, muestra satisfacción	93
3. Concuerda, muestra aceptación pasiva, comprende, coincide, obedece	94
4. Da sugerencia, indicación, da autonomía al otro	94
5. Da opinión, evaluación, análisis, expresa sentimiento, desea	95
6. Da orientación, información, repetición y confirmación	95
7. Pide orientación, información, repetición y confirmación	96
8. Pide opinión, evaluación, análisis, expresión de sentimientos	96
9. Pide sugerencia, indicación, posibles caminos de acción	97
10. Discrepa (está en desacuerdo), muestra rechazo pasivo, formalidad, retiene recursos	97
11. Muestra tensión, pide ayuda, deja de prestar atención	97
12. Muestra antagonismo/oposición, rebaja el estado de los demás, defiende o afirma el suyo	98
Apéndice II: Etiquetas EAGLE usadas por FreeLing	100
Adjetivos	100
Adverbios	101
Determinantes	102
Nombres	103
Verbos	103
Pronombres	105
Conjunciones	106
Interjecciones	106
Preposiciones	106

Puntuación	106
Numerales	107
Fechas y horas	107
Apéndice III: Formato ARFF utilizado por WEKA	108
ARFF (Attribute-Relation File Format)	108
Apéndice IV: Manual de usuario	112
Secciones de la aplicación	112
Procesar datos (simple)	112
Directo	113
Fases	114
Fase 1, Fase 2 y Fase 3	116
Procesar datos (avanzado)	118
Análisis de datos	120
Glosario	120
Referencias	122

Índice de tablas

[Tabla 2.1: Categorías y subcategorías IPA](#)

[Tabla 2.2: Conductas IPA y valores límite](#)

[Tabla 2.3: Técnicas de minería de datos](#)

[Tabla 2.4: Etiquetas EAGLE utilizadas por FreeLing](#)

[Tabla 3.1: ejemplo de N-gramas](#)

[Tabla 3.2: Técnicas de minería de datos e implementación en WEKA](#)

[Tabla 5.1: Formato de salida de la clasificación directa.](#)

[Tabla 5.2: Porcentaje de interacciones para cada conducta Bales para una persona.](#)

[Tabla 5.3: Conductas IPA y sus valores límite.](#)

[Tabla 5.4: Desviación para cada conducta Bales para una persona.](#)

[Tabla 6.1: Precisión \(%\) para los modelos de la clasificación directa.](#)

[Tabla 6.2: Precisión \(%\) para los modelos de la clasificación en fases \(único clasificador\).](#)

[Tabla 6.3: Precisión \(%\) para los modelos de la clasificación en fases \(múltiples clasificadores\).](#)

[Tabla 6.4: Matriz de confusión, mostrada con totales para tuplas positivas y negativas.](#)

[Tabla 6.5: Matriz de confusión, clasificador SMO con enfoque directo.](#)

[Tabla 6.6: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 1\).](#)

[Tabla 6.7: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 2\), entidades clasificadas como Socio-emocional en la fase 1.](#)

[Tabla 6.8: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 2\), entidades clasificadas como Orientado a la tarea en la fase 1.](#)

[Tabla 6.9: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 3\), entidades clasificadas como Positiva en la fase 2.](#)

[Tabla 6.10: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 3\), entidades clasificadas como Negativa en la fase 2.](#)

[Tabla 6.11: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 3\), entidades clasificadas como Pregunta en la fase 2.](#)

[Tabla 6.12: Matriz de confusión, clasificador SMO con enfoque en fases \(fase 3\), entidades clasificadas como Respu en la fase 2.](#)

[Tabla 6.13: tiempos de entrenamiento por clasificador y por enfoque de entrenamiento.](#)

[Tabla A2.1: Etiqueta EAGLE/FreeLing para adjetivos](#)

[Tabla A2.2: Etiqueta EAGLE/FreeLing para adverbios](#)

[Tabla A2.3: Etiqueta EAGLE/FreeLing para determinantes](#)

[Tabla A2.4: Etiqueta EAGLE/FreeLing para nombres](#)

[Tabla A2.5: Etiqueta EAGLE/FreeLing para verbos](#)

[Tabla A2.6: Etiqueta EAGLE/FreeLing para pronombres](#)

[Tabla A2.7: Etiqueta EAGLE/FreeLing para conjunciones](#)

[Tabla A2.8: Etiqueta EAGLE/FreeLing para interjecciones](#)

[Tabla A2.9: Etiqueta EAGLE/FreeLing para preposiciones](#)

[Tabla A2.10: Etiqueta EAGLE/FreeLing para puntuación](#)

[Tabla A2.11: Etiqueta EAGLE/FreeLing para numerales](#)

[Tabla A2.12: Etiqueta EAGLE/FreeLing para fechas y horas](#)

[Tabla A3.1: ejemplo de encabezado de un archivo ARFF.](#)

[Tabla A3.2: ejemplo de la sección de datos de un archivo ARFF.](#)
[Tabla A3.3: formato de @relation en un archivo ARFF.](#)
[Tabla A3.4: formato de @attribute en un archivo ARFF.](#)
[Tabla A3.5: formato de @attribute para atributos nominales en un archivo ARFF.](#)
[Tabla A3.6: ejemplo del formato de @attribute para atributos de cadenas de texto en un archivo ARFF.](#)
[Tabla A3.7: formato de @attribute para atributos de tipo fecha en un archivo ARFF.](#)
[Tabla A3.8: comienzo de la sección de datos en un archivo ARFF.](#)
[Tabla A3.9: ejemplo de formato una instancia en un archivo ARFF.](#)
[Tabla A3.10: ejemplo de un archivo ARFF.](#)
[Tabla A3.11: formato de las fechas en un archivo ARFF.](#)
[Tabla A4.1: Formato de salida de la clasificación directa.](#)
[Tabla A4.2: Formato de salida de la 1º fase de clasificación](#)
[Tabla A4.3: Formato de salida de la 2º fase de clasificación](#)
[Tabla A4.4: Formato de salida de la 3º fase de clasificación](#)
[Tabla G.1: ejemplo de un dataset de entrenamiento](#)
[Tabla G.2: ejemplo de un dataset de entrada](#)

Índice de figuras

[Figura 3.1: Categorías y subcategorías consideradas en IPA \(Bales\).](#)
[Figura 4.1: Diagrama de casos de uso](#)
[Figura 4.2: Patrón arquitectónico *Model-View-Controller*](#)
[Figura 4.3: Diagrama de componentes](#)
[Figura 4.4: Diagrama de componentes - Modelo](#)
[Figura 4.5: Diagrama de componentes - Vista](#)
[Figura 4.6: Diagrama de componentes - Controlador](#)
[Figura 4.7: Diagrama de clases completo \(simplificado\) de la aplicación](#)
[Figura 4.8: Diagrama de clases del parser de archivos JSON a archivos ARFF](#)
[Figura 4.9: Diagrama de las clases *GoogleHangoutsJsonParser* y *HangoutsJSON*.](#)
[Figura 4.10: Diagrama de clases del conversor de archivos Microsoft Excel a ARFF.](#)
[Figura 4.11: Diagrama de clases de la clase *Weka* y las clases que la extienden](#)
[Figura 4.12: Diagrama de clases de la clase *Weka*, algunas de sus hijas y clases propias de WEKA.](#)
[Figura 4.13: Diagrama de clases de las clases propias de WEKA utilizadas por nuestra aplicación](#)
[Figura 4.14: Diagrama de clases de las clases creadas para manejar *FreeLing*.](#)
[Figura 4.15: Diagrama de clases de las clases propias de *FreeLing* utilizadas por nuestra aplicación](#)
[Figura 4.16: Diagrama de clases de la clase *ProcessDataset*.](#)
[Figura 4.17: Diagrama de clases utilizadas para el procesamiento directo.](#)
[Figura 4.18: Diagrama de clases utilizadas para el procesamiento en fases \(utilizando un clasificador\).](#)
[Figura 4.19: Diagrama de clases utilizadas para el procesamiento en fases \(utilizando varios clasificadores\).](#)

[Figura 4.20: Diagrama de clases correspondientes al módulo *Procesar Dataset*.](#)

[Figura 4.21: Diagrama de clases del módulo *Análisis IPA*.](#)

[Figura 4.22: Dataset clasificado en menos de 5 minutos \(UCM\).](#)

[Figura 4.23: Elegir la cantidad de miembros del grupo \(UCM\).](#)

[Figura 4.24: Elegir el enfoque de clasificación y clasificar un dataset \(UCM\).](#)

[Figura 5.1: resultados del análisis de datos de un grupo.](#)

[Figura 5.2: diagrama de flujo del algoritmo para la creación de nuevos grupos de trabajo colaborativo buscando minimizar los conflictos.](#)

[Figura 6.1: Distribución de las interacciones de las categorías IPA.](#)

[Figura 6.2: Enfoque directo de clasificación.](#)

[Figura 6.3: Enfoque en fases de clasificación \(único clasificador\).](#)

1. Introducción

1.1 Motivación

Hoy en día, es frecuente que personas de distintos lugares del mundo se reúnan a través del uso de computadoras con conexión a internet y sus comunicaciones se realizan en línea por medio de mensajería instantánea de texto (comúnmente conocido como chat). Durante esta comunicación, cada persona interactúa con los demás integrantes del grupo y durante esta comunicación, es posible que se generen problemas o conflictos y que el cumplimiento del objetivo del grupo no se cumpla. Bales [Bales, 1950] identificó que estos problemas o conflictos pueden ser *de comunicación, de evaluación, de control, de decisión, de reducción de tensiones y de reintegración*. Por estos motivos, tanto la detección temprana de problemas colaborativos en un grupo como la creación de grupos que reduzcan la probabilidad de conflictos en el mismo es esencial.

Fahy [Fahy, 2006] realizó una investigación comparando las interacciones en grupos presenciales (como lo hacía Bales en sus estudios) con las interacciones de estudiantes en línea y descubrió que todas las interacciones realizadas por los estudiantes en línea eran posible clasificarlas dentro de alguna de las categorías mencionadas por Bales y que el comportamiento de los estudiantes de Fahy era similar al observado por Bales.

El análisis de las interacciones registradas entre los miembros del grupo se realizará utilizando un método denominado Análisis del Proceso de Interacción (Interaction Process Analysis – IPA) (Bales, 1950). Según IPA existen 12 categorías dentro de las cuales puede ser clasificada una interacción. Bales también define distintos rangos donde es menos probable que en un grupo de trabajo surjan problemas colaborativos. Estos rangos están dados por porcentajes (determinado por la cantidad de interacciones en el grupo) y cada categoría tiene su rango correspondiente. Si el grupo, con sus interacciones, se sale de estos “rangos seguros” se dice que este grupo tiene más chances de no poder cumplir su objetivo, lo cual puede provocar pérdida de tiempo, dinero, malestar en el grupo y hasta la no realización del trabajo. Por lo tanto se hace primordial detectar estos conflictos mencionados por Bales lo antes posible y realizar alguna acción para remediarlo.

Como este método está orientado particularmente al análisis de interacciones textuales, y la clasificación de estas interacciones en categorías IPA debe ser llevada a cabo por analistas designados para esta tarea, lo que se busca es tener la posibilidad de automatizar esta tarea mediante el uso de computadoras y hacer este proceso de clasificación más rápido. Una manera de hacerlo es a través de la combinación de las técnicas de Machine Learning y los estudios de Bales. Haciendo esto, se podrán detectar los posibles conflictos mencionados por Bales en un grupo de trabajo lo antes posible y poder tomar acción para resolverlos. El aprendizaje de máquina tiene como objetivo desarrollar técnicas que le permitan a las computadoras “aprender”. De forma más concreta, se trata de crear programas capaces de generalizar comportamientos a partir de una información suministrada en forma de ejemplos. Una de sus aplicaciones se centra en el procesamiento de lenguaje natural (NLP). A su vez, el modelado de usuarios permite recolectar información relevante sobre cada individuo, a partir del estudio de interacciones. Esto permite, disponer información de valor sobre el perfil de una persona y estudiar un

enfoque inteligente para, por ejemplo: asistir a mejorar la calidad del trabajo a través de la formación de grupos en base a criterios que reduzcan las diferencias entre los integrantes.

Aunque se ha estudiado el desarrollo de herramientas que automaticen esta clasificación [Cincunegui et al., 2015], no se han logrado resultados alentadores. Es por esto que la propuesta plantea la búsqueda de enfoques alternativos, para hacer efectiva la clasificación automática y posibilitar un ahorro en los recursos humano-temporales que esta tarea requiere. Para resolver este problema se utilizará un enfoque de aprendizaje de máquina (Machine Learning) para la detección de conductas colaborativas en texto libre surgidos en la dinámica de grupos de colaboración en línea.

1.2 Propuesta

Este trabajo pretende generar una herramienta que permita asistir y facilitar el trabajo de personas de que trabajan con grupos de trabajo colaborativo dándoles distintos tareas para que resuelvan. Los integrantes del grupo van a interactuar utilizando herramientas de chat online para resolver el problema dado. Se espera que la herramienta desarrollada en esta tesis, facilite la tarea de detectar conductas de colaboración y conflictos mencionados por Bales en los grupos de trabajo, permitiéndolo, a partir de los perfiles generados para cada persona, generar grupos de trabajo para los cuales se minimice la posibilidad de que surjan conflictos colaborativos.

En este trabajo se proponen distintas variantes de clasificación para analizar las interacciones en los trabajos colaborativos y clasificar el texto libre obtenido del chat de la plataforma colaborativa en alguna de las categorías IPA. Se evalúa un enfoque en el contexto del aula, obteniendo mejoras significativas con respecto a enfoques previos. A partir de la clasificación de las interacciones dentro de las categorías IPA, se alertará cuando el porcentaje de la cantidad de interacciones de algún grupo se encuentren fuera de los rangos propuestos por Bales. Por otro lado, la solución propuesta en esta tesis creará los perfiles de cada persona y brindará la opción de generar nuevos grupos de trabajo con los perfiles generados y así buscar tener grupos de trabajo que tengan baja probabilidad de tener conflictos colaborativos.

1.3 Conclusión

El objetivo de este trabajo es generar una herramienta que tendrá 2 objetivos claros:

- Que a partir de un chat grupal, se permita clasificar automáticamente cada interacción del chat en una de las categorías del método IPA de Bales e identificar si el grupo se encuentra en un posible conflicto.
- A partir de los perfiles de personas creados a partir de sus interacciones, sugerir la formación de grupos de trabajo para reducir la probabilidad de que se generen los conflictos mencionados en los estudios de Bales.

El resto de este informe está organizado de la siguiente manera. En el capítulo 2 se presenta el marco teórico y trabajos relacionados. En el capítulo 3 se presentará el desarrollo de la propuesta para resolver la problemática antes mencionada. En el capítulo 4 se presentarán en detalle el diseño e implementación de la herramienta. El capítulo 5

presenta la aplicación de análisis de datos. En el sexto capítulo se verá la descripción del experimento y se harán las conclusiones en el capítulo 7.

2. Marco Teórico

En el presente capítulo se hará una introducción a los estudios e investigaciones en los que se basa este trabajo, las herramientas disponibles en el mercado que se pueden utilizar para su resolución y los trabajos relacionados sobre el tema.

2.1 El modelo IPA de Bales

El propósito del sistema de Análisis del Proceso de Interacción (IPA, por sus siglas en inglés Interaction Process Analysis) de Bales es identificar y registrar la naturaleza (no el contenido) de cada acto por separado en la interacción de un grupo. En la búsqueda de usar el sistema IPA, el primer problema es aprender a identificar una “unidad” de interacción. La definición básica desarrollada por Bales es que es una sentencia simple o su equivalente. La unidad a ser registrada es un “acto”. Una *acto* es una comunicación o una indicación, ya sea verbal o no verbal, que, dentro del contexto, puede ser identificada por otro miembro como equivalente a una sentencia simple. Un acto puede ser definido como una comunicación o interacción suficientemente completa para permitir a la otra persona interpretarla, reaccionar en relación a su contenido y a quién emitió el acto.

El método IPA es una forma de clasificar interacciones directas, cara a cara mientras está sucediendo, acto por acto, y una serie de formas de resumir y analizar los datos resultantes para que aporten información útil. La clasificación es clara e inequívocamente una cuestión de interpretación, esto quiere decir que involucra que quien está clasificando le de significado, la “lectura” del contenido, la inferencia de que el comportamiento tiene funciones, ya sea por intención o por efecto.

Bales definió en su modelo que cada acto puede ser clasificado dentro de distintas categorías, separadas en 3 niveles. El primer nivel define el “área” en la que puede ser clasificado el acto, las áreas pueden ser “socio-emocional” o “tarea”. Luego, existe un nivel intermedio donde se define la “reacción”, estas reacciones pueden ser “positiva” o “negativa” (dentro del área *socio-emocional*) y “respuesta” o “pregunta” (dentro del área *tarea*). Finalmente, está el tercer nivel que define la “conducta”, esta última tiene 12 categorías distintas, 3 para cada *reacción*. En la siguiente tabla (Tabla 2.1) se puede ver el modelo completo.

Para más detalles sobre cada una de las conductas vea el Apéndice I.

Área	Reacción	Conducta
Socio-emocional	Positiva	C1. Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa
		C2. Muestra liberación de tensión, bromea, ríe, muestra satisfacción
		C3. Concuerda, muestra aceptación pasiva, comprende, coincide, obedece
	Negativa	C10. Discrepa (está en desacuerdo), muestra rechazo pasivo, formalidad, retiene recursos
		C11. Muestra tensión, pide ayuda, deja de prestar atención
		C12. Muestra antagonismo/oposición, rebaja el estado de los demás, defiende o afirma el suyo
Tarea	Respuestas	C4. Da sugerencia, indicación, da autonomía al otro
		C5. Da opinión, evaluación, análisis, expresa sentimiento, desea
		C6. Da orientación, información, repetición y confirmación
	Preguntas	C7. Pide orientación, información, repetición y confirmación
		C8. Pide opinión, evaluación, análisis, expresión de sentimientos
		C9. Pide sugerencia, indicación, posibles caminos de acción

Tabla 2.1: Categorías y subcategorías IPA

Bales también define distintos rangos donde es menos probable que en un grupo de trabajo surjan problemas colaborativos. Estos rangos están dados por porcentajes (determinado por la cantidad de interacciones en el grupo) y cada categoría tiene su rango correspondiente (Tabla 2.1). Estos problemas o conflictos fueron denominados: *problemas de comunicación, de evaluación, de control, de decisión, de reducción de tensiones y de reintegración*.

Conflicto	Conducta	Límite inferior	Límite superior
Comunicación	Da orientación, información, repetición y confirmación	14%	30%
	Pide orientación, información, repetición y confirmación	2%	11%
Evaluación	Da opinión, evaluación, análisis, expresa sentimiento, desea	21%	40%
	Pide opinión, evaluación, análisis, expresión de sentimientos	1%	9%
Control	Da sugerencia, indicación, da autonomía al otro	4%	11%
	Pide sugerencia, indicación, posibles caminos de acción	0%	5%
Decisión	Concuerda, muestra aceptación pasiva, comprende, coincide, obedece	6%	20%
	Discrepa (está en desacuerdo), muestra rechazo pasivo, formalidad, retiene recursos	3%	13%
Reducción de tensión	Muestra liberación de tensión, bromea, ríe, muestra satisfacción	3%	14%
	Muestra tensión, pide ayuda, deja de prestar atención	1%	10%
Reintegración	Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa	0%	5%
	Muestra antagonismo/oposición, rebaja el estado de los demás, defiende o afirma el suyo	0%	7%

Tabla 2.2: Conductas IPA y valores límite

2.2 Minería de Datos (Data Mining)

No sorprende que la minería de datos, como un tema realmente interdisciplinario, se pueda definir de muchas maneras diferentes. Incluso el término minería de datos en realidad no presenta todos los componentes principales. Para hablar de la extracción de oro que se encuentra en rocas o arena, se dice minería de oro en lugar de extracción de roca o arena. Análogamente, la minería de datos debería haberse denominado más apropiadamente "extracción de conocimiento a partir de datos", que desafortunadamente es algo larga. Sin embargo, su forma abreviada, la *minería del conocimiento* puede no reflejar el énfasis en la minería a partir de grandes cantidades de datos. Sin embargo, la minería es un término vívido que caracteriza el proceso que encuentra un pequeño conjunto de pepitas preciosas a partir de una gran cantidad de materia prima. Por lo tanto, una denominación tan incorrecta que lleva tanto "datos" como "minería" se convirtió en la elección popular. Además, muchos otros términos tienen un significado similar a la minería de datos, por ejemplo, *minería de conocimiento a partir de datos*, *extracción de conocimiento*, *análisis de datos/patrones*, *arqueología de datos* y *dragado de datos* [Han et al., 2012].

Muchas personas tratan la minería de datos como un sinónimo de otro término popularmente utilizado, el **descubrimiento de conocimiento a partir de datos**, o **KDD** (Knowledge Discovery from Data) por sus siglas en inglés, mientras que otros ven la minería de datos como simplemente un paso esencial en el proceso de descubrimiento de conocimiento. El proceso de descubrimiento de conocimiento se muestra como una secuencia iterativa de los siguientes pasos:

1. Limpieza de datos (para eliminar ruido y datos inconsistentes).
2. Integración de datos (donde se pueden combinar múltiples fuentes de datos)
3. Selección de datos (donde los datos relevantes para la tarea de análisis se recuperan de la base de datos).
4. Transformación de datos (donde los datos se transforman y consolidan en formas apropiadas para la minería mediante la realización de operaciones de resumen o agregación).
5. Minería de datos (un proceso esencial donde se aplican métodos inteligentes para extraer patrones de datos).
6. Evaluación de patrones (para identificar los patrones verdaderamente interesantes que representan el conocimiento basados en medidas de interés).
7. Presentación del conocimiento (donde las técnicas de visualización y representación del conocimiento se utilizan para presentar el conocimiento minero a los usuarios).

Los pasos 1 a 4 son diferentes formas de preprocesamiento de datos, donde se preparan los datos para la minería. El paso de minería de datos puede interactuar con el usuario o una base de conocimiento. Los patrones interesantes se presentan al usuario y pueden almacenarse como nuevos conocimientos en la base de conocimiento.

Lo detallado anteriormente muestra la extracción de datos como un paso en el proceso de descubrimiento de conocimiento, aunque esencial porque descubre patrones ocultos para la evaluación. Sin embargo, en la industria, en los medios y en

el medio de investigación, el término *minería de datos* se usa a menudo para referirse a todo el proceso de descubrimiento de conocimiento (tal vez porque el término es más corto que el *descubrimiento de conocimiento de los datos*). Por lo tanto, se adopta una visión amplia de la funcionalidad de minería de datos: la **minería de datos** es el proceso de descubrir patrones y conocimientos interesantes a partir de grandes cantidades de datos. Las fuentes de datos pueden incluir bases de datos, almacenes de datos, la Web, otros repositorios de información o datos que se transmiten al sistema de forma dinámica [Han et al., 2012].

2.2.1 Técnicas de minería de datos

Las técnicas de Minería de Datos buscan obtener patrones o modelos a partir de los datos recopilados. Se clasifican en dos grandes categorías: *supervisadas o predictivas* y *no supervisadas o descriptivas* [Weiss et al., 1998][Molina López et al., 2006], en la siguiente tabla se muestra una lista de las principales técnicas de minería de datos.

Técnicas de Minería de Datos			
Técnicas	No supervisadas	Agrupamiento	N Numérico
			N Conceptual
			N Probabilístico
		Asociación	A priori
		Predicción	Regresión
			Árboles de predicción
			Estimador de núcleos
	Supervisadas	Clasificación	Tabla de decisión
			Árboles de decisión
			Inducción de reglas
			Bayesianas
			Basado en ejemplares
			Redes neuronales

Tabla 2.3: Técnicas de minería de datos

Una técnica constituye el enfoque conceptual para extraer la información de los datos, y, en general es implementada por varios algoritmos. Cada algoritmo representa, en la práctica, la manera de desarrollar una determinada técnica paso a paso, de forma que es necesario un entendimiento de alto nivel de los algoritmos para saber cuál es la técnica más

apropiada para cada problema. De la misma manera, es necesario entender los parámetros y las características de los algoritmos para preparar los datos a analizar.

El *aprendizaje inductivo no supervisado* estudia el aprendizaje sin la ayuda del *maestro*; es decir, se aborda el aprendizaje sin supervisión, que trata de ordenar los ejemplos en una jerarquía según las regularidades en la distribución de los pares atributo-valor sin la *guía* del atributo especial *clase*. Éste es el proceder de los sistemas que realizan **clustering conceptual** y de los que se dice también que adquieren nuevos conceptos. Otra posibilidad contemplada para estos sistemas es la de sintetizar conocimiento cualitativo o cuantitativo, objetivo de los sistemas que llevan a cabo tareas de descubrimiento.

En el *aprendizaje inductivo supervisado* existe un atributo especial, normalmente denominado *clase*, presente en todos los ejemplos que especifica si el ejemplo pertenece o no a un cierto concepto, que será el objetivo del aprendizaje. Mediante una generalización del papel del atributo *clase*, cualquier atributo puede desempeñar ese papel, convirtiéndose la **clasificación** de los ejemplos según los valores del atributo en cuestión, en el objeto del aprendizaje. Expresado en una forma breve, el objetivo del aprendizaje supervisado es: a partir de un conjunto de ejemplos, denominados de entrenamiento, de un cierto dominio *D* de ellos, construir criterios para determinar el valor del atributo *clase* en un ejemplo cualquiera del dominio. Esos criterios están basados en los valores de uno o varios de los otros pares (atributo; valor) que intervienen en la definición de los ejemplos. Dentro de este tipo de aprendizaje se pueden distinguir dos grandes grupos de técnicas: la *predicción* y la *clasificación*.

En la siguiente sección se presenta de manera breve una descripción de cada una de las técnicas *supervisadas*, estas técnicas son las que mejor se adaptan a las necesidades de este trabajo.

2.2.1.a Clasificación

La clasificación es el proceso de dividir un conjunto de datos en grupos mutuamente excluyentes [Waiss, 1991][Langley, 1996][Mitchell, 1997][Molina López et al., 2006], de tal forma que cada miembro de un grupo esté lo mas cerca posible de otros y grupos diferentes estén lo más lejos posible de otros, donde la distancia se mide con respecto a las variables especificadas, que se quieren predecir.

2.2.1.a.i Tabla de Decisión

La tabla de decisión constituye la forma más simple y rudimentaria de representar la salida de un algoritmo de aprendizaje, que es justamente *representarlo como la entrada*.

Estos algoritmos consisten en seleccionar subconjuntos de atributos y calcular su precisión para predecir o clasificar los ejemplos. Una vez seleccionado el mejor de los subconjuntos, la tabla de decisión estará formada por los atributos seleccionados (más la *clase*), en la que se insertarán todos los ejemplos de entrenamiento únicamente con el subconjunto de atributos [Molina López et al., 2006].

2.2.1.a.ii Árboles de Decisión

El aprendizaje de árboles de decisión está englobado como una metodología del aprendizaje supervisado. La representación que se utiliza para las descripciones del concepto adquirido es el árbol de decisión, que consiste en una representación del

conocimiento relativamente simple y que es una de las causas por la que los procedimientos utilizados en su aprendizaje son más sencillos que los de sistemas que utilizan lenguajes de representación más potentes, como redes semánticas, representaciones en lógica de primer orden, etc. No obstante, la potencia expresiva de los árboles de decisión es también menor que la de esos otros sistemas. El aprendizaje de árboles de decisión suele *ser más robusto frente al ruido y conceptualmente sencillo*, aunque los sistemas que han resultado del perfeccionamiento y de la evolución de los más antiguos se complican con los procesos que incorporan para ganar fiabilidad. La mayoría de los sistemas de aprendizaje de árboles suelen ser no incrementales, pero existe alguna excepción [Utgoft, 1988][Molina López et al., 2006].

2.2.1.a.iii Reglas de Clasificación

Las técnicas de Inducción de Reglas [Quinlan, 1987][Quinlan, 1993][Molina López et al., 2006] surgieron hace más de dos décadas y permiten la generación y contraste de árboles de decisión, o reglas y patrones a partir de los datos de entrada. La información de entrada será un conjunto de casos donde se ha asociado una clasificación o evaluación a un conjunto de variables o atributos. Con esa información, estas técnicas obtienen el árbol de decisión o conjunto de reglas que soportan la evaluación o clasificación [Clark et al., 1989][Hong et al., 1986][Molina López et al., 2006]. En los casos en que la información de entrada posee algún tipo de “ruido” o defecto (insuficientes atributos o datos, atributos irrelevantes o errores u omisiones en los datos), estas técnicas pueden habilitar métodos estadísticos de tipo probabilístico para generar árboles de decisión recortados o podados. También en estos casos pueden identificar los atributos irrelevantes, la falta de atributos discriminantes o detectar huecos de conocimiento [Major et al., 1995][Molina López et al., 2006].

2.2.1.a.iv Clasificación Bayesiana

Los clasificadores Bayesianos [Duda et al., 1973][Molina López et al., 2006] son clasificadores estadísticos, que pueden predecir tanto las probabilidades del número de miembros de clase, como la probabilidad de que una muestra dada pertenezca a una clase particular.

La clasificación Bayesiana se basa en el teorema de Bayes, y los clasificadores Bayesianos han demostrado una *alta exactitud y velocidad cuando se han aplicado a grandes bases de datos*. Diferentes estudios comparando los algoritmos de clasificación han determinado que un clasificador Bayesiano sencillo conocido como el clasificador “*naive Bayesiano*” [John, 1997][Molina López et al., 2006] es comparable en rendimiento a un árbol de decisión y a clasificadores de redes de neuronas.

2.2.1.a.v Aprendizaje basado en ejemplos

El aprendizaje basado en ejemplos o instancias [Briscoe et al., 1996][Molina López et al., 2006] tiene como principio de funcionamiento, en sus múltiples variantes, el almacenamiento de ejemplos: en unos casos todos los ejemplos de entrenamiento, en otros solo los más representativos, en otros los incorrectamente clasificados cuando se clasifican por primera vez, etc. La clasificación posterior se realiza por medio de una función que mide la proximidad o parecido. Dado un ejemplo para clasificar, se le clasifica de acuerdo al

ejemplo o ejemplos más próximos. El bias (sesgo) que rige este método es la proximidad; es decir, la generalización se guía por la proximidad de un ejemplo a otros.

Se han enumerado ventajas e inconvenientes del aprendizaje basado en ejemplares [Briscoe et al., 1996][Molina López et al., 2006], pero se suele considerar no adecuado para el tratamiento de atributos no numéricos y valores desconocidos. Las mismas medidas de proximidad sobre atributos simbólicos suelen proporcionar resultados muy dispares en problemas diferentes.

2.2.1.a.vi Redes de neuronas

Las redes de neuronas constituyen una nueva forma de analizar la información con una diferencia fundamental con respecto a las técnicas tradicionales: son capaces de detectar y aprender complejos patrones y características dentro de los datos [Molina López et al., 2006]. Se comportan de forma parecida a nuestro cerebro aprendiendo de la experiencia y del pasado, y aplicando tal conocimiento a la resolución de problemas nuevos. Este aprendizaje se obtiene como resultado del adiestramiento y éste permite la sencillez y la potencia de adaptación y evolución ante una realidad cambiante y muy dinámica. Una vez adiestradas las redes de neuronas pueden hacer previsiones, clasificaciones y segmentación. Presentan además, una eficiencia y fiabilidad similar a los métodos estadísticos y sistemas expertos, si no mejor, en la mayoría de los casos. En aquellos casos de muy alta complejidad las redes neuronales se muestran como especialmente útiles dada la dificultad de modelado que supone para otras técnicas. Sin embargo las redes de neuronas tienen el inconveniente de la dificultad de acceder y comprender los modelos que generan y presentan dificultades para extraer reglas de tales modelos. Otra característica, es que son capaces de trabajar con datos incompletos e, incluso, contradictorios lo que, dependiendo del problema, puede resultar una ventaja o un inconveniente. Las redes de neuronas poseen las dos formas de aprendizaje: supervisado y no supervisado; ya comentadas [Weiss et al., 1998][Molina López et al., 2006] derivadas del tipo de paradigma que usan: el no supervisado (usa paradigmas como los ART "Adaptive Resonance Theory"), y el supervisado que suele usar el paradigma del "Backpropagation" [Molina López et al., 2006][Rumelhart et al., 1986].

2.3 Machine Learning

El campo *Machine Learning* o del *Aprendizaje de Máquina* aborda el problema de cómo construir programas de computadora que mejoren automáticamente con la experiencia. En los últimos años se han desarrollado muchas aplicaciones exitosas de aprendizaje de máquina, desde programas de extracción de datos que detectan transacciones fraudulentas con tarjeta de crédito, sistemas de filtrado de información que aprenden las preferencias de lectura de los usuarios, hasta vehículos autónomos que aprenden a conducir en autopistas públicas. Al mismo tiempo, ha habido avances importantes en la teoría y los algoritmos que forman los cimientos de este campo. En el campo conocido como minería de datos, los algoritmos de aprendizaje de máquina se utilizan de forma rutinaria para descubrir conocimientos valiosos de grandes bases de datos comerciales que contienen registros de mantenimiento de equipos, solicitudes de préstamos, transacciones financieras, registros médicos y similares.

Algunos logros específicos permiten vislumbrar el estado del arte: se han desarrollado programas que aprenden con éxito a reconocer las palabras habladas [Waibel et al., 1989; Lee 1989], predicen tasas de recuperación de pacientes con neumonía [Cooper et al., 1997], detectan uso fraudulento de tarjetas de crédito, conducir vehículos autónomos en autopistas públicas [Pomerleau 1989] y jugar juegos como el backgammon en niveles que se acercan a la actuación de campeones mundiales humanos [Tesauro 1992, 1995]. El aprendizaje automático es un campo intrínsecamente multidisciplinario. Se basa en los resultados de inteligencia artificial, probabilidad y estadística, teoría de complejidad computacional, teoría de control, teoría de la información, filosofía, psicología, neurobiología y otros campos.

A continuación se presentan algunas disciplinas y ejemplos de su influencia sobre machine learning [Mitchell, 1997]:

- Inteligencia artificial: aprender representaciones simbólicas de conceptos. Aprendizaje automático como un problema de búsqueda. Aprender como un enfoque para mejorar la resolución de problemas. Usar el conocimiento previo junto con los datos de capacitación para guiar el aprendizaje.
- Métodos bayesianos: el teorema de Bayes como base para calcular probabilidades de hipótesis. El clasificador ingenuo de Bayes. Algoritmos para estimar valores de variables no observadas.
- Teoría de la complejidad computacional: límites teóricos sobre la complejidad inherente de las diferentes tareas de aprendizaje, medidos en términos de esfuerzo computacional, número de ejemplos de entrenamiento, cantidad de errores, etc. requeridos para aprender.
- Teoría de control: procedimientos que aprenden a controlar procesos para optimizar objetivos predefinidos y que aprenden a predecir el siguiente estado del proceso que están controlando.
- Teoría de la información: medidas de entropía y contenido de información. La descripción mínima se acerca al aprendizaje. Códigos óptimos y su relación con secuencias de entrenamiento óptimas para codificar una hipótesis.
- Filosofía: la navaja de Occam, sugiriendo que la hipótesis más simple es la mejor. Análisis de la justificación para generalizar más allá de los datos observados.
- Psicología y neurobiología: la ley de poder de la práctica, que establece que en un rango muy amplio de problemas de aprendizaje, el tiempo de respuesta de las personas mejora con la práctica de acuerdo con una ley de poder. Estudios neurobiológicos que motivan modelos de aprendizaje de redes neuronales artificiales.
- Estadísticas: caracterización de los errores (por ejemplo, sesgo y varianza) que se producen al estimar la precisión de una hipótesis basada en una muestra limitada de datos. Intervalos de confianza, pruebas estadísticas

2.4 Modelado de usuarios

Según Rich [Rich, 1989], existen muchas definiciones distintas sobre el modelado de usuarios, entre las que se encuentran:

- El modelado de usuario es conocimiento sobre el usuario de un sistema, codificado para el propósito de mejorar la interacción.

- Un modelo de usuario es una fuente de conocimiento en un sistema de diálogo de lenguaje natural que contiene suposiciones sobre todos los aspectos de un usuario que pueden ser relevantes para el comportamiento de diálogo del sistema [Wahlster, Kobsa, 1986].
- El modelado de usuarios es una subclase de modelado de agentes. Un modelado de agente es un modelo de una entidad, sin importar su relación con el sistema haciendo el modelado. Un modelado de usuario es el modelo del agente que está actualmente interactuando con el sistema [Kass, Finin, 1988].

Fischer [Fischer, 2000], en su libro también da su definición sobre el modelado de usuario:

- Los modelos de usuario se definen como el modelo que un sistema tiene de las características de los usuarios, que residen dentro de un entorno computacional.

Para el caso presentado en este trabajo, se considera que la definición que más se adapta al modelado de usuarios que se está realizando es la proporcionada por [Kass, Finin, 1988].

Durante el desarrollo de este trabajo se utilizará el modelado de usuarios para tener una representación de los actos que realiza cada persona y luego, estos perfiles de usuarios se usarán para la creación de nuevos grupos de personas buscando minimizar la posibilidad de que surjan los conflictos definidos por Bales para el modelo IPA mencionado anteriormente. La información necesaria para crear un perfil de usuario es simplemente contabilizar la cantidad de veces que esta persona escribe en el chat actos de cada uno de los comportamientos IPA.

2.5 Procesamiento de lenguaje natural

El objetivo de este campo es lograr que las computadoras realicen tareas útiles relacionadas con el lenguaje humano, tareas como permitir la comunicación entre humanos y máquinas, mejorar de la comunicación humano-humano o simplemente el procesamiento útil de texto o voz [Jurafsky, 2009].

Lo que distingue a las aplicaciones de procesamiento de lenguaje de otros sistemas de procesamiento de datos es su uso del conocimiento del lenguaje. Considere el programa wc de Unix, que cuenta el número total de bytes, palabras y líneas en un archivo de texto. Cuando se usa para contar bytes y líneas, wc es una aplicación de procesamiento de datos común. Sin embargo, cuando se usa para contar las palabras en un archivo, requiere conocimiento sobre lo que significa ser una palabra y, por lo tanto, se convierte en un sistema de procesamiento de lenguaje.

Una de las ideas clave de los últimos 50 años de investigación en el procesamiento del lenguaje es que los diversos tipos de conocimiento relacionados con el procesamiento de lenguaje se pueden capturar mediante el uso de un pequeño número de modelos formales o teorías. Afortunadamente, estos modelos y teorías provienen de los kits de herramientas estándar de informática, matemática y lingüística, y deberían ser, en general, familiares para las personas capacitadas en esos campos. Entre los modelos más importantes se encuentran las máquinas de estado, los sistemas de reglas, la lógica, los modelos probabilísticos y los modelos de espacio vectorial. Estos modelos, a su vez, se

prestan a un pequeño número de algoritmos, entre los más importantes están los algoritmos de búsqueda de espacio de estado como la programación dinámica y algoritmos de aprendizaje automático tales como clasificadores y Expectativa-Maximización (EM) y otros algoritmos de aprendizaje.

Las herramientas de aprendizaje de máquina, como los clasificadores y los modelos de secuencia, desempeñan un papel importante en muchas tareas de procesamiento del lenguaje. En función de los atributos que describe cada objeto, un clasificador intenta asignar un solo objeto a una sola clase, mientras que un modelo de secuencia intenta clasificar conjuntamente una secuencia de objetos en una secuencia de clases.

Por ejemplo, en la tarea de decidir si una palabra está escrita correctamente o no, los clasificadores como los árboles de decisión, las máquinas de vectores de soporte, los modelos de mezclas gaussianas y la regresión logística podrían usarse para tomar una decisión binaria (correcta o incorrecta) para una palabra a la vez. Los modelos de secuencia como los modelos de Markov ocultos, los modelos de Markov de entropía máxima y los campos aleatorios condicionales podrían usarse para asignar etiquetas correctas/incorrectas a todas las palabras en una oración a la vez.

Finalmente, los investigadores en el procesamiento del lenguaje utilizan muchas de las mismas herramientas metodológicas que se usan en la investigación del aprendizaje de máquina: el uso de distintas series de entrenamiento y pruebas, técnicas estadísticas como la validación cruzada y la evaluación cuidadosa de sistemas entrenados.

2.6 EAGLES

EAGLES (Expert Advisory Group on Language Engineering Standards) [EAGLES, 1996a][EAGLES, 1996b] es una iniciativa de la Comisión Europea, dentro del programa de Investigación e Ingeniería Lingüística de la DG XIII, cuyo objetivo es acelerar la provisión de estándares para:

- Recursos lingüísticos a gran escala (tales como corpus de textos, léxicos computacionales y corpus de habla).
- Los medios para manipular dicho conocimiento, a través de formalismos lingüísticos computacionales, marcan idiomas y varias herramientas de software.
- Medios de tasar y evaluar recursos, herramientas y productos.

Los principales esfuerzos en EAGLES se concentran en los siguientes tipos de actividades, que, como se ve a continuación, muestran cómo, en líneas muy generales, el trabajo se organiza en los cinco grupos de trabajo.

- Detectar aquellas áreas maduras para la estandarización a corto plazo vs. áreas que todavía necesitan investigación y desarrollo básicos.
- Evaluar y descubrir áreas en las que existe un consenso sobre recursos lingüísticos, formalismos, sistemas de habla y prácticas comunes existentes.
- Examinar y evaluar las propuestas disponibles o las especificaciones aportadas para evaluar el potencial de armonización y convergencia y la aparición de normas.
- Proponer especificaciones comunes para conjuntos básicos de fenómenos básicos, recomendaciones de buenas prácticas, metodologías estándar, etc., sobre las cuales se pueda encontrar un consenso.

- Establecer pautas para la representación de conjuntos básicos de características básicas, para la representación de recursos, etc.
- Estudios de viabilidad para áreas menos maduras.
- Sugerir acciones a seguir para un procedimiento gradual que conduzca a la creación de recursos reutilizables multilingües, la elaboración de metodologías y herramientas de evaluación, etc.

EAGLES tiene la intención de ser capaz de codificar todas la características morfológicas de la mayoría de los lenguajes Europeos. El etiquetado gramatical de EAGLES consiste en etiquetas de largo variable donde cada letra corresponde a una característica morfológica. La primera letra en la etiqueta es siempre la categoría. La categoría determina el largo de la etiqueta y la interpretación de cada letra en la etiqueta.

2.6.1 Atributos/valores recomendados

Estos se especifican a continuación bajo encabezados de parte del discurso (part-of-speech). Cada encabezado numerado se refiere al número asignado en la categoría principal. No todos los idiomas de la Unión Europea instanciarán todos los atributos o todos los valores de un atributo individual. Para cada atributo, 0 designa un valor cero, lo que significa que "este atributo no es aplicable" para el idioma particular, o para una palabra de texto particular en ese idioma. El requisito estándar para estos atributos/valores recomendados es que, si se producen en un idioma particular, es aconsejable que el conjunto de etiquetas de ese idioma los codifique.

Categoría	Código
Adjetivo	A
Adverbio	R
Determinante	D
Nombre	N
Verbo	V
Pronombre	P
Conjunción	C
Interjección	I
Preposición	S
Puntuación	F
Cifra	Z
Fecha/Hora	W

Tabla 2.4: Etiquetas EAGLE utilizadas por FreeLing

Para mayor detalle, en el Apéndice II se presentan, en formato de tablas, las etiquetas EAGLE de la forma en que FreeLing las utiliza.

2.7 Plataformas de espacios compartidos

2.7.1 Sistema de control de versiones

Un control de versiones es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, de modo que se aposite recuperar versiones específicas más adelante. Dicho sistema permite regresar a versiones anteriores de los archivos, regresar a una versión anterior del proyecto completo, comparar cambios a lo largo del tiempo, ver quién modificó por última vez algo que pueda estar causando problemas, ver quién introdujo un problema y cuándo, y mucho más. Usar un sistema de control de versiones (VCS por sus siglas en inglés) también significa generalmente que si se arruinan o pierden archivos, será posible recuperarlos fácilmente [Chacon et al., 2014].

2.7.1.a GIT

Git es un software distribuido, gratuito y de código abierto diseñado para el control de versiones creado por Linus Torvalds, pensando en la eficiencia y la confiabilidad del mantenimiento de versiones de proyectos de todo tipo de tamaño.

2.7.2 Servicio de mensajería instantánea

La mayoría de los sistemas de mensajería instantánea funcionan de la misma manera. Cuando uno inicia la aplicación, el cliente de mensajería se conecta al servidor de mensajería. El servidor de mensajería verifica el nombre de usuario y la contraseña y registra al cliente. Una vez que ha iniciado sesión, el cliente envía al servidor su dirección IP, el número de puerto que se ha asignado al servicio de mensajería instantánea y los nombres de todos en la lista de contactos del usuario. El servidor crea un archivo de sesión temporal que contiene la información de conexión y comprueba para ver quién en la lista de contactos también está conectado. Como el cliente tiene la dirección IP y el número de puerto de la computadora de la persona a la que se envió el mensaje, el mensaje se envía directamente al cliente en la computadora de esa persona. En otras palabras, el servidor no está involucrado en este punto. Toda la comunicación se realiza directamente entre los dos clientes [Rouse, 2003][Jeff et al., 2001].

Cuando el servidor encuentra contactos que han iniciado sesión, envía un mensaje al cliente con su información de conexión y envía la información de conexión a los contactos. Tan pronto como toda la información de conexión haya sido enviada y confirmada, la mensajería instantánea puede comenzar. La otra persona recibe su mensaje instantáneo y responde. Los mensajes instantáneos de cada persona aparecen en esta ventana en ambas computadoras [Rouse, 2003].

El lugar de trabajo moderno es inherentemente colaborativo, y esta colaboración se basa en la comunicación efectiva entre los compañeros de trabajo. Muchas herramientas de comunicación se han vuelto cada vez más disponibles y aceptados en las comunicaciones en el lugar de trabajo [Turner et al., 2010]. Según Woodard [Woodard, 2011], señaló que con la introducción de Internet, gran parte de la comunicación se ha realizado a través de la pantalla de nuestros monitores de computadora o incluso a través de nuestros teléfonos. Además, señala que el mundo se ha convertido en una pequeña aldea global, e Internet es la súper autopista que conecta cada casa, organización e institución en este mundo del tamaño de un puño de hoy.

D'Silva [D'Silva, 2013] observa que "además de ser un portal de información, entretenimiento y educación, Internet desempeña otro papel muy integral que el de un dispositivo de comunicación". Hoy se ha demostrado que Internet es una de las formas más económicas en que las personas de todos los continentes se pueden comunicar entre sí. El correo electrónico y la mensajería instantánea han ocupado hoy el lugar del servicio de mensajería y fax como dispositivos confiables de comunicación corporativa. Una de las muchas adiciones a esto fue la de la mensajería instantánea. La mensajería instantánea, o IM, permite la comunicación instantánea a cualquier persona en la misma ciudad, en un estado diferente o en cualquier parte del mundo [D'Silva, 2013].

2.8 Trabajos relacionados

El Análisis del Proceso de Interacción (IPA) para la comunicación mediada por computadoras se ha utilizado en algunas investigaciones en el área de Ciencias de la Computación. Birnholtz [Birnholtz et al., 2013], por ejemplo, presentó un estudio experimental de grupos de dos personas de edición de documentos juntos. El enfoque del

estudio, sin embargo, no fue clasificar automáticamente las interacciones en las categorías de IPA, sino analizar el mantenimiento del grupo, el manejo de las impresiones y el comportamiento centrado en las relaciones. Savolainen [Savolainen, 2011] estudió la medida en que los blogs se utilizan como foros interactivos en los que la gente puede pedir y compartir información. Diferentemente del enfoque de este trabajo, el autor sólo consideró las cuatro reacciones de la categorización IPA: reacciones positivas y negativas, y hacer y responder preguntas. Experimentos en ocho blogs revelaron que las reacciones emocionales exhibidas por los contribuyentes del blog eran predominantemente positivas, lo que sugiere que los blogs son particularmente fuertes en la construcción de comunidades de apoyo de interés. Además, el estudio reveló que las interacciones se produjeron principalmente a través de la información y la opinión, mientras que el papel de hacer preguntas fue secundario.

Hay algunos trabajos relacionados que se basan en las clasificaciones IPA, pero, de manera diferente al enfoque de este trabajo, las clasificaciones son hechas manualmente por expertos humanos. Kim [Kim et al., 2007], por ejemplo, clasificó manualmente segmentos de grabaciones de video en las categorías IPA. Después de que el comportamiento grupal fue manualmente clasificado, se analizó el efecto de la práctica (de creatividad grupal), para luego investigar las interacciones desde la perspectiva personal en grupos de estudiantes. Löfstrand y Zakrisson [Löfstrand et al., 2014] siguieron un enfoque similar. Los autores mapearon manualmente las grabaciones de vídeo en categorías IPA con el objetivo de identificar el comportamiento competitivo y no competitivo. Más recientemente, Delic [Delic et al., 2016] presentó un método para observar y medir la evolución de las preferencias y acciones de los usuarios en una tarea de toma de decisiones en turismo. Los autores usaron grabaciones de audio de discusiones de grupo y manualmente identificaron y categorizaron cada unidad de interacción de acuerdo a las doce categorías de comportamiento.

Buscar reducir el esfuerzo necesario para clasificar la dinámica de grupo en las categorías de IPA introdujeron el concepto de abridores de oraciones. Los abridores de oraciones predefinen un conjunto limitado de oraciones que pueden iniciar un acto de comunicación. Cada abridor de oraciones se empareja directamente a una categoría IPA. Enfoques como AcademicTalk [McAlister et al., 2004], Group Leader Tutor [Israel et al., 2007] y EPSILON [Goodman et al., 2005] son ejemplos del uso de este enfoque. Clasificadores basados en abridores de oraciones pueden sufrir de un alto índice de error. Por ejemplo, Israel y Aiken [Israel et al., 2007] informan de un error de clasificación del 25%. Encontraron que esta alta tasa de error se puede atribuir al hecho de que el contenido de muchas interacciones no se relaciona con el abridor de oraciones seleccionado por los usuarios. Por ejemplo, el abridor de oraciones "Alternativamente" seguido del texto "pregunte al profesor porque no estoy seguro sobre esta solución" podría no conducir a una clasificación correcta. Para mejorar la precisión de la clasificación, la versión más reciente de Group Leader Tutor [Israel et al., 2007] también considera un conjunto de palabras clave que deben estar presentes en la parte de texto libre de las interacciones para confirmar que el abridor de oraciones fue seleccionado correctamente. AcademicTalk utiliza oraciones abridoras en combinación con reglas que especifican el tipo de respuestas apropiado. Más recientemente, Mansilla [Mansilla et al., 2014] también usa abridores de oraciones en un entorno de Aprendizaje Colaborativo Soportado por Computadoras (Computer-Supported Collaborative Learning - CSCL) para detectar problemas colaborativos. Los estudiantes

interactúan en una interfaz parecida a un foro, limitada por abridores de oraciones para crear nuevos mensajes. Basándose en el análisis realizado por un sistema multi-agente, el maestro es alertado cuando se detectan problemas. Entre los aspectos positivos de los abridores de oraciones, se destaca el hecho de que permiten una fácil identificación de las intenciones comunicativas, sin necesidad de utilizar ninguna técnica de procesamiento del lenguaje natural. Además, los usuarios pueden ser más críticos en las discusiones, ya que los abridores de oraciones dan "permiso" para desafiar otras contribuciones, dejando de lado los aspectos de cortesía. Por otro lado, el discurso está restringido ya que los usuarios necesitan seleccionar de un conjunto predefinido de oraciones para iniciar cada comunicación. Es posible que los usuarios no encuentren un abridor de oraciones que se adecue a la idea que desean expresar, dando lugar al mal uso de las frases o a la omisión de la interacción.

En cuanto a la clasificación de las interacciones de texto sin formato en las categorías de IPA, el trabajo de Zhang y Zhang [Zhang et al., 2005] clasificó el contenido de los mensajes de grupo de noticias (medios asíncronos) en categorías IPA con un esquema de clasificación de una fase utilizando SVM. Los mensajes de texto fueron preprocesados mediante la eliminación de stopwords y la ejecución de stemming. Los caracteres especiales y emoticonos también fueron considerados como parte de los mensajes. Además del contenido del mensaje, los autores consideraron el tipo de mensaje, la posición en el grupo de noticias y las respuestas recibidas. A diferencia del enfoque de este trabajo, el rendimiento del clasificador se midió a nivel de reacción de la taxonomía IPA, es decir, cada oración se clasificó en cuatro categorías: pregunta, respuesta, positivo y negativo. Los autores realizaron experimentos en dos conjuntos de datos obteniendo una precisión de 85,1 / 86,6% para la categoría "pregunta", 85,1 / 91,5% de precisión para la categoría "respuesta", 70,3 / 70,7% de precisión para la categoría "positivo" y 95,9 / 100% de precisión para la categoría "negativo". Un enfoque similar se consideró en [Cincunegui et al., 2015] en un ambiente de trabajo colaborativo soportado por computadoras (Computer-supported collaborative work - CSCW) (medios sincrónicos). Las interacciones también fueron pre-procesadas mediante la eliminación de stopwords y la ejecución stemming. Los autores formaron y compararon diferentes clasificadores (SVM, Árboles de Decisión y Naive Bayes) para identificar el nivel de reacción de la taxonomía IPA (cuatro clases) y también el nivel de categoría (doce clases). Los resultados de esta investigación se mejoraron más tarde en [Cincunegui et al., 2015], mejorando el preprocesamiento de las interacciones de texto. El principal problema al clasificar las interacciones en sólo cuatro categorías correspondientes a las reacciones en lugar de las categorías identificadas por el IPA es que las reacciones no son suficientes para detectar problemas colaborativos.

2.9 Conclusión

Esta revisión de la literatura permite afirmar que la automatización del procedimiento de clasificación presentado en este informe permitirá mejorar los sistemas existentes y desarrollar asistentes de software con el objetivo de ayudar a la dinámica de grupo.

La principal diferencia de los enfoques descritos en esta sección con el enfoque presentado en este trabajo es que se propone un esquema de clasificación de tres fases para identificar mejor la categoría IPA de las interacciones de texto en bruto. En el capítulo

6 se muestra que el esquema de clasificación propuesto supera el esquema de una fase utilizado en enfoques previos al problema.

En el siguiente capítulo se verá en detalle el desarrollo de la propuesta. Se detalla la estructura de la herramienta, algunas consideraciones especiales y se explican en profundidad los enfoques de clasificación propuestos.

3. Desarrollo de la propuesta

Habiendo sentado las bases teóricas de la investigación de este trabajo, se procede a detallar el desarrollo de la propuesta, donde se hablará de la estructura de la herramienta a implementar, las herramientas que se tuvieron en cuenta para la realización del trabajo junto con las elegidas y las que se descartaron. Para finalizar el capítulo, se hablará de los enfoques de clasificación propuestos (directo y en fases).

3.1. Estructura de la herramienta a implementar

En base al problema presentado en capítulos anteriores, se comenzó con la evaluación de las acciones necesarias para poder dar con una solución. Se debe tener en cuenta que el objetivo de la tesis consiste en automatizar la detección de los *problemas o conflictos* que se puedan generar en un grupo de trabajo planteados, de acuerdo con los lineamientos propuestos por Bales en sus estudios, con la variante de que estos grupos se van a comunicar a través de internet mediante alguna plataforma de mensajería instantánea. Una vez detectado alguno de estos conflictos, se generará una alerta que será comunicada a la persona encargada de la formación y/o administración de los grupos de trabajo. Por otro lado, la tesis desarrollada también pretende crear perfiles de usuarios, utilizando el historial de las personas que fueron analizadas, y a partir de estos perfiles crear grupos de trabajo (permitiéndole elegir al usuario el tamaño de los grupos) buscando reducir la probabilidad de que surjan conflictos.

Para poder plantear una solución se deberá contar con:

- Una plataforma que permita a las personas pertenecientes al grupo comunicarse mediante mensajes instantáneos a través de internet. Esta plataforma deberá dar la opción de exportar (a través de un archivo, exponiendo una API, etc) los “chats” del grupo que serán los datos de entrada de la aplicación que de solución al problema.
- Algún mecanismo para poder leer e incorporar al sistema los datos exportados por la herramienta mencionada en el punto anterior.
- Una herramienta que permita hacer un análisis de los datos ingresados al sistema y clasificar cada una de los chats de las personas del grupo dentro de una de las categorías IPA planteadas por Bales. Las herramientas de *Data Mining* y *Machine Learning* pueden ser de mucha utilidad en este punto.
- Es conveniente que los datos que se ingresen al sistema estén pre-formateados para que el analizador los reciba de la manera más uniforme posible. Esto es debido a que el lenguaje natural es muy amplio y variado.
- También será necesario crear el perfil de cada persona para luego poder generar los grupos de trabajo donde la probabilidad de que surjan conflictos de colaboración sea reducida.
- Algún método para presentar los resultados al usuario final.

3.2. Consideraciones especiales a tener en cuenta

En esta sección se presentarán todas las herramientas elegidas para la realización del trabajo, así como también las herramientas que se tuvieron en cuenta pero, por una razón u otra, se decidió no utilizarlas.

3.2.1 Servicio de mensajería instantánea

Se elige Google Docs como el entorno para desarrollar el trabajo colaborativo. Google Docs es una herramienta en línea que cualquier persona con acceso a Internet puede utilizar libremente para escribir documentos en colaboración con otros [Oishi, 2007]. Esta herramienta proporciona varias ventajas para los usuarios: la edición en tiempo real, la creación de comentarios y notas. Los servicios de Google también incluyen otra herramienta que es una ventaja para nuestro objetivo: la disponibilidad de una herramienta de mensajería instantánea de texto (Google Hangouts), gratuita para que los participantes discutan la tarea que se está realizando. Google Hangouts permite que los chats puedan ser exportados a un archivo de texto con formato JSON, estos archivos exportados serán los archivos de entrada de la aplicación propuesta en este trabajo.

JSON (JavaScript Object Notation - Notación de Objetos de JavaScript) es una sintaxis de texto que facilita el intercambio de datos estructurados entre todos los lenguajes de programación. JSON es una sintaxis de llaves, corchetes y comas que es útil en muchos contextos, perfiles y aplicaciones. La sintaxis JSON no es una especificación de un intercambio completo de datos. El intercambio significativo de datos requiere un acuerdo entre un productor y el consumidor sobre la semántica asociada a un uso particular de la sintaxis JSON. Lo que JSON proporciona es el marco sintáctico al que se puede vincular dicha semántica [ECMA, 2017].

3.2.2 Machine Learning

Se buscó una herramienta que sea capaz de trabajar con Minería de Datos (*Data Mining*) para que la aplicación sea capaz de *entrenar* utilizando un conjunto de datos y luego poder *clasificar* otro conjunto de datos a partir del entrenamiento previo.

Aunque existe una gran variedad de herramientas de este tipo, se eligió WEKA porque está implementada en Java, es de uso libre, tiene una gran cantidad de algoritmos implementados y es ampliamente adoptada por la comunidad de minería de datos.

3.2.2.a WEKA

WEKA (Waikato Environment for Knowledge Analysis) fue desarrollada en la Universidad de Waikato en Nueva Zelanda; por sus siglas en inglés Entorno de Waikato para el Análisis del Conocimiento. El sistema está escrito en Java y distribuido bajo los términos de la Licencia Pública General de GNU. Se ejecuta en casi cualquier plataforma y ha sido probado en los sistemas operativos Linux, Windows y Macintosh [Hall et al., 2009][Frank et al., 2016]. Es una colección de algoritmos de aprendizaje de máquina para tareas de minería de datos. Los algoritmos se pueden aplicar directamente a un conjunto de datos o llamar desde su propio código Java. WEKA contiene herramientas para el

preprocesamiento de datos, clasificación, regresión, clustering, reglas de asociación y visualización. También es adecuado para desarrollar nuevos esquemas de aprendizaje automático [Dan et al., 2013].

Esta herramienta es la principal para resolver el problema planteado en este trabajo y será la encargada de clasificar las interacciones de los miembros del grupo colaborativo dentro de alguna de las categorías descritas por Bales.

Esta herramienta se utiliza de diversas maneras dentro del proyecto. Las principales funcionalidades utilizadas son las siguientes:

- **Evaluar un clasificador:** esta evaluación permite saber qué tan preciso es el clasificador para clasificar un conjunto de datos. Se realiza una evaluación teniendo en cuenta un conjunto de datos de entrenamiento y se evalúa utilizando una validación cruzada (*cross validation*) que será explicado más adelante.
- **Entrenar un clasificador:** al entrenar un clasificador se utilizará el conjunto de datos de entrenamiento completo para entrenar, se le deberá dar la cantidad de *cross validation* que se desean usar y tendrá como salida un modelo de datos, este modelo se guarda en un archivo para ser utilizado posteriormente en la clasificación.
- **Clasificar un set de datos utilizando un clasificador:** para realizar la clasificación se utiliza un set de datos para clasificar y el modelo de datos generado al entrenar un clasificador.
- **N-gram:** Un *N-grama* [Cavnar et al., 1994] es un segmento de *N-unidades* (pueden ser palabras o caracteres) de una cadena más larga. Aunque en la literatura el término puede incluir la noción de cualquier conjunto concurrente de caracteres o palabras en una cadena (por ejemplo, un *N-grama* compuesto por el primer y tercer carácter de una palabra), en este trabajo se utiliza el término para un segmento continuo de palabras solamente. Por lo general, se corta el texto en un conjunto de *N-gramas* superpuestos. Por lo tanto, la frase "Lorem ipsum dolor sit amet" estaría compuesta por los siguientes N-gramas:

2-gramas	[Lorem ipsum], [ipsum dolor], [dolor sit], [sit amet]
3-gramas	[Lorem ipsum dolor], [ipsum dolor sit], [dolor sit amet]
4-gramas	[Lorem ipsum dolor sit], [ipsum dolor sit amet]
5-gramas	[Lorem ipsum dolor sit amet]

Tabla 3.1: ejemplo de N-gramas

El beneficio clave que proporciona la adaptación basada en N-gramas deriva de su propia naturaleza: dado que cada texto se descompone en partes pequeñas, cualquier error que esté presente tiende a afectar solo a un número limitado de esas partes, dejando el resto intacto. Si se cuentan los *N-gramas* que son comunes a dos cadenas de caracteres, se obtiene una medida de su similitud que es resistente a una amplia variedad de errores textuales.

- **Cross-validation:** En la validación cruzada (*cross-validation*), se puede decidir sobre un número fijo de particiones de los datos. Suponiendo que se usan tres. Luego, los

datos se dividen en tres particiones aproximadamente iguales; cada uno a su vez se usa para las pruebas y el resto se usa para el entrenamiento. Es decir, se usan dos tercios de los datos para el entrenamiento y un tercio para las pruebas, y se repite el procedimiento tres veces para que, al final, cada instancia se haya utilizado exactamente una vez para la prueba. Esto se llama validación cruzada triple [Hall et al., 2009].

WEKA define un formato especial de archivos que serán los archivos de entrada y salida de esta herramienta. Un archivo ARFF (formato de archivo de atributo-relación) es un archivo de texto ASCII que describe una lista de instancias que comparten un conjunto de atributos. Para más información sobre el formato de archivos ARFF, vea el Apéndice III.

En la Tabla 3.2 se muestran las técnicas de minería de datos nombradas en el capítulo anterior, junto con alguna de las implementaciones que tiene WEKA para cada categoría.

Técnicas de Minería de Datos				Ejemplos de implementaciones en WEKA
Técnicas	No supervisadas	Agrupamiento	Numérico	WEKA.clusterers.SimpleKMeans.java
			Conceptual	WEKA.clusterers.Cobweb.java
			Probabilístico	WEKA.clusterers.EM.java
		Asociación	A priori	WEKA.associations.Apriori.java
		Predicción	Regresión	WEKA.classifiers.functions.LinearRegression.java WEKA.classifiers.LWR.java
			Árboles de predicción	WEKA.classifiers.m5.M5Prime.java
			Estimador de núcleos	WEKA.classifiers.KernelDensity.java
	Supervisadas	Clasificación	Tabla de decisión	WEKA.classifiers.rules.DecisionTable.java
			Árboles de decisión	WEKA.classifiers.trees.ID3.java WEKA.classifiers.trees.J48.java WEKA.classifiers.trees.DecisionStump.java
			Inducción de reglas	WEKA.classifiers.rules.OneR.java WEKA.classifiers.rules.Prism.java WEKA.classifiers.rules.PART.java
			Bayesianas	WEKA.classifiers.bayes.NaiveBayes.java WEKA.classifiers.bayes.BayesNet
			Basado en ejemplares	WEKA.classifiers.lazy.IBk WEKA.classifiers.lazy.KStar
			Redes neuronales	WEKA.classifiers.functions.neural.NeuralConnection.java

Tabla 3.2: Técnicas de minería de datos e implementación en WEKA

3.2.3 Ingreso de datos al sistema

El sistema debe ser capaz que permitirle al usuario ingresar los datos de los chats de una manera simple. Esto es posible mediante la importación de 2 tipos de archivos donde se pueden encontrar alojados los chats de los participantes del grupo colaborativo, el primero es el tipo de archivo exportado por la herramienta Google Hangouts mencionada anteriormente, estos archivos tienen extensión JSON. El otro tipo de archivo aceptado por la aplicación es el de las hojas de cálculo de Microsoft Excel (archivos XLS y XLSX). Para poder utilizar estos archivos en el sistema, es necesario leerlos para luego hacer una conversión a un tipo de archivo aceptado por WEKA, archivos ARFF.

La lectura y conversión de los archivos JSON se realiza con una herramienta para leer y mapear archivos JSON en clases, esta herramienta es `com.fasterxml.jackson.core`. Utilizando esta herramienta, se mapean los datos del

archivo ARFF en clases, se consiguen los datos que son relevantes para este trabajo y se crea un nuevo archivo ARFF. De igual manera, esta aplicación es capaz de recibir hojas de cálculo XLS o XLSX, para leer estos archivos se utiliza una herramienta llamada *Apache POI*, que permite leer hojas de cálculo y, a partir de esas hojas de cálculo, se generan archivos ARFF.

3.2.4 Procesamiento de lenguaje natural

Debido a que el idioma español es muy rico y contiene muchas variaciones para cada palabra se buscó una herramienta que pueda hacer un procesamiento sobre los textos de entrada para que WEKA reciba datos preprocesados y de una manera más uniforme. Este preprocesado busca convertir las palabras a su palabra raíz, por ejemplo: pasar a infinitivo los verbos conjugados, pasar todas las palabras al singular, poner siempre de la misma forma las fechas, etc. Esta herramienta debería ser capaz de separar los textos en oraciones y en palabras. Luego de analizar dos herramientas, FreeLing¹ y WordNet², para llevar a cabo la tarea de procesamiento del lenguaje natural, se optó por utilizar FreeLing debido a las siguientes razones::

- FreeLing tiene las anotaciones de sentidos basados en *WordNet*.
- FreeLing es desarrollada en España y uno de los principales lenguajes para los que tiene soporte es el Español y por el contrario *WordNet* está desarrollada para analizar el lenguaje Inglés y hay que hacerle modificaciones para que funcione con el Español.
- Existe una gran comunidad que utiliza esta herramienta y es simple y rápido obtener respuestas a preguntas sobre cómo utilizarla.

3.2.4.1 FreeLing

FreeLing [Padró, Stanilovsky, 2012] es una biblioteca multilingüe de procesamiento de lenguaje de código abierto que proporciona una amplia gama de analizadores para varios idiomas. Ofrece funciones de procesamiento de texto y anotación de lenguaje a los desarrolladores de aplicaciones NLP (Natural Language Processing, Procesamiento de Lenguaje Natural), lo que reduce el costo de crear esas aplicaciones. FreeLing es personalizable, extensible y tiene una fuerte orientación a las aplicaciones del mundo real en términos de velocidad y robustez. Los desarrolladores pueden usar los recursos lingüísticos predeterminados (diccionarios, léxicos, gramáticas, etc.), ampliarlos/adaptarlos a dominios específicos o, dado que la biblioteca es de código abierto, desarrollar otros nuevos para idiomas específicos o necesidades de aplicaciones especiales.

La amplia comunidad de usuarios ha ampliado los tres idiomas iniciales (inglés, español y catalán) a nueve (agregando gallego, italiano, galés, portugués, asturiano y ruso) más la variante diacrónica del español de los siglos XII al XVI [Sánchez-Marco et al., 2011]. Además, la naturaleza de código abierto del proyecto y su arquitectura modular han hecho posible incluir código de otros proyectos similares, como el módulo *Word Sense Disambiguation* basado en UKB [Aguirre, Soroa, 2009].

La versión utilizada en este trabajo (versión 4.0) provee identificación de lenguaje, tokenización, separación de sentencias, análisis morfológico, detección y clasificación NE

¹ <http://nlp.lsi.upc.edu/freeling>

² <https://wordnet.princeton.edu>

(Named Entities o entidades nombradas), reconocimiento de fechas, números, magnitudes físicas, monedas, codificación fonética, PoS tagging (part-of-speech tagging o etiquetado gramatical) basados en la propuesta de EAGLE, parseo superficial, parseo de dependencias, anotaciones de sentidos basados en WordNet, desambiguación del sentido de las palabras, etiquetado del papel semántico y resolución de correferencia.

Está diseñado para ser usado como una librería externa desde cualquier aplicación que requiera este tipo de servicios. Si la aplicación está escrita en C++ se pueden realizar llamadas nativas a la librería. Alternativamente, se proveen APIs para invocar a las funcionalidades principales de FreeLing desde programas en Perl o Python.

Las funciones principales que se utilizarán son: tokenización, separado de sentencias, analizador morfológico, conversión de palabras a la palabra raíz (si la palabra es un verbo conjugado devuelve el verbo en infinitivo), búsqueda de palabras alternativas si es que esta no se encuentra en el diccionario, etiquetado gramatical, clasificación de entidades nombradas.

3.3. Enfoques de clasificación propuestos

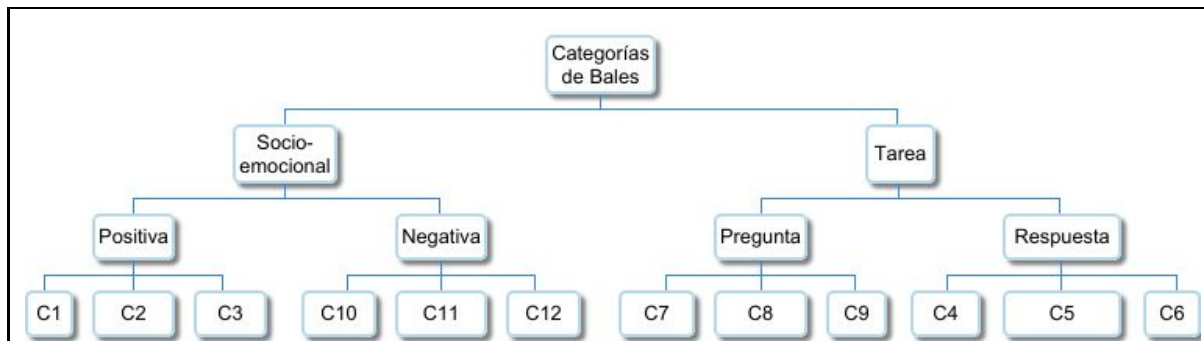
WEKA y FreeLing son las principales herramientas que se van a utilizar para el desarrollo de todo el proyecto y serán las que le den forma al diseño y desarrollo. Como se verá a continuación, la combinación y diferentes usos que se le puede dar a estas herramientas es muy variada y amplia. La solución propuesta presenta distintas modalidades para clasificar las interacciones de los participantes del grupo colaborativo dentro de las categorías IPA, a continuación se presentarán todas estas variantes. Para consultas acerca de algunos de los términos utilizados a continuación referirse al Glosario.

3.3.1 Enfoque de clasificación directa

El primer enfoque que se exploró es lo que se llamó *clasificación directa*. En el enfoque de clasificación directa, la aplicación es entrenada a partir de un archivo de entrada (conjunto de datos de entrenamiento) y luego a partir de ese entrenamiento, es capaz de clasificar otro conjunto de datos diferente al utilizado como entrenamiento. Una vez que el entrenamiento genera el modelo de datos, se recorre todo el conjunto de datos que se desea clasificar y se predice, interacción por interacción, la categoría más probable, dentro de alguna de las 12 categorías IPA definidas por Bales.

3.3.2 Enfoque de clasificación en fases utilizando un clasificador

El segundo enfoque explorado, considera el hecho de que los estudios de Bales presentan las categorías en forma de árbol (ver Figura 3.1). El enfoque de clasificación en fases, entonces, busca hacer algo similar a la hora de clasificar.



Referencia:

- C1. Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa
- C2. Muestra liberación de tensión, bromea, ríe, muestra satisfacción
- C3. Concuerda, muestra aceptación pasiva, comprende, coincide, obedece
- C4. Da sugerencia, indicación, da autonomía al otro
- C5. Da opinión, evaluación, análisis, expresa sentimiento, desea
- C6. Da orientación, información, repetición y confirmación
- C7. Pide orientación, información, repetición y confirmación
- C8. Pide opinión, evaluación, análisis, expresión de sentimientos
- C9. Pide sugerencia, indicación, posibles caminos de acción
- C10. Discrepa, muestra rechazo pasivo, formalidad, retiene recursos
- C11. Muestra tensión, pide ayuda, deja de prestar atención
- C12. Muestra antagonismo, rebaja el estado de los demás, defiende o afirma el suyo

Figura 3.1: Categorías y subcategorías consideradas en IPA (Bales).

En este segundo enfoque, la clasificación se hará en 3 fases utilizando siempre el mismo clasificador WEKA. Al usar este enfoque, primero se hace una clasificación binaria, esto quiere decir que las interacciones van a ser clasificadas dentro de dos grupos, denominados "Áreas" por Bales: "Socio-emocional" y "Tarea". Luego para cada una de las áreas, se realiza otra clasificación binaria para lo que Bales denominó "Reacción". Las interacciones clasificadas como "Socio-emocional" serán clasificadas como "Positiva" o "Negativa" y para las clasificadas como "Tarea" serán clasificadas como "Pregunta" o "Respuesta". Finalmente para cada una de las reacciones se hará una clasificación ternaria, para finalmente hacer la clasificación de cada interacción dentro de una de las 12 categorías.

3.3.3 Enfoque de clasificación en fases utilizando múltiples clasificadores

Finalmente, se desarrolló un tercer enfoque para la clasificación. Este enfoque busca darle más flexibilidad al usuario a la hora de elegir los clasificadores que desea utilizar en cada fase. Este último enfoque le permite al usuario elegir varios clasificadores, dependiendo de la fase en la que se encuentre y se lo denominó "Clasificador en fases con múltiples clasificadores".

- Fase 1: Para la clasificación de la primera fase el usuario debe elegir un clasificador, este será encargado de clasificar las interacciones entre "Socio-emocional" o "Tarea".

- Fase 2: En la segunda fase, el usuario debe elegir dos clasificadores, uno para las interacciones clasificadas como “Socio-emocional” y otro, igual o distinto, para las interacciones clasificadas como “Tarea” en la primera fase. El primer clasificador elegido por el usuario, será el encargado de clasificar las interacciones dentro de las categorías “Positiva” o “Negativa”, mientras que el segundo clasificador lo hará en las categorías “Respuestas” o “Preguntas”.
- Fase 3: En la tercera fase el usuario debe elegir entre 4 clasificadores. Uno para cada una de las clasificaciones realizadas en la segunda fase (“Positiva”, “Negativa”, “Respuestas” o “Preguntas”). El primer clasificador elegido, será el encargado de clasificar las interacciones dentro de las 3 *conductas* para la categoría “Positiva”, el segundo clasificador elegido, será el encargado de clasificar las interacciones dentro de las 3 *conductas* para la categoría “Negativa”, el tercer clasificador elegido, será el encargado de clasificar las interacciones dentro de las 3 *conductas* para la categoría “Respuestas” y el cuarto clasificador elegido, será el encargado de clasificar las interacciones dentro de las 3 *conductas* para la categoría “Preguntas”.

3.4 Conclusión

A lo largo de este capítulo se presentaron las necesidades que tiene la aplicación que se quiere desarrollar y luego se vieron las posibles herramientas que pueden ayudar a resolver estas necesidades de la aplicación. El capítulo finaliza explicando los enfoques de clasificación (directa y en fases) donde se planteó la necesidad de una clasificación con más precisión, teniendo como hipótesis que la clasificación en fases mejorará esta precisión.

En los siguientes capítulos se verá el diseño creado para resolver esta problemática, la implementación llevada a cabo y se descubrirá si la hipótesis planteada en este capítulo es cierta o no.

4. Diseño e implementación de la herramienta

En esta sección se presentará el diseño completo de la herramienta propuesta para dar solución al problema planteado en capítulos anteriores, así como también la implementación de la solución resultante. Se irá presentando la solución gradualmente, primero mostrando el diseño de manera general para luego ir entrando en detalle en cada una de sus partes.

4.1. Requerimientos funcionales y restricciones de diseño

A partir de la problemática planteada, se puede generar la lista de requerimientos funcionales para el trabajo:

- El sistema le debe permitir al usuario ingresar un dataset que contenga mensajes de un chat donde un grupo de personas colaboran para realizar un trabajo. Este será el archivo que el sistema clasificará utilizando las técnicas de *Machine Learning*.
- El sistema debe permitir ingresar un dataset para que este se entrene un clasificador utilizando técnicas de *Machine Learning*. Esta clasificación se hará dentro de las 12 diferentes categorías de conducta que se definen en el modelo de Proceso de Análisis de Interacciones (IPA de Bales).
- El sistema debe ser capaz de tomar como entrada archivos (tanto para clasificar como para entrenar) de tipo:
 - ARFF (formato de WEKA)
 - JSON (exportados de Google Hangouts)
 - XLS o XLSX (Microsoft Excel).
- El sistema debe permitir configurar los parámetros de las técnicas de *Machine Learning*.
- El sistema debe permitirle al usuario ejecutar distintas técnicas de *Machine Learning* sobre el conjunto de datos de entrada para identificar la técnica con la que se obtienen mejores resultados y de esta forma obtener el mejor clasificador.
- El sistema debe mostrar por pantalla los resultados de la ejecución de cada técnica de *Machine Learning*.
- El sistema debe emitir un log sobre los resultados de la ejecución de cada técnica de *Machine Learning*.
- El sistema debe tener como salida el/los clasificador/es que se obtienen con cada técnica de *Machine Learning*.
- El sistema debe mostrar por pantalla los resultados de los análisis de la clasificación de los grupos de personas trabajando de forma colaborativa.
- El sistema debe emitir un log de los resultados de los análisis de la clasificación de los grupos de personas trabajando de forma colaborativa.
- El sistema debe crear perfiles para cada persona que interactuó en el/los grupos de trabajo colaborativo.
- El sistema debe permitir crear nuevos grupos de trabajo donde se busque minimizar la probabilidad de que surjan problemas colaborativos a partir de los perfiles de las personas que interactuaron en los chats.

- El sistema debe permitir elegir el tamaño que se desea para la creación de los grupos de trabajo colaborativo.
- El sistema debe permitir elegir las personas que se desean utilizar para la creación de nuevos grupos de trabajo colaborativo.
- El sistema debe mostrar por pantalla los resultados de la creación de los nuevos grupos de trabajo colaborativo.
- El sistema debe emitir un log de los resultados de la creación de los nuevos grupos de trabajo colaborativo.

Además de las consideraciones mencionadas anteriormente, cabe aclarar un deseo particular sobre la implementación que fue establecido al principio del trabajo, que conforman las llamadas **“restricciones del diseño”**:

- El sistema debe poder utilizarse en computadoras con sistema operativo Windows y Mac.

Observando los requerimientos funcionales y restricciones de diseño descriptas, queda claro que se trata de una aplicación clásica de escritorio encargada de procesar secuencialmente gran volumen de datos.

4.2. Especificación de requerimientos

A continuación se describen de forma general los requerimientos a través de un diagrama de casos de uso, vea la figura 4.1.

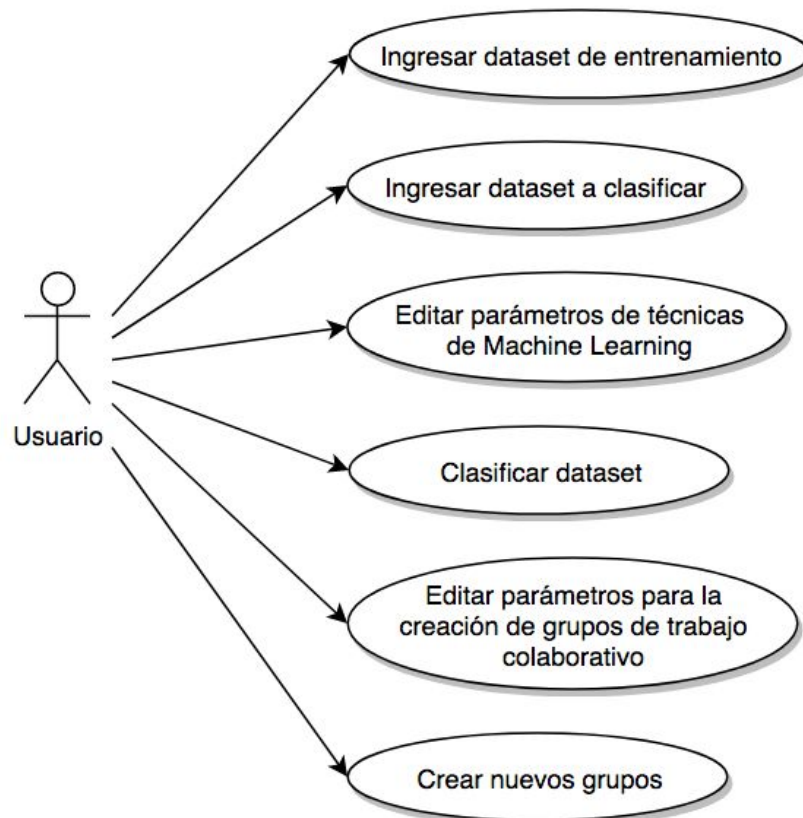


Figura 4.1: Diagrama de casos de uso

1. **Ingresar dataset de entrenamiento:** el usuario puede ingresar un dataset de entrenamiento (archivos ARFF, JSON, XLS o XLSX), estos archivos deben tener un formato particular.
2. **Ingresar dataset a clasificar:** el usuario debe ingresar el dataset que desee clasificar (archivos ARFF, JSON, XLS o XLSX), estos archivos deben tener un formato particular.
3. **Editar parámetros de técnicas de *Machine Learning*:** el usuario puede editar los parámetros que desea utilizar en las técnicas de *Machine Learning*. La configuración consta de ciertos datos relevantes a la forma en que se clasificarán los datasets. Eligiendo la forma en la que se quieren procesar los datos (directa o en fases), el clasificador que se va a utilizar y otras opciones disponibles.
4. **Clasificar dataset:** el usuario puede poner a correr el procesamiento de datos para clasificar el dataset. Al finalizar, podrá ver los resultados por pantalla y se generará un archivo con los mismos resultados.
5. **Editar parámetros para la creación de grupos de trabajo colaborativo:** el usuario puede elegir el tamaño que desea que tengan los nuevos grupos y el conjunto de personas de donde serán creados los grupos.
6. **Crear nuevos grupos:** el usuario puede crear los nuevos grupos y verá los resultados por pantalla y se generará un archivo con los mismos resultados.

4.3. Diseño de la solución

4.3.1 Model-View-Controller

El diseño arquitectónico que se utilizó para la resolución de los requerimientos funcionales es el patrón *Model-View-Controller*. Este patrón permite separar de forma clara las distintas partes de la aplicación.

- **Modelo:** este contendrá todo lo relacionado con las interacciones de las personas integrantes del grupo, la forma de convertir las interacciones a archivos ARFF (a partir de archivos de Excel o JSON exportador desde Google Hangouts), los modelos creados al entrenar un clasificador utilizando WEKA, los resultados del entrenamiento, la clasificación y los resultados del análisis de las interacciones, así como también la creación de los nuevos grupos de trabajo a partir de los perfiles de las personas.
- **Vista:** todas las vistas de la aplicación están contenidas dentro de esta sección del patrón arquitectónico. En estas vistas el usuario podrá interactuar con la aplicación y utilizándola para ingresar datos y leyendo los resultados por pantalla.
- **Controlador:** es el encargado mantener las vistas y el modelo actualizados a partir de la entrada de datos del usuario o luego que el modelo termina de realizar sus cálculos.

Se eligió este patrón arquitectónico debido a los siguientes atributos de calidad positivos:

- **Diseño modular:** permite separar claramente la vistas del modelo y del controlador. El código es claro y fácil de mantener.

- Reusabilidad: permite crear múltiples vistas para un mismo modelo.
- Modificabilidad: permite modificar las vistas sin necesidad de modificar el resto del código.

Este patrón también tiene algunos atributos negativos:

- Complejidad: se necesita un buen entendimiento en el flujo de control entre la vista, el modelo y el controlador.
- Portabilidad: inevitablemente es necesario modificar las vistas y los controladores al querer portar la aplicación.

A pesar de las desventajas que este patrón presenta, se eligió debido a que cuando se pensó en el problema que se debía resolver, se diferenciaban claramente dos funcionalidades. La primera estaba relacionada con la parte visual de la aplicación, donde el usuario iba a poder ingresar sus datos y también ver los resultados por pantalla. La otra funcionalidad claramente identificable era la que se relaciona con el procesamiento de los datos de entrada para brindar un resultado. Esta diferenciación de las funcionalidades llevó a pensar en el patrón MVC y decidir usarlo.

Model-View-Controller (MVC)

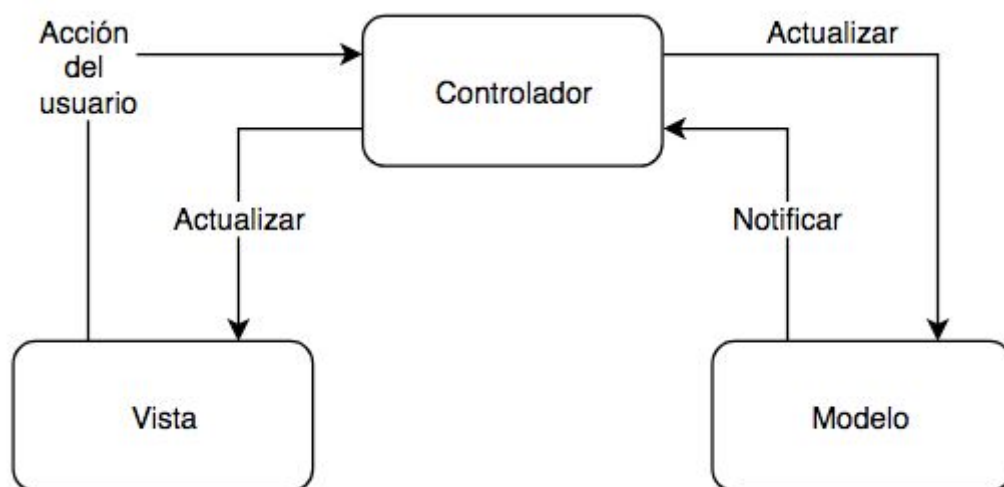


Figura 4.2: Patrón arquitectónico *Model-View-Controller*

Según Buschmann [Buschmann et al., 2001] el patrón de arquitectura *Model-View-Controller* (MVC) divide una aplicación interactiva en tres componentes, figura 4.2. El *modelo* contiene la funcionalidad y los datos básicos. Las *vistas* muestran información al usuario. Los *controladores* manejan la entrada del usuario. Las *vistas* y los *controladores* juntos comprenden la interfaz de usuario. Un mecanismo de propagación de cambios garantiza la coherencia entre la interfaz de usuario y el *modelo*.

- El componente *modelo* encapsula los datos y la funcionalidad del núcleo. El *modelo* es independiente de representaciones de salida específicas o comportamiento de entrada.

- Los componentes *vista* muestran información al usuario. Una *vista* obtiene los datos del *modelo*. Puede haber múltiples *vistas* para un *modelo*. Cada *vista* tiene un componente *controlador* asociado.
- Los *controladores* reciben una entrada, generalmente como eventos. Los eventos se traducen en solicitudes de servicio para el *modelo* o la *vista*. El usuario interactúa con el sistema únicamente a través de *controladores*.

La separación del *modelo* de los componentes *vista* y *controlador* permite múltiples *vistas* del mismo *modelo*. Si el usuario cambia el *modelo* a través del *controlador* de una *vista*, todas las demás *vistas* que dependen de estos datos deben reflejar los cambios. El *modelo* por lo tanto notifica a todas las *vistas* cada vez que sus datos cambian. Las *vistas* a su vez recuperan datos nuevos del *modelo* y actualizan la información mostrada.

Con el esquema de *Model-View-Controller* en mente, se pueden comenzar a distinguir las responsabilidades de cada componente con un primer diagrama de componentes, figura 4.3:

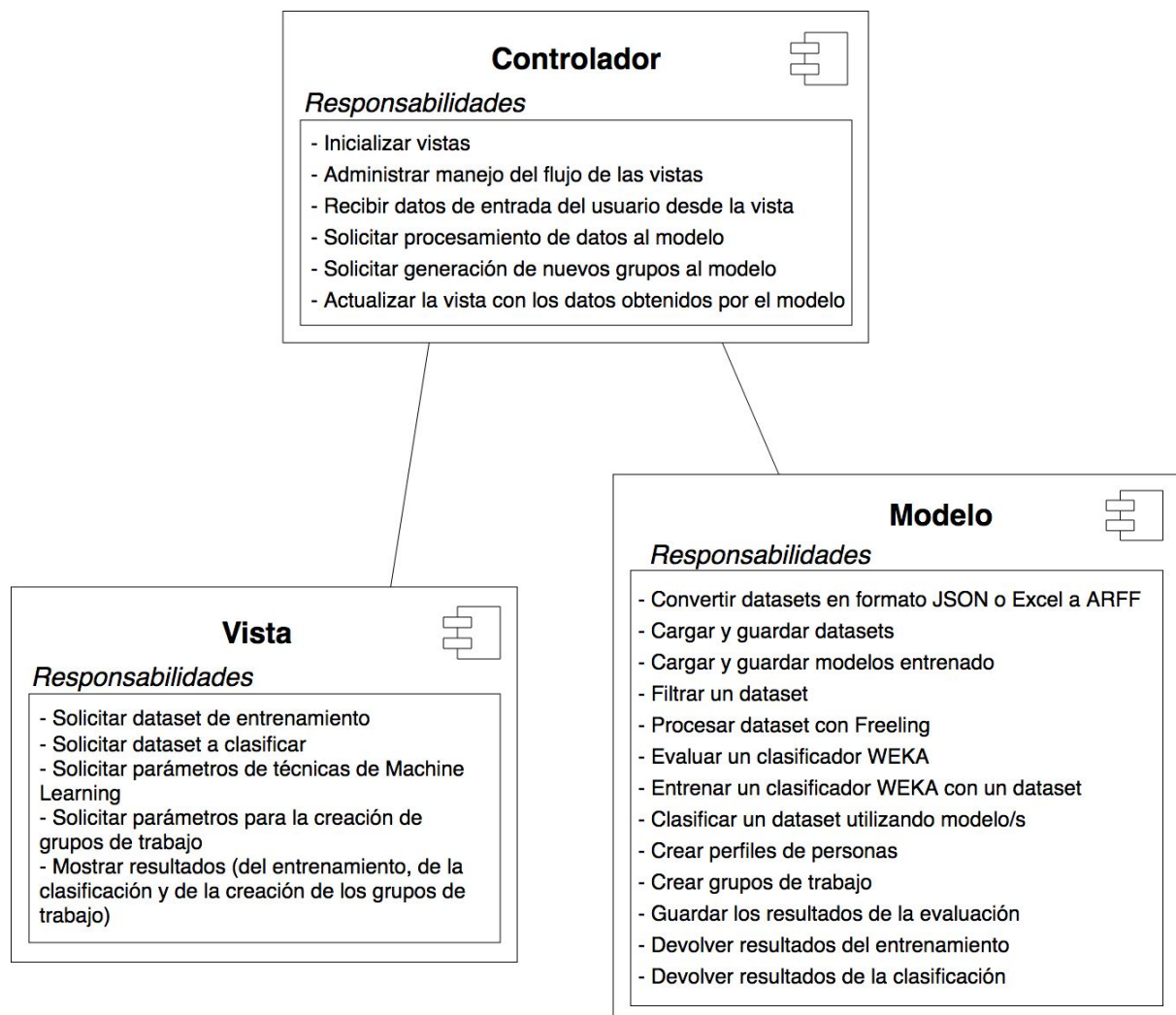


Figura 4.3: Diagrama de componentes

Teniendo en cuenta los requerimientos funcionales, las restricciones y el diseño arquitectónico elegido, se comenzó a pensar en qué lenguaje de programación debería utilizarse para llevar a cabo este trabajo. También se tuvo en cuenta las herramientas disponibles en el mercado que se deberían utilizar y los conocimientos en lenguajes de programación ya adquiridos. El lenguaje de programación elegido es Java, ya que resuelve la restricción de diseño “El sistema debe poder utilizarse en computadoras con sistema operativo Windows y Mac”, las herramientas que se estudiaron para resolver este problema son compatibles con Java (tanto WEKA como FreeLing) y es el lenguaje mejor conocido por quien escribe esta tesis.

A continuación se describen cómo se aplican las tecnologías en cada uno de los componentes.

4.3.1.a Modelo

Este es el componente de más importancia dentro de la aplicación. Es el encargado de todo el proceso de datos, desde la conversión de los archivos de entrada a un formato necesario para el trabajo de la información, evaluación y entrenamiento de los modelos de Machine Learning, clasificación de los datasets, filtrado de datos, creación de perfiles, armado de nuevos grupos de trabajo colaborativo entre otros. Dentro de este componente se hará uso de las herramientas mencionadas anteriormente como FreeLing y WEKA, este componente también deberá contar con funcionalidad para la lectura y manejo de los archivos JSON exportados de Google Hangouts, todo lo relacionado con el procesamiento de los datos y la creación de los perfiles de usuario y creación de nuevos grupos de trabajo colaborativo. Por estas razones, este componente será el que contenga la mayoría de las clases de la aplicación, la que mayor tiempo y diseño se dedicó y en la que más se va a hacer incapié a la hora de presentar el diseño.

Como se puede ver en la Figura 4.4, ya se comenzó a separar las funcionalidades mencionadas en módulos más pequeños.

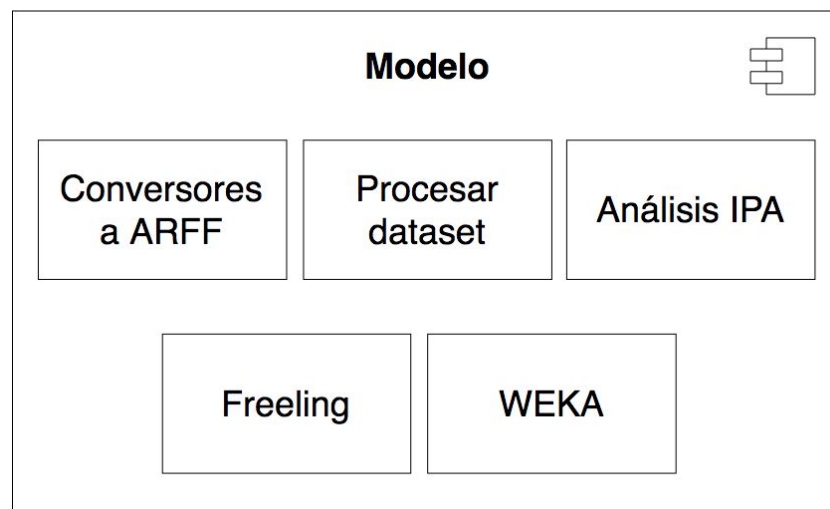


Figura 4.4: Diagrama de componentes - Modelo

4.3.1.b Vista

Para la interfaz gráfica se utilizó la biblioteca gráfica de **Java** conocida como **Swing**. Esta biblioteca cuenta con muchos elementos (como campos de texto, botones, menús, pestañas, etc.) para incluir en la interfaz gráfica para que combinándolos, den forma a la aplicación deseada.

Teniendo en cuenta la naturaleza de la aplicación que se desea realizar, se considera que el componente *Vista* es el de menor importancia dentro del patrón MVC, por lo tanto es el componente con menor funcionalidad y en el que menor tiempo y esfuerzo se invirtió.

Como se puede ver en la Figura 4.5, dentro del módulo *Vista* se tendrán algunas vistas que se encargarán de implementar la funcionalidad buscada.

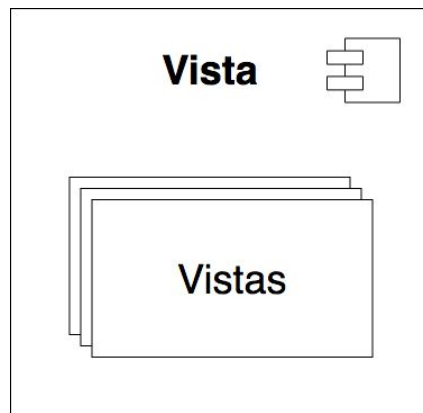


Figura 4.5: Diagrama de componentes - Vista

4.3.1.c Controlador

El controlador es el encargado de comunicar a las vistas de la aplicación con el modelos y viceversa. Será quien lleve los mensajes de un lado al otro haciendo que la experiencia del usuario sea la mejor posible. Este componente contendrá mucha lógica relacionada al manejo de las vistas y la interacción con el modelo, es por eso que en su interior utilizará a las vistas y al modelo, para esto tendrá una instancia de cada uno.

Como se puede ver en la Figura 4.6, toda la funcionalidad será contenida en un módulo.

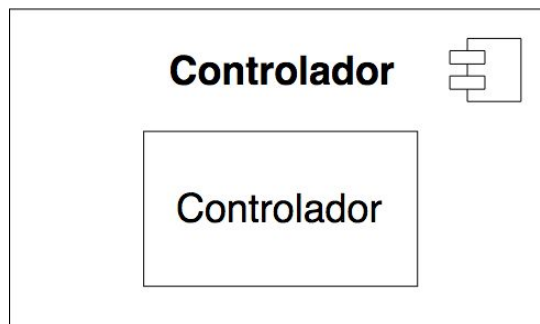


Figura 4.6: Diagrama de componentes - Controlador

4.3.2 Diagrama de clases detallado

Teniendo en cuenta los diagramas de componentes mencionados en las secciones anteriores, se hizo el diseño de clases que serán necesarias para la implementación de la aplicación, vea la Figura 4.7, este diagrama de clases está simplificado (se quitaron varias clases) para fines prácticos y se aprecie el diseño más claramente. En el diagrama se puede ver cómo dentro de cada una de los módulos de la Figura 4.4 se agregaron una o más clases. En las secciones siguientes se explicará cada parte de la aplicación en más detalle.

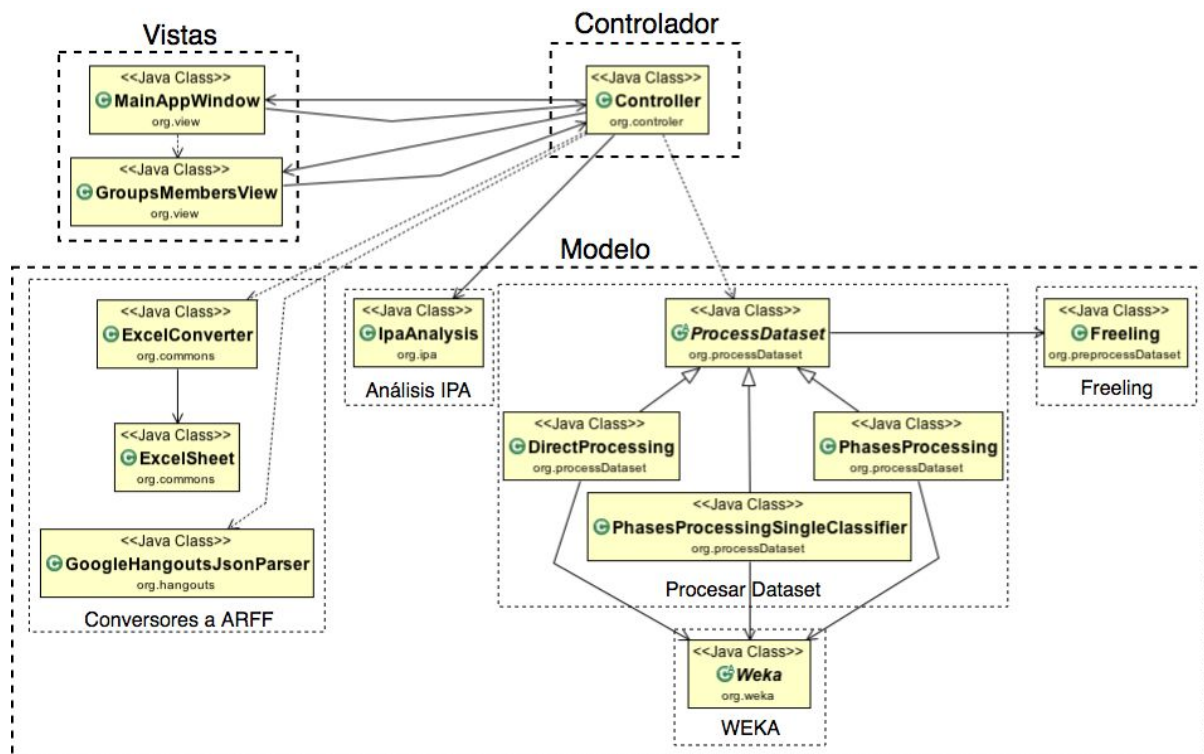


Figura 4.7: Diagrama de clases completo (simplificado) de la aplicación

4.3.3 Diseño del módulo *Conversores a ARFF*

Como se describe en los requerimientos funcionales, el sistema debe ser capaz que recibir archivos en 3 formatos: ARFF, JSON y Excel (XLS o XLSX). Para los archivos con extensión ARFF no es necesario hacer nada, ya que es el tipo de archivos aceptado por WEKA, que será quien trabaje con este tipo de archivos. Para los otros dos formatos es necesario hacer una conversión al formato ARFF.

4.3.3.a Google Hangouts JSON

Este tipo de archivos contienen mucha información y esta información está ordenada de una manera específica definida por Google Hangouts, pero lo que interesa son unos pocos datos de este JSON. Para poder utilizar la información contenida dentro de los archivos JSON en la aplicación, es necesario crear una estructura de clases que mapee el formato que tiene el archivo. Una vez creada esta estructura de clases, y utilizando un set de herramientas llamadas *Jackson Core*, *Jackson Annotations* y *Jackson Databind* es posible volcar el contenido de estos archivos JSON en la estructura de clases creada. No toda la información contenida en estos archivos JSON es necesaria, para este caso, solo se buscan identificar las distintas conversaciones (que corresponden a grupos de trabajo distintos), el nombre de la persona que escribió un mensaje y el mensaje en sí. Una vez obtenida toda la información necesaria del archivo JSON se creará un archivo ARFF con el formato necesario para que WEKA utilice. Este formato es el que se puede ver en la Tabla G.2 en el glosario.

En la Figura 4.8, donde se eliminaron algunas de las flechas de dependencias para la claridad del diagrama, se puede ver el diagrama de clases del conversor de archivos JSON a ARFF, llamado *GoogleHangoutsJsonParser*.

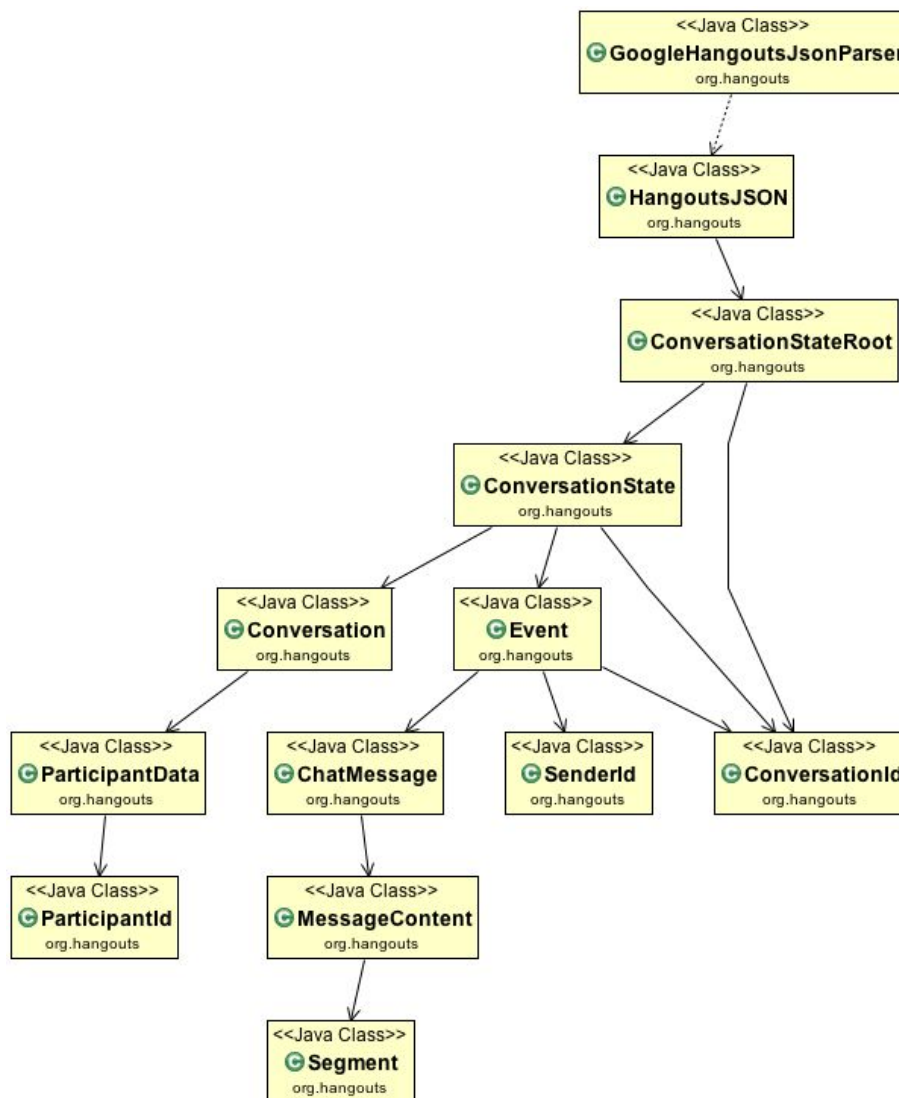


Figura 4.8: Diagrama de clases del parser de archivos JSON a archivos ARFF

Como se puede ver en el diagrama de la Figura 4.8, son muchas las clases involucradas en la conversión de los archivos JSON en archivos ARFF, de todos modos, la mayoría de ellas sirven para volcar los datos del archivo JSON para luego poder acceder a los datos que interesan para este trabajo (la conversación, el mensaje y el nombre de quien lo escribió). El procesamiento importante sucede dentro de la clase *GoogleHangoutsJsonParser*, que es la encargada de volcar el archivo JSON pasado por parámetro en las clases correspondientes, genera un archivo ARFF para cada conversación y guarda cada archivo ARFF por separado. En el diagrama de la Figura 4.9 se pueden ver en detalle las clases *GoogleHangoutsJsonParser* y *HangoutsJSON* con sus métodos. Tanto la clase *HangoutsJSON* como el resto de las que se usan para el volcado de los datos del archivo JSON son clases que contienen métodos llamados *getters* y *setters* que son métodos para obtener y establecer datos en la clase respectivamente.

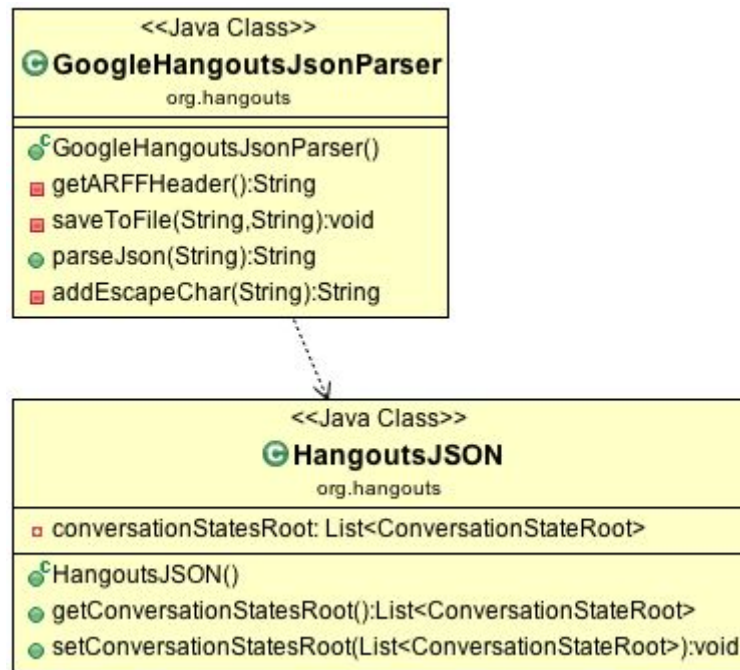


Figura 4.9: Diagrama de las clases *GoogleHangoutsJsonParser* y *HangoutsJSON*.

4.3.3.b Conversor Microsoft Excel

La conversión de archivos de tipo Excel a ARFF se hace de manera similar a la conversión de archivos JSON a ARFF. Debido a que la lectura de los archivos de Excel es más simple, la estructura de clases para representar la funcionalidad también es más simple. Para esta conversión, se lee el archivo Excel y utilizando una herramienta llamada *Apache POI* se obtiene en un objeto todo el contenido del archivo Excel. Esta herramienta, llamada *Apache POI*, es una API de Java para acceder a los archivos con formato Microsoft (entre ellos, Microsoft Excel). En el diagrama de la Figura 4.10 se puede ver la simplicidad de este conversor, donde solo con pasar por parámetro la ruta del archivo que se desea convertir, hará todo el trabajo. Esto es, volcar los datos del archivo Excel en un objeto (utilizando la *Apache POI*) y recorrer todas las hojas del archivo, de cada hoja buscará los datos que son relevantes, estos datos con el nombre y el mensaje. Una vez finalizada la búsqueda de los datos en las hojas del Excel se genera un archivo ARFF para cada hoja y guarda cada archivo ARFF por separado.

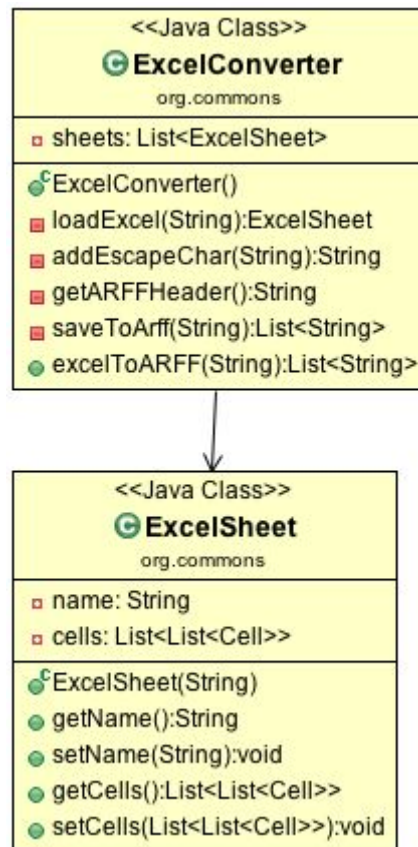


Figura 4.10: Diagrama de clases del conversor de archivos Microsoft Excel a ARFF.

4.3.4 Diseño del módulo *WEKA*

Se procede a explicar el diseño de una de las principales herramientas que utiliza la aplicación, WEKA. Como se dijo anteriormente, WEKA es una herramienta de *Machine Learning*, será la encargada de hacer todo el trabajo de la evaluación de un clasificador WEKA, entrenamiento de un dataset y clasificación de un dataset entre otras cosas. Para poder realizar todas estas tareas, se comenzó con lo principal que se necesitaba para hacer funcionar un clasificador WEKA y con el tiempo se le fueron agregando más funcionalidades y complejidad. Al comenzar a desarrollar esta sección, se creó una clase llamada *Weka*. Esta sería la encargada de cargar un dataset (ya sea para evaluar, entrenar o clasificar); guardar un dataset (ya clasificado o luego de un procesamiento de sus datos); evaluar un clasificador WEKA, entrenar un clasificador WEKA y clasificar un dataset. Una vez que se logró hacer funcionar lo mencionado anteriormente, se comenzó a agregar más funcionalidad como poder guardar un modelo ya entrenado y también poder cargarlo para utilizarlo. También se buscó que haya flexibilidad a la hora de utilizar clasificadores WEKA, al comenzar el desarrollo y hizo funcionar solo con un clasificador (J48) y luego se fueron agregando varios más. Para agregar estos clasificadores se le hicieron algunas modificaciones a la clase *Weka* y ésta se convirtió en una clase *Abstracta* y cada uno de los clasificadores que se desearan agregar, extenderían esta clase abstracta. De esta forma se puede reutilizar código y hacer uso del polimorfismo que presentan los lenguajes de programación orientada a objetos. Luego de trabajar con varios clasificadores WEKA, estudiar cada uno para ver si era posible que trabajen para cumplir el objetivo, se consiguió tener 13 clasificadores distintos para poder elegir a la hora de clasificar un dataset, cada

uno con distintas formas de trabajar con los datos mediante *Machine Learning*. En el diagrama de la Figura 4.11 se puede ver la clase *Weka* junto a todas las clases que heredan de ella.

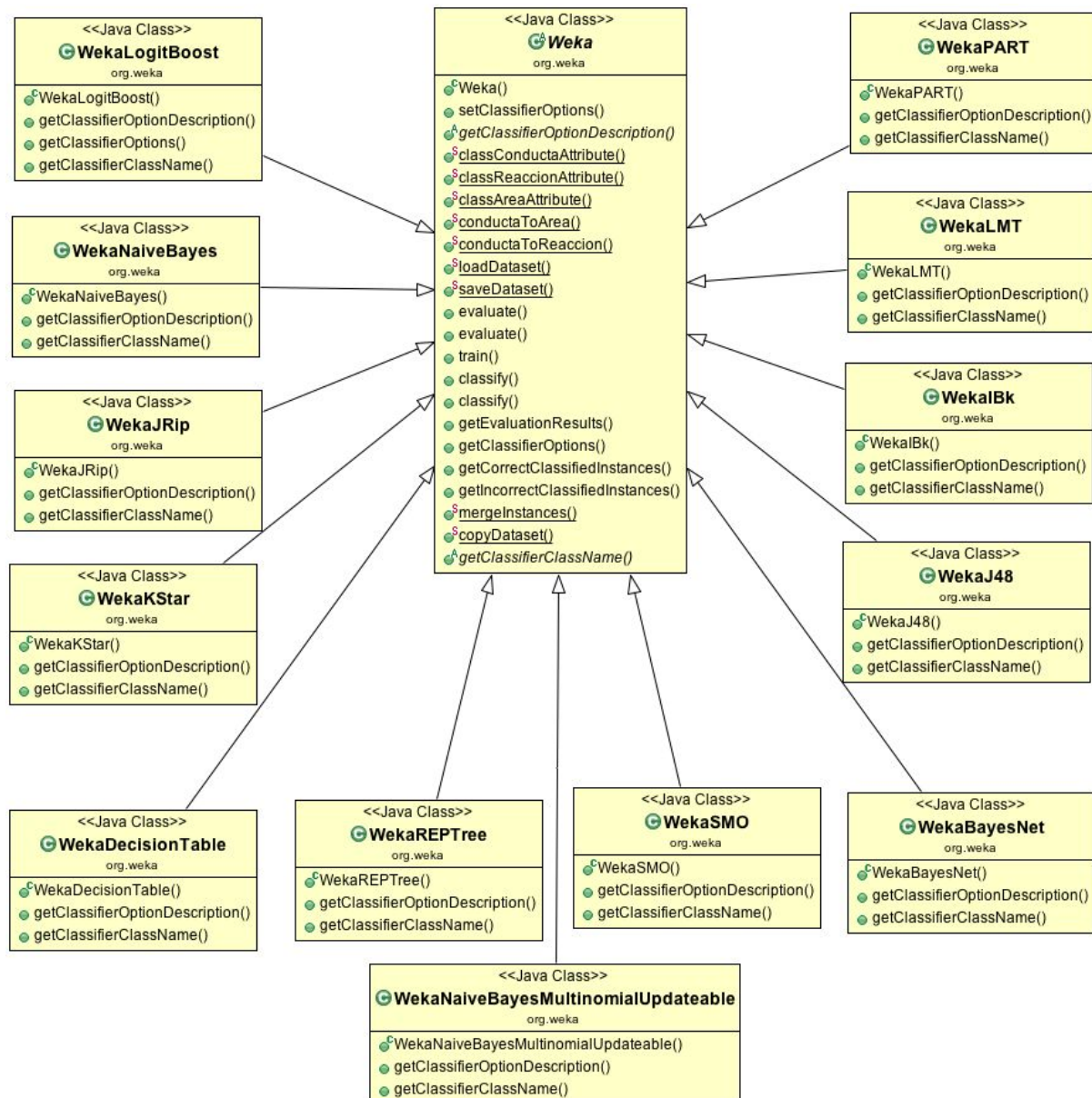


Figura 4.11: Diagrama de clases de la clase *Weka* y las clases que la extienden

Estas 13 clases que heredan de *Weka*, heredan todo su funcionamiento y al mismo tiempo implementan ciertos métodos que son abstractos, y por lo tanto no tienen implementación en la clase *Weka*. Estos métodos son solo dos, que permiten obtener las opciones por defecto de cada clasificador y el nombre de la clase (que se utilizará para la presentación de los resultados). Pero lo más interesante en estas clases “hijas”, está en su constructor, que es donde se crea el clasificador que se va a utilizar. Al crear el clasificador, todas las operaciones que se realicen en *Weka* utilizándolo, serán hechas a través de este. En la siguiente Figura 4.12 se puede ver cómo interactúan las clases *Weka*, sus hijas (para mejor visualización, se agregaron solo 2 clases hijas) y las clases propias de WEKA.

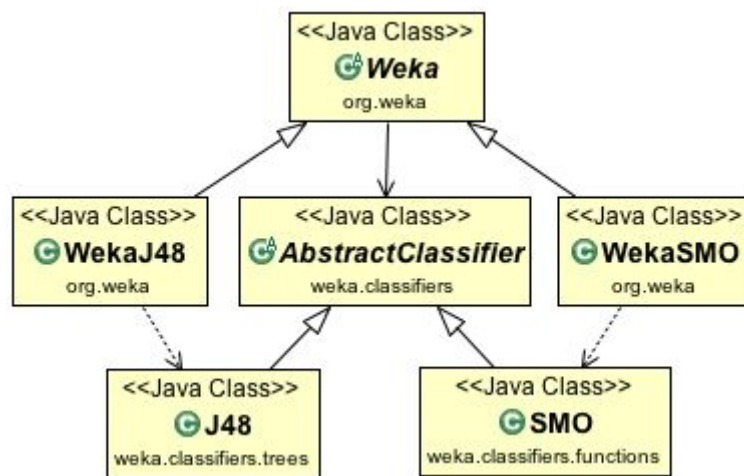


Figura 4.12: Diagrama de clases de la clase Weka, algunas de sus hijas y clases propias de WEKA.

De la misma manera, muchas clases propias de WEKA interactúan con nuestra clase *Weka*. En la Figura 4.13 se puede ver el diagrama de todas las clases propias de la API de WEKA que son utilizadas.

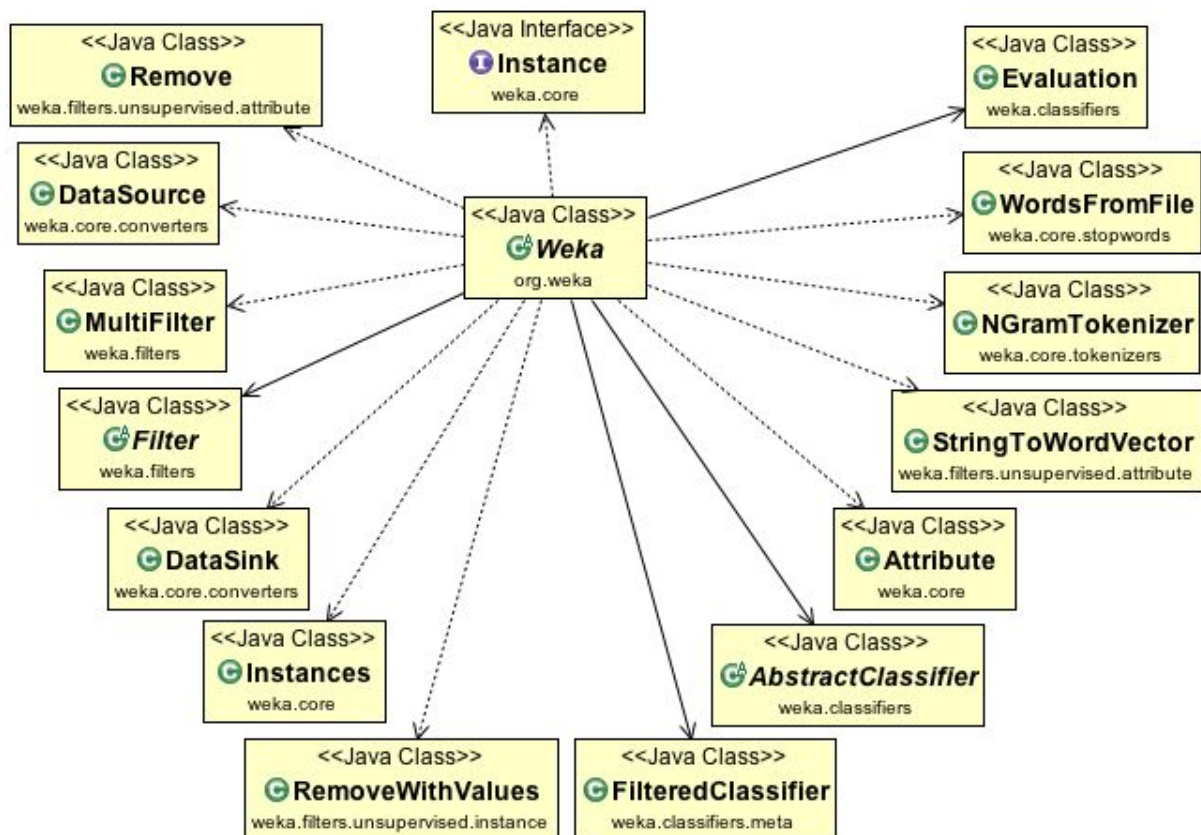


Figura 4.13: Diagrama de clases de las clases propias de WEKA utilizadas por nuestra aplicación

4.3.5 Diseño del módulo *FreeLing*

FreeLing es otra de las herramientas importantes utilizadas por el sistema. Es una herramienta de procesamiento de lenguaje natural con un poder enorme, del cual, para este trabajo sólo se usarán algunas de sus funciones. El diseño para el uso de esta herramienta es bastante simple, véase la Figura 4.14

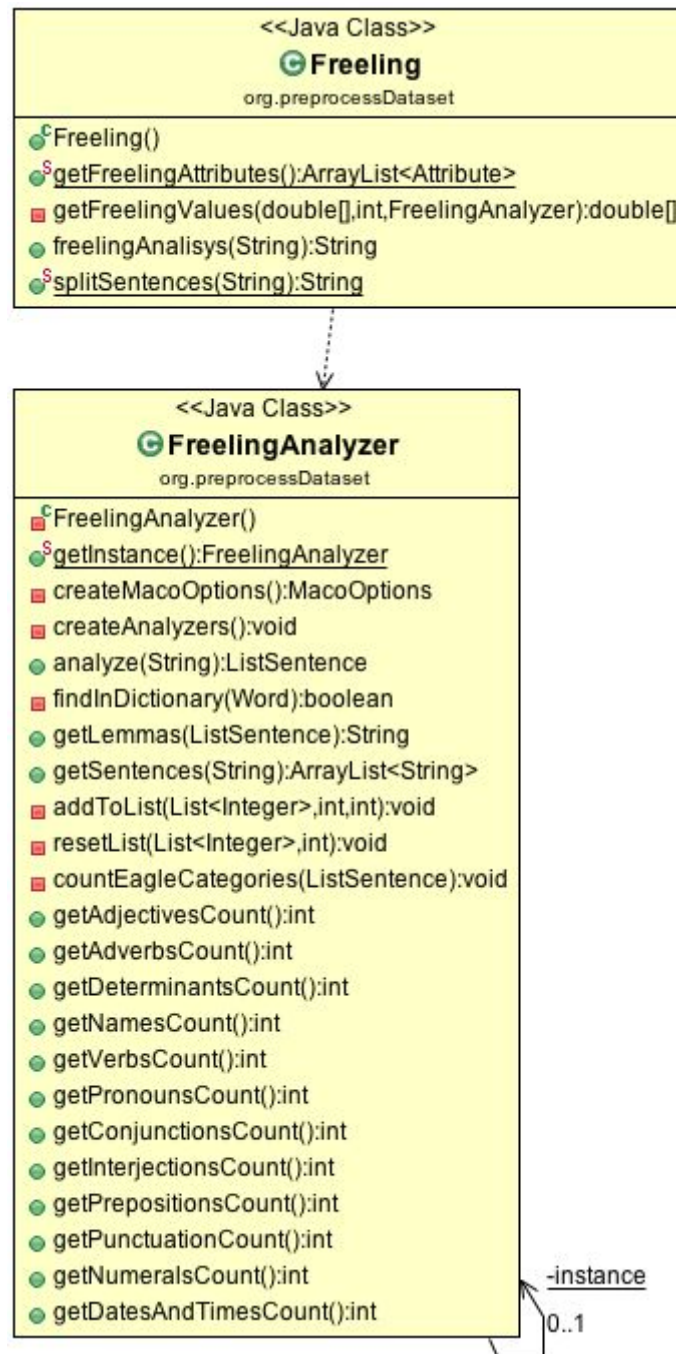


Figura 4.14: Diagrama de clases de las clases creadas para manejar FreeLing.

Como se puede ver en el diagrama de clases de la Figura 4.14, son solo dos clases, una de ellas es la llamada *FreeLing*, que es la clase que será utilizada por el resto de las

clases del sistema para hacer el procesamiento de los datos, la segunda clase (que es una instancia de la clase *FreeLing*), llamada *FreelingAnalyzer*, es la que se encarga de utilizar las clases de la herramienta FreeLing (no confundir con la clase *Freeling*). En el diagrama también se puede apreciar que la clase *FreelingAnalyzer* tiene una instancia de sí misma, esto es porque se la diseñó para que sea un *Singleton* para asegurarse de que una clase sólo tenga una instancia y proporcione un punto de acceso global a ella. Una variable global hace que un objeto sea accesible, pero no evita que cree instancias de múltiples objetos. Una mejor solución es hacer que la clase sea la responsable de realizar un seguimiento de su única instancia. La clase puede garantizar que no se pueda crear ninguna otra instancia (interceptando solicitudes para crear objetos nuevos), y puede proporcionar una forma de acceder a la instancia. Este es el patrón de Singleton [Gamma et al., 1994].

La clase *Freeling* tiene dos métodos principales, estos son *splitSentences(...)* y *freelingAnalysis(...)*.

El método *splitSentences(...)* tiene una función bastante básica, y es la de separar las sentencias dentro de un mismo mensaje y crear un nuevo dataset. Para esto toma como entrada un dataset, recorre todas sus instancias y de cada una de ellas toma el mensaje, con cada mensaje hace el llamado a la función de FreeLing de obtener las sentencias y guarda todo en un nuevo dataset. Se genera un nuevo dataset, con los mismos mensajes solo que separando las sentencias de cada mensaje en distintas instancias del archivo ARFF.

Finalmente, el método *freelingAnalysis(...)* será el que haga el análisis de un dataset pasado por parámetro. Se recorre todo el dataset y para cada mensaje, este se analiza y se convierte cada palabra a su lema correspondiente (un *lema* es la palabra raíz de la palabra analizada, por ejemplo, si la palabra es un verbo, se convertirá en el verbo en infinitivo), finalmente se agrega un conjunto de datos que se espera que sirvan para aportar más datos al propio dataset. Estos datos representan la cantidad de veces que cada una de las categorías EAGLE (Tabla 2.4) es utilizada en el mensaje. Por ejemplo, el mensaje:

```
6, 'Abajo no te aparece quién está conectado, te aparecen quienes ya leyeron lo último que se escribió'
```

luego de pasar por el procesado de datos de FreeLing queda de la siguiente manera:

```
6, 'abajo no te aparecer quién estar conectar , te aparecer quien ya leer el último que se escribir ', 1, 3, 1, 0, 6, 6, 0, 0, 0, 1, 0, 0
```

Cada uno de los números que se ven al finalizar el mensaje corresponden las categorías EAGLE, las categorías son: Adjetivo, Adverbio, Determinante, Nombre, Verbo, Pronombre, Conjunción, Interjección, Preposición, Puntuación, Cífra, Fecha/Hora. Esto quiere decir que el mensaje mostrado más arriba contiene 1 adjetivo, 3 adverbios, 1 signo de puntuación, etc.

Cuando se dice que el método *FreelingAnalysis(...)* analiza los mensajes, se refiere a que se ejecuta una batería de métodos de la herramienta FreeLing sobre cada mensaje. A continuación se puede ver un extracto del código fuente encargado de hacer el análisis de los mensajes:

```

public ListSentence analyze(String text) {

    ListWord l = tokenizer.tokenize(text);

    ListSentence ls = splitter.split(l);

    maco.analyze(ls);
    alternatives.analyze(ls);
    probabilities.setActivateGuesser(true);
    probabilities.analyze(ls);

    hmmTagger.analyze(ls);
    ner.analyze(ls);
    nec.analyze(ls);
    chartParser.analyze(ls);

    countEagleCategories(ls);

    return ls;
}

```

Código 4.1: Analizado de un mensaje utilizando FreeLing

Primero se crea una lista de palabras (`ListWord`) obteniendo los tokens del mensaje y luego se separa en sentencias (`ListSentence`) el mensaje. Una vez que se tiene el mensaje separado en sentencias, este está listo para ser analizado por los siguientes módulos:

4.3.5.a Módulo Morfológico (`maco`)

El analizador morfológico es un meta-módulo que no realiza ningún procesamiento propio. Es solo un módulo de conveniencia para simplificar la creación de instancias y llamar a otros submódulos.

En el momento de la instanciación, recibe un objeto `MacoOptions`, que contiene información sobre qué submódulos deben crearse y qué archivos se deben usar para crearlos [Padró, Stanilovsky, 2012].

4.3.5.b Módulo Alternativas (`alternatives`)

Este módulo puede recuperar de su diccionario las entradas más similares a las palabras entrada. La similitud se calcula según una medida de distancia de edición de cadena configurable.

El *módulo alternativas* se puede crear para realizar una búsqueda directa de la palabra en un diccionario, o bien para realizar una búsqueda de la transcripción fonética de la forma en un diccionario de transcripciones fonéticas. Para el caso de esta tesis, este módulo es utilizado para la corrección de palabras mal escritas [Padró, Stanilovsky, 2012].

4.3.5.c Módulo Probabilidades (`probabilities`)

Este módulo finaliza la subcadena del análisis morfológico y tiene dos funciones: primero, asigna una probabilidad a priori a cada análisis de cada palabra. Estas

probabilidades serán necesarias para el etiquetador PoS (categorías EAGLE) más tarde. En segundo lugar, si una palabra no tiene análisis (ninguno de los módulos aplicados anteriormente tuvo éxito para analizarla), este módulo intenta adivinar cuáles son sus posibles etiquetas PoS, en función de la finalización de la palabra [Padró, Stanilovsky, 2012].

4.3.5.d Módulo Etiquetador (hmmTagger)

El etiquetador gramatical (PoS tagging) es el módulo `HmmTagger` (Hidden Markov Model Tagger, por sus siglas en inglés modelo etiquetador de Markov oculto), que es un etiquetador clásico Markoviano.

Este módulo contiene los datos estadísticos para el Modelo de Markov Oculto, más algunos datos adicionales para suavizar los valores perdidos. Probabilidades iniciales, probabilidades de transición, probabilidades léxicas, etc [Padró, Stanilovsky, 2012].

4.3.5.e Módulo (ner)

El módulo de NER simplemente detecta secuencias de palabras en mayúsculas, teniendo en cuenta algunas palabras funcionales (por ejemplo, el Banco de Inglaterra) y mayúsculas al comienzo de las oraciones.

4.3.5.f Módulo (nec)

La misión del módulo de Clasificación de Entidades Nombradas (Named Entities Classification, NEC) es asignar una clase a entidades nombradas en el texto. Es un módulo basado en *Machine Learning*, por lo que las clases pueden ser cualquier cosa que el modelo haya sido entrenado para reconocer. Cuando se clasifica, la etiqueta PoS de la palabra se cambia a la etiqueta definida en el modelo. Este módulo depende de que un módulo NER se aplique previamente. Si no se reconocen entidades, ninguna se puede clasificar. Los modelos provistos con FreeLing distinguen cuatro clases: Persona (etiqueta NP00SP0), Ubicación geográfica (NP00G00), Organización (NP00O00) y Otros (NP00V00) [Padró, Stanilovsky, 2012].

4.3.5.g Módulo Parseador (chartParser)

Este módulo enriquece cada objeto `Sentence` con un objeto `parseTree`, cuyas hojas tienen un enlace a las palabras `Sentence` [Padró, Stanilovsky, 2012].

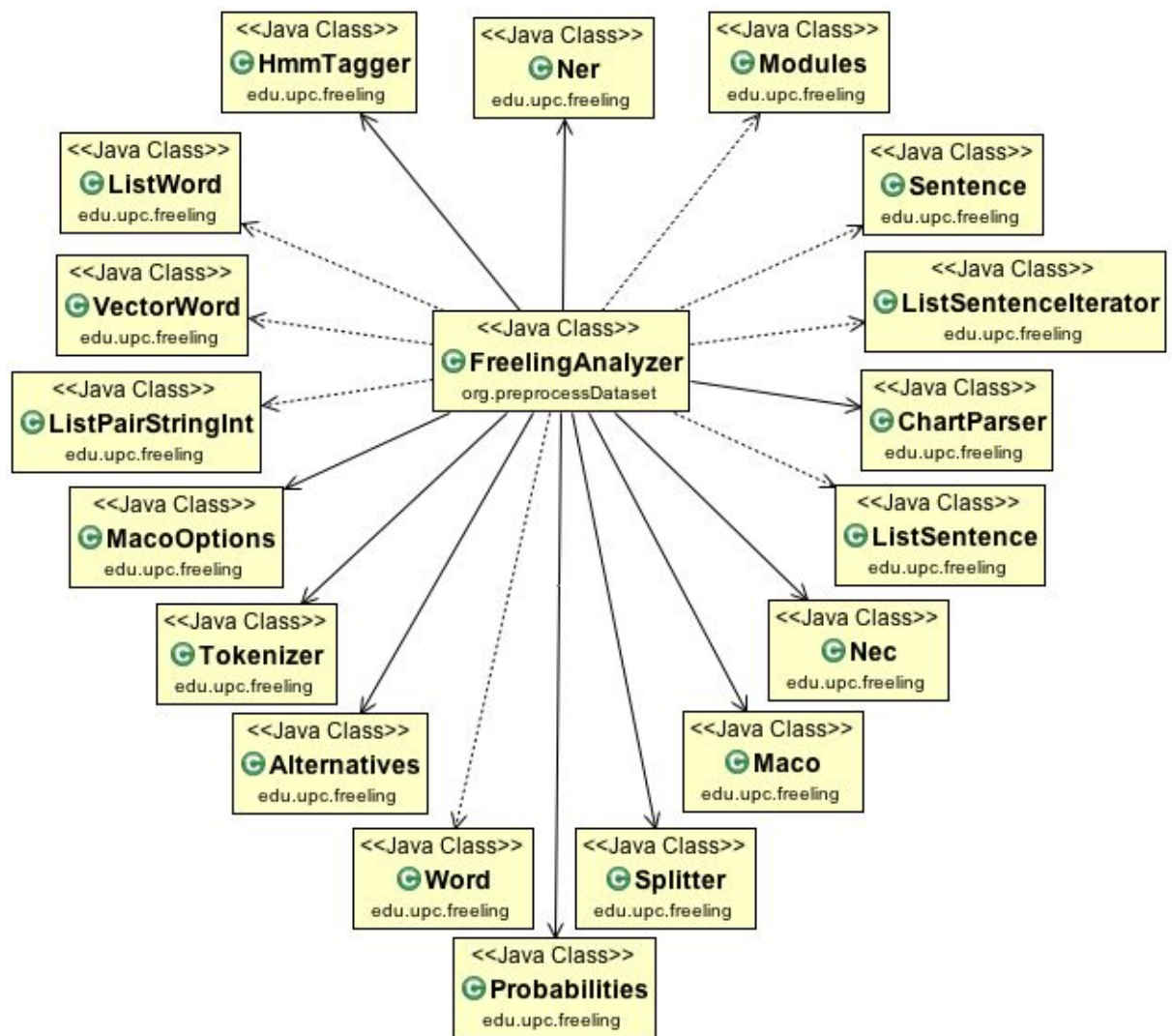


Figura 4.15: Diagrama de clases de las clases propias de FreeLing utilizadas por nuestra aplicación

En la figura 4.15 se puede ver el diagrama de todas las clases propias de la API de FreeLing que son utilizadas.

4.3.6 Diseño del módulo Procesar Dataset

El procesamiento de datos es la parte más importante de este trabajo y la sección a la que más tiempo y empeño se le dedicó. Como se mencionó anteriormente, en el capítulo 3.3, la aplicación cuenta con distintas formas para clasificar a los datasets (de forma directa o en fases).

Para comenzar con la explicación del diseño de esta funcionalidad, se presenta la clase que define los métodos principales. Esta clase es *ProcessDataset*, es una clase abstracta que define la interfase para evaluar, entrenar y clasificar datasets, así como también define métodos para obtener los resultados de la evaluación de un clasificador y los resultados de la clasificación, vea la figura 4.16.

Se presentará esta sección de la misma manera que se presentó anteriormente, mostrando el diseño del procesamiento de los datos de forma directa y luego se pasará a la explicación del procesamiento en fases.

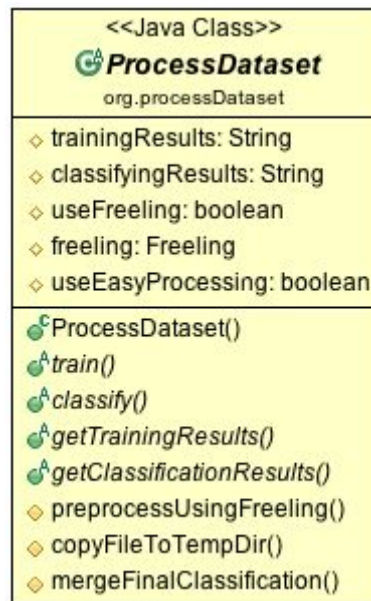


Figura 4.16: Diagrama de clases de la clase *ProcessDataset*.

4.3.6.a Directo

En la Figura 4.17 se puede ver cómo la clase *DirectProcessing* implementa los métodos abstractos de su clase padre *ProcessDataset*. Los principales métodos que implementa son *train(..)* y *classify(...)* los cuales entrenan y clasifican respectivamente. El entrenamiento y la clasificación se hacen de forma bastante directa utilizando una instancia de la clase *Weka*.

Para el entrenamiento, la aplicación no solo entrena, sino que también evalúa el modelo elegido utilizando el dataset de entrada y el método *Cross-validation folds* (vea el Glosario para más detalles), una vez finalizada la evaluación del clasificador, se procede a entrenarlo y guardar el modelo para utilizar en el futuro.

Para la clasificación, lo que se hace es recorrer todas las instancias que se desean clasificar y utilizando el modelo guardado anteriormente, se clasifican una por una las instancias y se guarda el archivo clasificado con formato ARFF.

La clase *DirectProcessing* solo se encarga de manejar en flujo de los datos, hacer un pre-proceso de los datos (por medio de FreeLing) y utilizar la funcionalidad provista por WEKA para evaluar un clasificador, entrenarlo y clasificar un nuevo dataset.

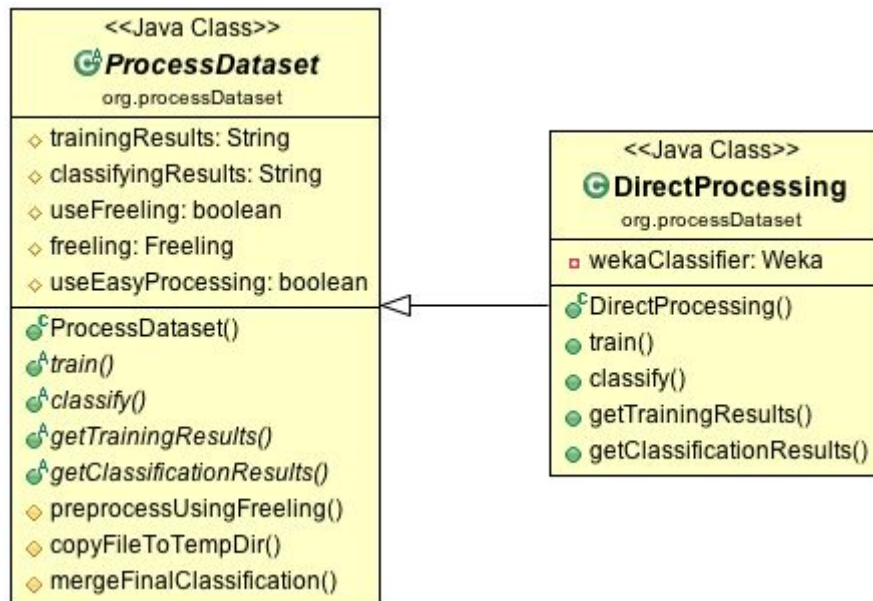


Figura 4.17: Diagrama de clases utilizadas para el procesamiento directo.

4.3.6.b Fases

En la Figura 4.18 se puede ver cómo la clase *PhasesProcessingSingleClassifier* implementa los métodos abstractos de su clase padre *ProcessDataset* y hace uso de otras clases que agrupan código común (*PhaseXProcessing* y sus clases hijas). Al igual que para el procesado directo, los principales métodos que implementa son *train(..)* y *classify(...)* los cuales entrenan y clasifican respectivamente. El entrenamiento y la clasificación se hacen en 3 pasos, uno para cada fase y para todos los pasos se utiliza la misma instancia del clasificador WEKA, es decir, el mismo clasificador. Luego, la forma de trabajo es igual a la clasificación directa.

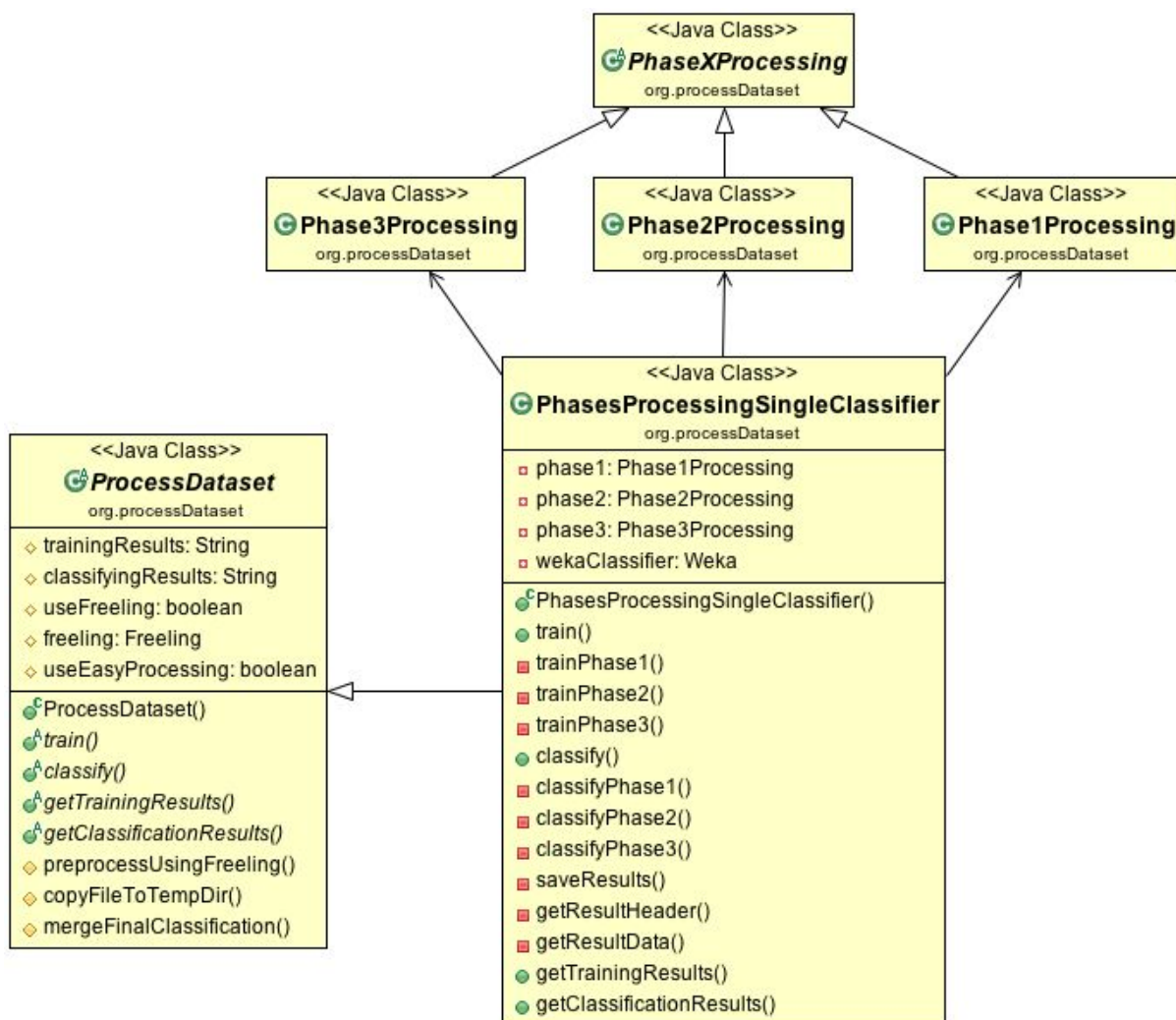


Figura 4.18: Diagrama de clases utilizadas para el procesamiento en fases (utilizando un clasificador).

4.3.6.c Fase1, Fase 2 y Fase 3

En la Figura 4.19 se puede ver cómo la clase *PhasesProcessing* implementa los métodos abstractos de su clase padre *ProcessDataset* y hace uso de otras clases que agrupan código común (*PhaseXProcessing* y sus clases hijas). Al igual que para el procesado directo y el procesamiento en fases explicado más arriba, los principales métodos que implementa son *train(..)* y *classify(...)* los cuales entrenan y clasifican respectivamente. El entrenamiento y la clasificación se hacen en 3 pasos, uno para cada fase, solo que en este caso, se cuenta con múltiples instancias del clasificador WEKA. En la primera fase se utilizará 1 clasificador, en la segunda 2 clasificadores y en la tercera 4 clasificadores, lo interesante con este método es que para el entrenamiento y la clasificación se cuentan con varios clasificadores, entonces dependiendo la fase en la que se encuentre, es necesario filtrar datos para obtener los datos que se necesitan para la fase actual y los datos necesarios para este clasificador.



Figura 4.19: Diagrama de clases utilizadas para el procesamiento en fases (utilizando varios clasificadores).

Para finalizar con esta sección, se presenta a modo ilustrativo el conjunto de las clases involucradas en el módulo *Procesar Dataset*, vea la Figura 4.20.

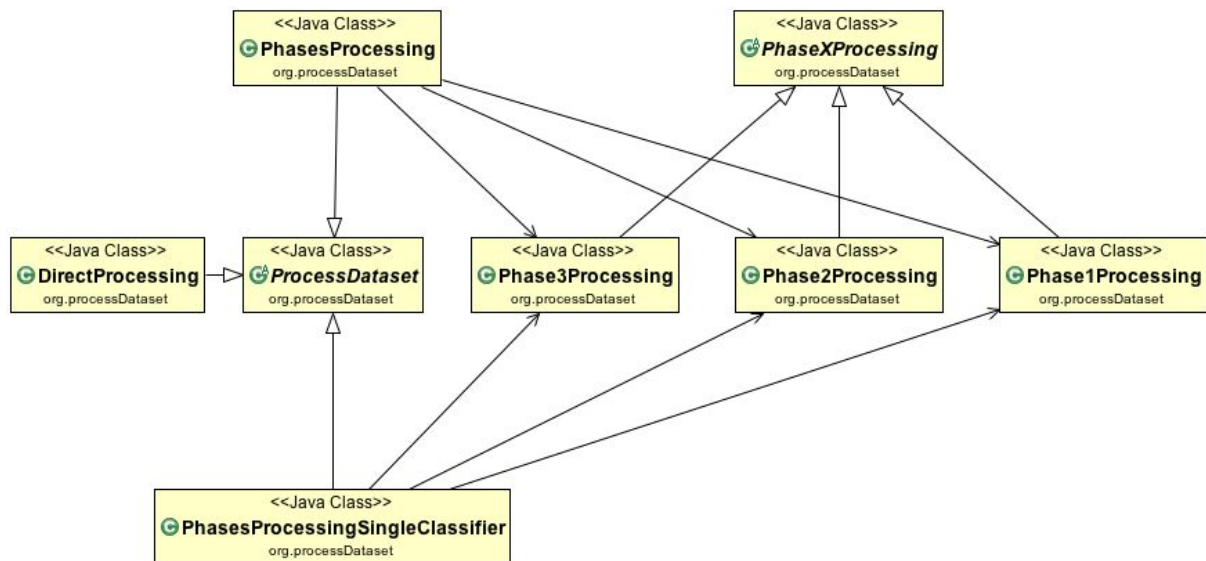


Figura 4.20: Diagrama de clases correspondientes al módulo *Procesar Dataset*.

4.3.7 Diseño del módulo *Análisis IPA*

Este módulo se encarga de resolver dos problemas. El primero es el de analizar los grupos de trabajo colaborativo, datos que se obtienen de la clasificación. El segundo es el de generar los grupos de trabajo colaborativo a partir de todos los miembros de los grupos que se clasificaron y el análisis que se hizo previamente. Como se puede ver en la Figura 4.21 hay un gran número de clases, pero su funcionalidad es simple, la mayoría de ella se utiliza para representar algún elemento de la realidad, como la información de una fila (clase *InfoRow*) o un grupo con la información necesaria para el análisis IPA (clase *IpaGroup*).

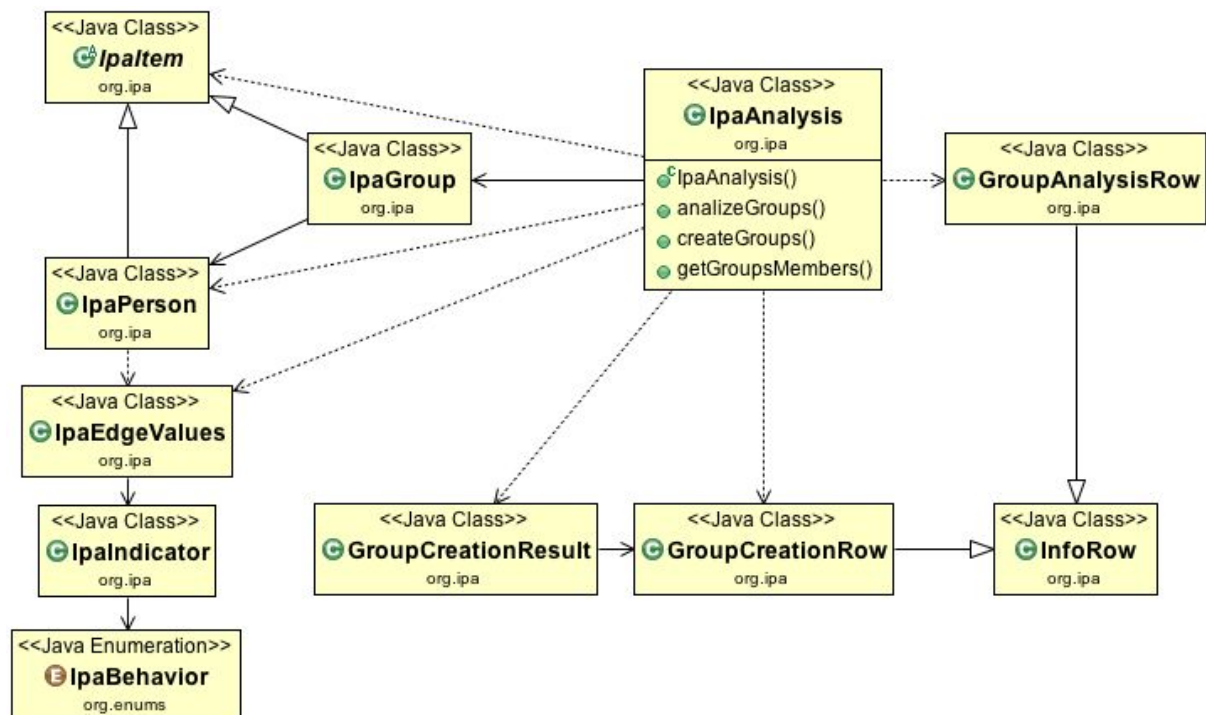


Figura 4.21: Diagrama de clases del módulo *Análisis IPA*.

4.4. Persistencia de los datos

Desde el comienzo del procesamiento de los datos hasta el final, la aplicación está constantemente trabajando con archivos, ya sea leyendo o guardándolos. La aplicación cargará en memoria datasets o modelos WEKA, también guardará datasets ya clasificados o modelos WEKA recién entrenados, así como también guardará archivos intermedios y los resultados finales. Lo que se buscó cuando se trabajó con el desarrollo de esta aplicación es que guarde la mayor cantidad de datos posibles para que quede registro de cada paso que hace la aplicación para llegar al objetivo final y que después todos esos datos se puedan utilizar para hacer un análisis del funcionamiento de los clasificadores. A continuación se hablará sobre la persistencia de los datos que hace la aplicación.

4.4.1 Persistencia de resultados

Estos datos que se quieren persistir son los más importantes para luego hacer un análisis de los resultados. Como es de esperar, se guardan los resultados de la evaluación del clasificador, la clasificación, el análisis de los grupos y la creación de grupos.

Dentro de una carpeta llamada *results* se crearán 3 carpetas llamadas *evaluation*, *labeled* y *analysis*. Dentro de estas carpetas se guardarán respectivamente los archivos resultantes de la evaluación de el/los clasificadores elegidos, los archivos resultantes de la clasificación de un dataset y dentro de la carpeta *analysis* los resultados del análisis de los grupos de trabajo y los resultados de los nuevos grupos de trabajo creados.

4.4.2 Persistencia de datos intermedios

Cuando la aplicación comienza con su trabajo, empieza a crear archivos temporales y archivos intermedios. Esto se hace para no modificar ninguno de los archivos originales que se usan de entrada al sistema.

Al entrenar, la aplicación hace una copia en una carpeta *temp* del dataset de entrada que se desea usar para el entrenamiento para después trabajar con este archivo temporal. Lo mismo pasa cuando se elige utilizar la herramienta FreeLing, se toma el dataset y se lo procesa con FreeLing, al finalizar este proceso se guarda este archivo con el mismo nombre, solo que se le agrega al final “-freeling” para poder diferenciarlo. También cuando se finaliza de entrenar un clasificador, este se guarda para poder utilizarlo a la hora de clasificar un dataset.

La conversión de archivos, tanto JSON como Excel, a ARFF genera archivos intermedios. Estos archivos se guardarán dentro de las carpetas *hangouts* y *excels* respectivamente.

A medida que la aplicación está clasificando un archivo, sobre todo cuando se elige la clasificación en fases, la aplicación va creando los archivos intermedios de la clasificación (el resultado de la clasificación de la primera fase, luego los de la segunda y para finalizar con la tercera).

4.4.3 Caso especial: *Entrenamiento Automático*

Esta es la primera vez en este informe que se habla del *Entrenamiento Automático*, esto es debido a que no es una funcionalidad requerida para la aplicación. Se ideó e implementó esta funcionalidad para facilitar la creación de los modelos de datos para todos los clasificadores WEKA y con los distintos enfoques de clasificación (tanto directa como en fases). Al trabajar con grandes archivos de entrenamiento y grandes archivos para clasificar, se notó que la aplicación demoraba demasiado tiempo (utilizando algunos clasificadores) en procesar todos los datos, sobre esto se va a hablar más adelante, en el capítulo 4.5 *Atributos y Escenarios de Calidad*, por el momento solo tenga en cuenta la gran demora en entrenar un clasificador.

Lo que se busca con el entrenamiento automático es contar con un set de modelos ya entrenados para agilizar la clasificación de los datasets. Si se cuenta con un modelo de datos, esto quiere decir que ya se pasó por todo el proceso de entrenamiento y ayudará a reducir los tiempos. El entrenamiento automático entrena cada uno de los clasificadores WEKA que están disponibles en la aplicación para los tipos de clasificación directa y en fases, teniendo como salida dos carpetas llamadas *models* y *results* dentro de otra carpeta llamada *autotrain*. La primera carpeta, *models*, contendrá los modelos de datos ya entrenados y la segunda carpeta, *results*, contiene los datos de la evaluación del clasificador que está siendo procesado utilizando el método *Cross-validation folds* con el dataset de entrenamiento.

Estos archivos de los modelos que vienen incluidos con la aplicación fueron creados con un conjunto de datos fijos. Estos son los datos y sus valores:

- Parámetros utilizados para el clasificador WEKA: el predeterminado para el clasificador elegido.
- N-Grama mínimo: 1.
- N-Grama máximo: 3.
- Cross validation folds: 10.
- Utilizar FreeLing: sí.
- Dataset de entrenamiento: 2149 interacciones.

4.5. Atributos y Escenarios de calidad

4.5.1 Atributo de calidad: Modificabilidad

Escenario de calidad 1: Un desarrollador desea agregar un nuevo clasificador WEKA. Se realizará este cambio en el código en tiempo de diseño, tomará menos de 4 horas para realizarlo y testearlo y no producirá efectos secundarios.

- Fuente: un desarrollador
- Estímulo: desea agregar un clasificador WEKA
- Entorno: en tiempo de diseño
- Artefacto: Código
- Respuesta: la modificación es hecha sin efectos secundarios
- Medida de respuesta: en 4 horas

Para dar respuesta a este escenario de calidad se aprovechó la estructura de herencia detallada en secciones anteriores (principalmente en la sección 4.3.4 *Diseño del módulo WEKA*). De esta forma, solo hace falta crear una nueva clase que extienda la clase *abstracta* Weka y que esta nueva clase importe el clasificador que se desea agregar. Luego con hacer unas simples modificaciones a dos archivos más (`Classifier.java` y `classifiers-options-description.properties`) se tienen listos los cambios para tener un nuevo clasificador listo para utilizar. Si se desea poder utilizar nuestra aplicación en el modo *Procesar datos (simple)* se deberán crear los modelos para este nuevo clasificador WEKA recién agregado y ubicarlos en la carpeta correspondiente.

Escenario de calidad 2: Un desarrollador agrega un nuevo enfoque para clasificar los datos. Se realizará este cambio en el código en tiempo de diseño, tomará menos de 3 días para realizarlo y testearlo y no producirá efectos secundarios.

- Fuente: un desarrollador
- Estímulo: desea agregar un enfoque de clasificación
- Entorno: en tiempo de diseño
- Artefacto: Código
- Respuesta: la modificación es hecha sin efectos secundarios
- Medida de respuesta: en 3 días

Para dar respuesta a este escenario de calidad se aprovechó la estructura de herencia detallada en secciones anteriores (principalmente en la sección 4.3.6 *Diseño del módulo Procesar Dataset*). De esta forma, solo hace falta crear una nueva clase que extienda la clase *abstracta* `ProcessDataset` (como ya lo están haciendo las clases `DirectProcessing`, `PhasesProcessingSingleClassifier` y `PhasesProcessing`) y que esta nueva clase implemente los métodos abstractos correspondientes. Luego con hacer unas simples modificaciones en la clase `Controller.java` para que esta use e inicialice el nuevo enfoque de clasificación se tienen listos los cambios a nivel código. Para finalizar se deberá agregar toda la parte gráfica que se soporte a este nuevo enfoque de clasificación.

4.5.2 Atributo de calidad: Performance

Escenario de calidad 3: Un usuario inicia una clasificación de un dataset bajo operaciones normales, el dataset es clasificado en menos de 5 minutos.

- Fuente: un usuario
- Estímulo: inicia clasificación de un dataset
- Entorno: bajo operaciones normales
- Artefacto: el sistema
- Respuesta: el dataset es procesado
- Medida de respuesta: menos de 5 minutos

Debido a que el procesamiento de grandes textos requiere mucho uso del procesador y teniendo en cuenta que para este tipo de clasificaciones se hacen varios procesamientos intermedios, es normal que el tiempo que se demore en realizar todo el procesamiento sea elevado. Es por eso que este escenario de calidad es importante. Para

poder cumplir con este escenario de calidad es que se desarrolló la funcionalidad *Procesar datos (simple)*. Esta forma de procesamiento de datos es la que utiliza los modelos de datos pre-entrenados que ya vienen incluidos con la aplicación (explicado en la sección 4.4.3). El tipo de procesamiento de datos simple permite que casi cualquier dataset que se desea clasificar, tarde menos de 5 minutos (dependiendo del tamaño del dataset). Al tener los modelos pre-entrenados se ahorra muchísimo tiempo de clasificación, ya que este entrenamiento del modelo es el que mayor tiempo consume (en comparación con la clasificación en sí) a la hora de querer clasificar un dataset. En la Figura 4.22 se puede ver el UCM (Use Case Map) que representa el escenario de calidad 3.

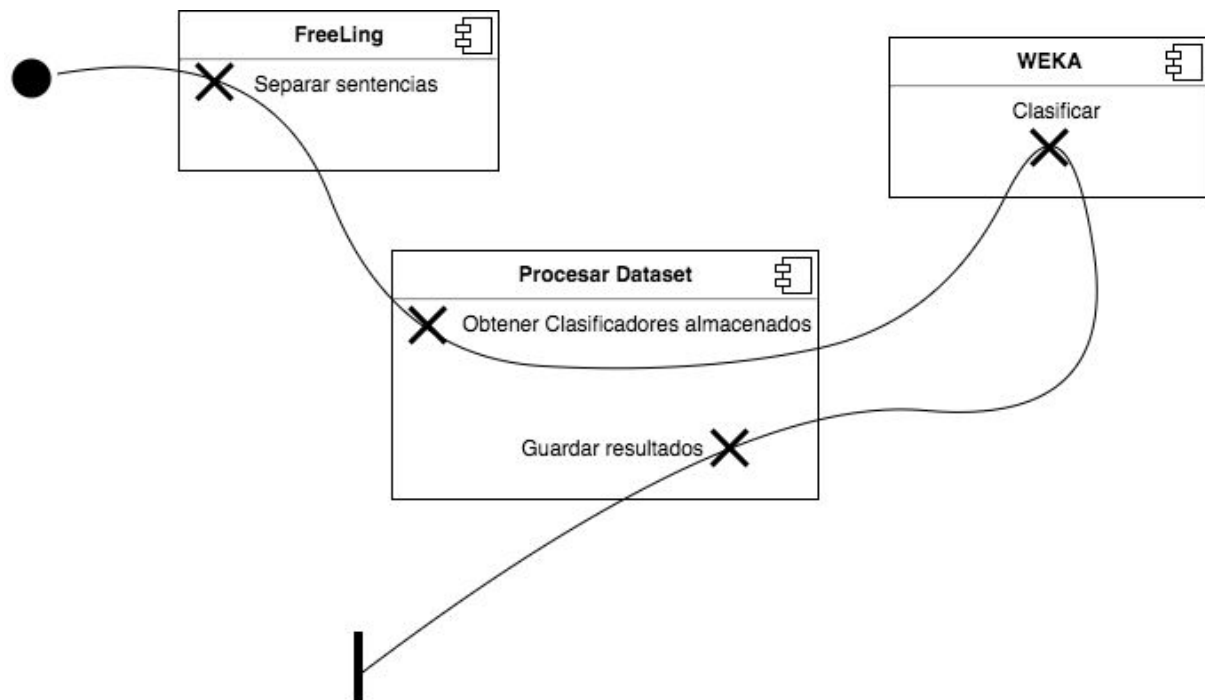


Figura 4.22: Dataset clasificado en menos de 5 minutos (UCM).

4.5.3 Atributo de calidad: Usabilidad

Escenario de calidad 4: Un usuario desea hacer uso eficiente de la aplicación para clasificar un dataset. El usuario pone la aplicación a procesar en menos de 1 minuto.

- Fuente: un usuario
- Estímulo: uso eficiente del sistema
- Entorno: tiempo de ejecución
- Artefacto: el sistema
- Respuesta: desea clasificar un dataset
- Medida de respuesta: inicia la clasificación en menos de 1 minuto

La interfaz de usuario de nuestra aplicación es bastante sencilla y simple de entender. Básicamente, requiere que el usuario provea un archivo para que la aplicación se entrene (solo es que se utiliza el enfoque de clasificación avanzado), otro archivo para clasificar, elegir un clasificador (o varios si se utiliza la clasificación en fases) y presionar el botón “Comenzar”. Luego de hacer esto se verán los resultados del entrenamiento (si se está utilizando la clasificación avanzada), los resultados de la clasificación y los resultados

del análisis de los datos (pestaña “Análisis de datos”). Si se quiere generar nuevos grupos de trabajo basta con ingresar el número de personas que se desea que cada grupo tenga y presionar el botón “Crear grupos” donde se mostrará una nueva ventana y ahí se podrán elegir las personas para la creación de los grupos. Todo este proceso se puede realizar en poco tiempo (excluyendo los tiempos de procesado de los datos) y el manual de usuario, que es un anexo incluido en este informe aclara todas las posibles acciones que permite realizar la aplicación.

Escenario de calidad 5: El usuario desea, a la hora de crear nuevo grupos de trabajo colaborativos, tener la capacidad de elegir la cantidad de miembros, así como también elegir las personas que quiere que integren los grupos en pocos y simples pasos.

- Fuente: un usuario
- Estímulo: uso eficiente del sistema, en pocos y simples pasos
- Entorno: tiempo de ejecución
- Artefacto: el sistema
- Respuesta: desea crear nuevo grupo de trabajo colaborativo eligiendo el tamaño de los grupos y los miembros a utilizar
- Medida de respuesta: el sistema genera los grupos de trabajo colaborativos, los muestra por pantalla y genera un archivo con los resultados

Para resolver este escenario de calidad se creó un módulo que contiene un grupo de clases que trabajan en colaboración y también es apoyado por las pantallas generadas para este propósito. Este módulo fue explicado en la sección 4.3.7 *Diseño del módulo Análisis IPA*. Todas las clases del módulo trabajan en conjunto teniendo como entrada los resultados de las clasificaciones, el tamaño del grupo elegido por el usuario y los miembros de los grupos que se quieren utilizar. Con toda esta información el sistema crea los perfiles de usuarios, los muestra por pantalla y genera archivos con la misma información.

En la Figura 4.23 se puede ver el UCM (Use Case Map) que representa el escenario de calidad 5.

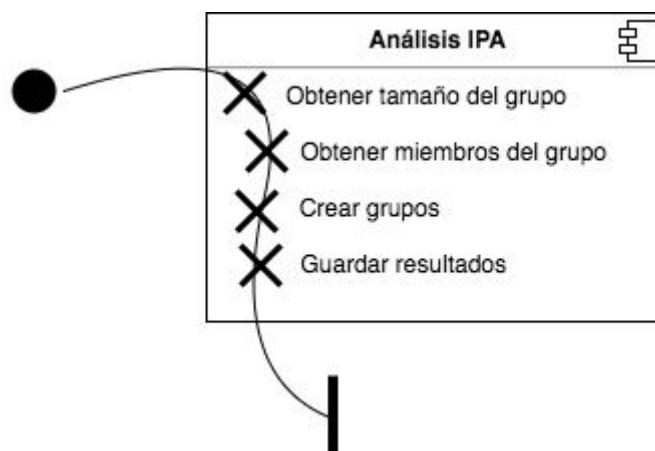


Figura 4.23: Elegir la cantidad de miembros del grupo (UCM).

Escenario de calidad 6: El usuario desea, a la hora de clasificar un dataset, tener la posibilidad de elegir entre distintos enfoques de clasificación (directo o en fases) de forma clara.

- Fuente: un usuario
- Estímulo: uso eficiente del sistema, se puede elegir el enfoque de forma clara
- Entorno: tiempo de ejecución
- Artefacto: el sistema
- Respuesta: desea clasificar un dataset eligiendo el enfoque de clasificación
- Medida de respuesta: el sistema muestra de forma clara los distintos enfoques y le permite al usuario utilizarlo, al finalizar se muestran los resultados por pantalla y se generan archivos con los mismos resultados.

Esta es una de las principales funcionalidades que tiene esta aplicación (clasificar datasets). Al iniciarse la aplicación, el usuario se va a encontrar con tres pestañas, dos de ellas están orientadas a recibir datos para la clasificación de datasets. En estas dos pestañas, *Procesar datos (simple)* y *Procesar datos (avanzado)*, el usuario podrá elegir el enfoque de clasificación que desea utilizar, ya sea directo o en fases. Una vez elegido el enfoque y el resto de los datos, la aplicación procesará el dataset y presentará los resultados por pantalla (también generará unos archivos con los resultados de la clasificación).

En la Figura 4.24 se puede ver el UCM (Use Case Map) que representa el escenario de calidad 6. En el UCM solo está representando el caso de que el usuario elige el método de clasificación simple, *Procesar datos (simple)*, por motivos de simplicidad en el gráfico.

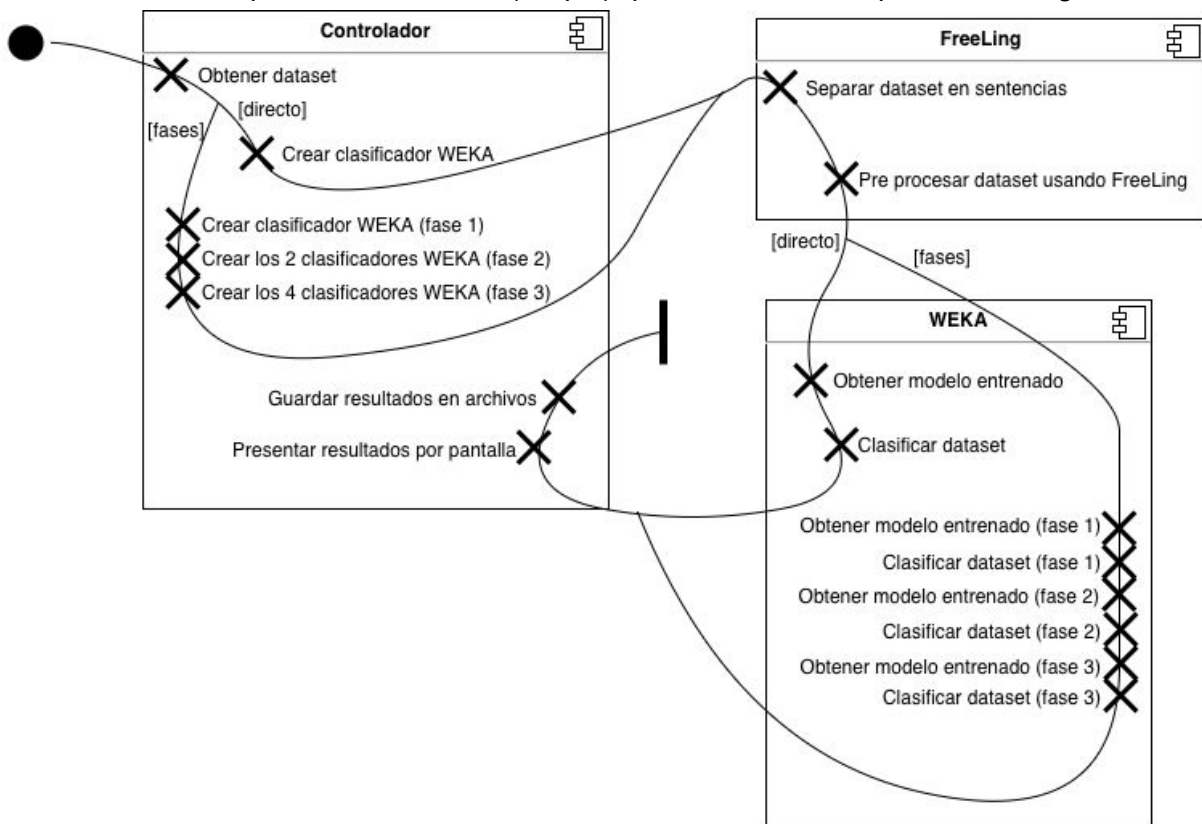


Figura 4.24: Elegir el enfoque de clasificación y clasificar un dataset (UCM).

4.5.4 Atributo de calidad: Portabilidad

Escenario de calidad 7: Los usuarios desean utilizar el sistema sin problemas tanto en computadoras con sistema operativo Windows como con MacOS.

- Fuente: los usuarios
- Estímulo: ejecución del sistema
- Entorno: sistemas operativos Windows o MacOS
- Artefacto: el sistema
- Respuesta: el sistema es ejecutado
- Medida de respuesta: sin problemas

Si bien la aplicación está escrita con el lenguaje de programación Java, presenta ciertas limitaciones para correrla en cualquier sistema operativo que soporte la máquina virtual de Java. Esta limitación se da debido a la herramienta FreeLing. FreeLing provee una API escrita en C/C++ (para utilizar en programas escritos en el mismo lenguaje) para correr sobre Linux. Lo que sus autores proveen es una forma de re-compilar su código y crear un “envoltorio” o “wrapper” para que se pueda utilizar desde aplicaciones escritas en Java (así como también para Perl, PHP, Python y Ruby). Este wrapper genera un archivo JAR que es el que se debe importar al proyecto y también es necesario generar unas librerías que serán usadas por la aplicación (un archivo con extensión DYLIB y otro DLL para MacOS y Windows respectivamente). Una vez que se cuenta con todos estos archivos (y se realizan algunas configuraciones en la computadora donde se desea correr la aplicación), es posible usar esta aplicación sin problemas.

4.6 Conclusión

Como se pudo ver en el capítulo 4 se describió el diseño realizado para resolver el problema de la clasificación de chats de grupos de trabajo colaborativo, así como también la generación de nuevos grupos de trabajo a partir de los perfiles de cada persona. Se vieron los requerimientos funcionales junto con sus restricciones y las especificaciones de los requerimientos. Luego se pasó al diseño de la solución, donde gradualmente se fue presentando este, comenzando de lo más general hasta llegar a un nivel más profundo. De todos modos, no se llegó a presentar por completo el diseño ni la implementación, si el lector desea conocer en más profundidad este proyecto, se puede referir al código fuente.

Durante el desarrollo de este capítulo también se vió cómo se resuelve la persistencia de datos a medida que se procesan los datos y se finalizó presentando los atributos y escenarios de calidad propuestos junto con la explicación de cómo se resuelve cada uno.

En el siguiente capítulo se verá cómo se procesan y utilizan los datos obtenidos de la clasificación para crear nuevos grupos de trabajo colaborativo buscando minimizar los conflictos en el mismo. Se verá en detalle el algoritmo encargado de realizar este trabajo y también cómo se crean los perfiles de las personas.

5. Aplicación de análisis de datos

Ya explicado el diseño de la aplicación en el capítulo anterior, es momento de detallar el proceso que sigue luego de la clasificación de los datasets. Al finalizar la clasificación, la aplicación se encarga de procesar esos datos. Para esto, obtendrá el porcentaje de interacciones que se realizaron en los grupo de trabajo colaborativo para cada una de las 12 conductas IPA, resaltando las conductas que estén por fuera de los límites libres de conflictos definidos por Bales. Por otro lado, también se genera un perfil por cada persona que haya participado de los grupos de trabajo y esta información será utilizada para la generación de grupos de trabajo colaborativo buscando minimizar los conflictos en cada grupo.

Luego que la aplicación termina de realizar la clasificación del dataset (que puede contener uno o más grupos para los casos en que el dataset es un JSON de Google Hangouts), se utilizarán los archivos ARFF generados (uno por cada grupo) para recabar la información necesaria. En la tabla 5.1 se puede ver el formato de estos archivos ARFF.

<pre>@relation [relación] ... @data [Número de conducta de Bales],[Nombre],[Mensaje] [Número de conducta de Bales],[Nombre],[Mensaje] [Número de conducta de Bales],[Nombre],[Mensaje]</pre>
Número de conducta de Bales: 1-12 Nombre: debe ser una cadena de caracteres. Delimitado por comillas, si contiene un espacio Mensaje: debe ser una cadena de caracteres. Delimitado por comillas, si contiene un espacio

Tabla 5.1: Formato de salida de la clasificación directa.

5.1 Detección de conflictos en un grupo

Como se explicó anteriormente, sección 2.1 *El modelo IPA de Bales* y más precisamente en la *Tabla 2.2: Conductas IPA y valores límite*, Bales definió para cada conducta un rango de porcentajes. Mientras un grupo de trabajo colaborativo se encuentre dentro del rango definido para una conducta dada, se dice que ese grupo está fuera de conflicto para esa conducta.

Para detectar los conflictos en un grupo, por un lado se contabiliza la cantidad total de interacciones hechas en el grupo; y por el otro se contabiliza la cantidad de interacciones que se hicieron para cada conducta. Con estos dos datos es posible calcular el porcentaje de interacciones que se hicieron para cada conducta. Contrastando los porcentajes obtenidos para cada una de las conductas para el grupo con los porcentajes definidos por Bales, se puede saber qué conductas están fuera de los rangos y por ende los posibles conflictos que tenga el grupo. En la siguiente figura (Figura 5.1) se puede ver un ejemplo de

los resultados mostrados por la aplicación. En la primera columna se presentan los conflictos a los que puede estar expuesto un grupo de trabajo colaborativo. La segunda columna muestra las conductas definidas en los estudios de Bales. Existen dos conductas asociadas a cada conflicto, estas conductas son complementarias. Las dos columnas siguientes sirven de referencia para tener presente los porcentajes de los límites inferiores y superiores para cada conducta. La última columna presenta el resultado del análisis de las interacciones predichas por la aplicación. Se puede ver en la figura que algunas filas que están en color rojo y otras en color negro. Las filas en color rojo representan las conductas cuyo porcentaje, obtenido del análisis de los datos, está por fuera de los rangos definidos, y por lo tanto es posible que exista un conflicto. Es aconsejable tomar alguna medida de corrección en estos casos. Las filas que están en color negro se encuentran dentro del rango “seguro” y no es necesario realizar ninguna acción correctiva. Pero estos valores de pueden observar con el paso del tiempo y si se detecta que el porcentaje de alguna de las conductas se está aproximando a alguno de los valores límite, se puede tomar alguna medida preventiva.

Conflicto	Conducta	Límite Inferior	Límite Superior	Resultado del Análisis
Reintegración	C1. Muestra solidaridad	0%	5%	11,11%
Reducción de tensión	C2. Muestra relajamiento	3%	14%	0,00%
Decisión	C3. Muestra acuerdo o aprueba	6%	20%	11,11%
Control	C4. Da sugerencia u orientación	4%	11%	22,22%
Evaluación	C5. Da opiniones	21%	40%	5,56%
Comunicación	C6. Da información	14%	30%	38,89%
Comunicación	C7. Pide información	2%	11%	0,00%
Evaluación	C8. Pide opinión	1%	9%	11,11%
Control	C9. Pide sugerencia u orientación	0%	5%	0,00%
Decisión	C10. Muestra desacuerdo	3%	13%	0,00%
Reducción de tensión	C11. Muestra tensión o molestia	1%	10%	0,00%
Reintegración	C12. Muestra antagonismo	0%	7%	0,00%

Figura 5.1: resultados del análisis de datos de un grupo.

5.2 Perfiles de personas

Los perfiles particulares a cada persona no son presentados al usuario por pantalla, sino que estos perfiles se crean para ser considerados a la hora de crear futuros grupos de trabajo colaborativo.

Cuando se hace el análisis de los datos recabados durante la clasificación de los datasets, parte de este análisis consiste en generar el perfil de cada persona. Para esto se crea un objeto *IpaPerson* por cada persona que haya participado en los grupos (para más detalle sobre esta clase vea la sección 4.3.7 *Diseño del módulo Análisis IPA* y el diagrama de clases de la Figura 4.21: *Diagrama de clases del módulo Análisis IPA*), si una misma persona participó en más de un grupo, la información relacionada a esta persona será contabilizada en el mismo objeto *IpaPerson*. Cada uno de estos objetos contiene el nombre de la persona, que se usa como identificador y la cantidad de interacciones que realizó para cada conducta IPA.

5.3 Generación de nuevos grupos de trabajo

Para la generación de los nuevos grupos de trabajo colaborativo, la aplicación necesita como entrada el tamaño deseado para los grupos (ingresado por el usuario), los nombres de las personas que van a formar parte de los grupos creados (el usuario selecciona las personas que quiere que sean parte de los nuevos grupos) y los resultados de la clasificación. Una vez que se tienen todos estos datos, se procede a crear los grupos buscando minimizar la probabilidad de que surjan conflictos en cada grupo, de acuerdo a los criterios sugeridos por Bales.

5.3.1 Minimizar conflictos

Crear nuevos grupos de trabajo buscando minimizar los conflictos es una tarea que requiere mucho procesamiento de datos y lógica interna. Para armar los grupos de forma óptima, se deberían hacer muchas combinaciones de personas en cada grupo, la complejidad computacional para realizar esta tarea puede llegar a ser muy elevada. Debido a que la creación de nuevos grupos no es el objetivo principal de este trabajo, se utilizó un algoritmo simple, evitando los elevados costos computacionales de utilizar técnicas evolutivas o de aprendizaje profundo. Antes de continuar con la explicación del algoritmo utilizado, se definirá el término “desviación”, concepto que se utilizará para la creación de los grupos.

Desviación: es un valor numérico que representa la cantidad de unidades porcentuales que una persona o un grupo se encuentran por fuera de los valores “seguros” definidos por Bales. En la Fórmula 5.1 se puede ver cómo se realiza el cálculo de la desviación. Siendo D la desviación, c la conducta, P_c el porcentaje para una conducta dada, l_c el límite inferior para una conducta dada y L_c el límite superior para una conducta dada.

$$D(c) = \begin{cases} P_c - l_c & \text{si } P_c < l_c \\ P_c - L_c & \text{si } P_c > L_c \\ 0 & \text{si } l_c \leq P_c \leq L_c \end{cases}$$

Fórmula 5.1: cálculo de la desviación de una persona para una conducta dada.

Por ejemplo: suponiendo la siguiente tabla (Tabla 5.2) que muestra el porcentaje de interacciones que hizo una persona en uno o más grupos. Como se ve en la tabla, el 25% de las interacciones corresponden a la conducta C1, otro 25% de las interacciones a la conducta C3 y el otro 50% a la conducta C6, el resto de las conductas no tuvieron interacciones.

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
25%	0%	25%	0%	0%	50%	0%	0%	0%	0%	0%	0%

Tabla 5.2: Porcentaje de interacciones para cada conducta Bales para una persona.

Si se compara la Tabla 5.2 con la tabla que contiene las conductas IPA y sus valores límite (Tabla 5.3), se ve que algunos de los porcentajes de la Tabla 5.2 están dentro de los límites definidos por Bales y otros no.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
Lím. inf.	0%	3%	6%	4%	21%	14%	2%	1%	0%	3%	1%	0%
Lím. sup.	5%	14%	20%	11%	40%	30%	11%	9%	5%	13%	10%	7%

Tabla 5.3: Conductas IPA y sus valores límite.

Para los valores que se encuentran dentro de los límites superiores e inferiores, el desvío es de cero; cuando el porcentaje es inferior al límite inferior definido por Bales, la desviación será negativa; cuando el porcentaje es superior al límite superior, la desviación será positiva. En la Tabla 5.4 se pueden ver los valores de la desviación para cada conducta. Para C1, que tiene el valor 20, se puede ver que el porcentaje de interacciones para esta conducta fue de 25% y el límite superior es de 5%. Para C2, se puede ver que tiene un valor de -3, esto es porque la persona tiene 0% de interacciones para esta conducta y el límite inferior es 3%. Por último, la conducta C9 no tiene desviación, esto es porque el porcentaje de interacciones está dentro de los límites superior e inferior.

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	Total
20	-3	5	-4	-21	20	-2	-1	0	-3	-1	0	80

Tabla 5.4: Desviación para cada conducta Bales para una persona.

Continuando con la explicación del algoritmo. Para minimizar la probabilidad de que surjan conflictos, al crear un nuevo grupo se comienza con la lista completa de personas que se desean agrupar, se busca la persona con mayor desviación y se la agrega a un nuevo grupo. Dentro del grupo se busca cuál es la conducta con mayor desvío y se busca otra persona que contrarreste el desvío del grupo. Este proceso es repetido hasta que el grupo llegue a tener la cantidad deseada de personas o hasta que no quede gente disponible para asignar en algún grupo. En el siguiente diagrama de flujo, Figura 5.2, se puede ver el algoritmo.

Una vez finalizado el algoritmo de creación de nuevos grupos de trabajo colaborativo, se muestran por pantalla los resultantes grupos y también se guardan en archivos.

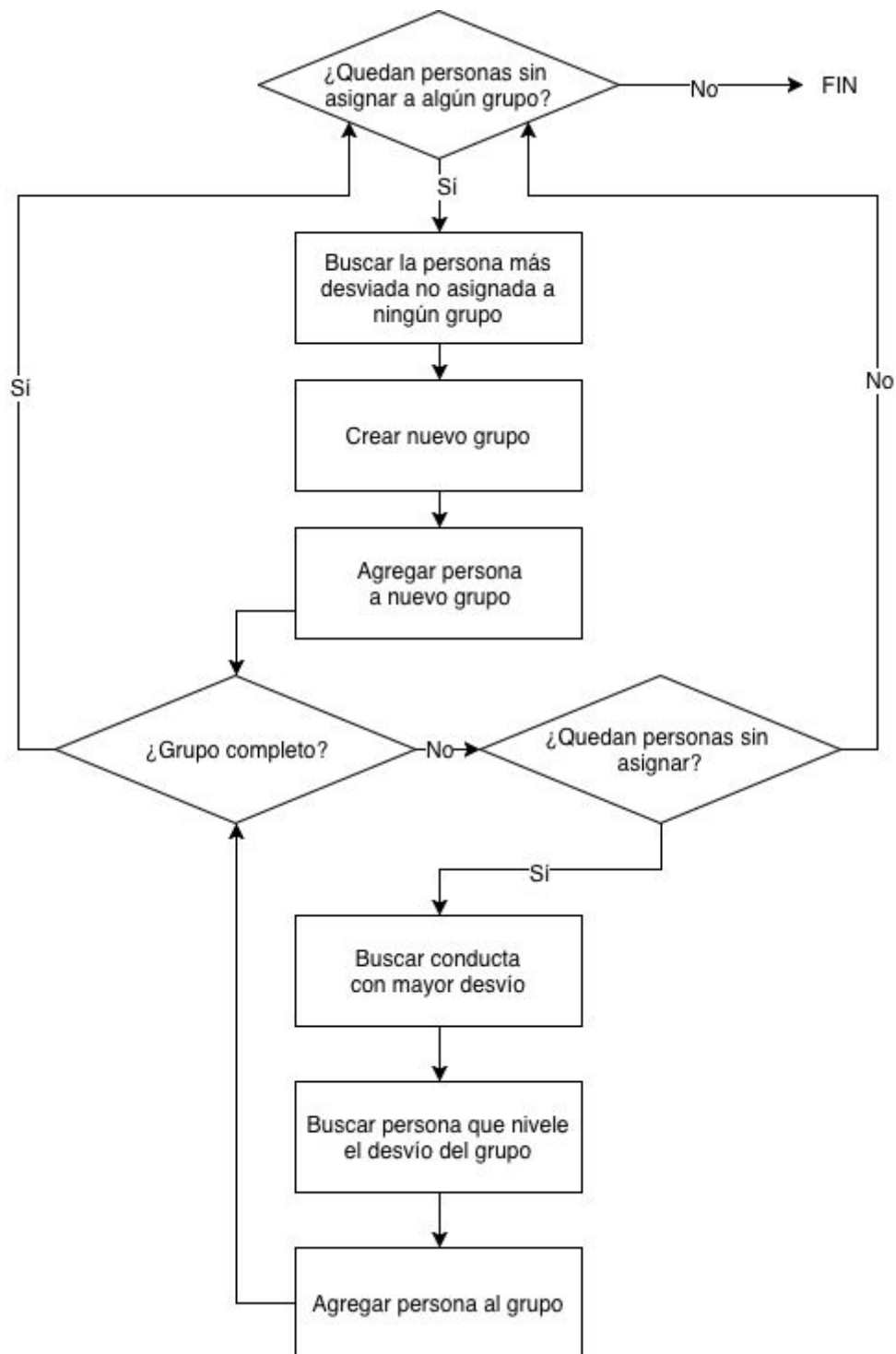


Figura 5.2: diagrama de flujo del algoritmo para la creación de nuevos grupos de trabajo colaborativo buscando minimizar los conflictos.

5.4 Conclusión

En este capítulo se discutió una posible aplicación de los resultados de la clasificación de mensajes textuales en diferentes conductas colaborativas.. Se detalló cuándo se considera que en un grupo surgieron conflictos colaborativos, cómo obtienen los

perfiles de los usuarios y cómo esta información puede utilizarse, por ejemplo, para la generación de nuevos grupos de trabajo colaborativo.

En el siguiente capítulo se describirán los distintos enfoques de clasificación analizados, el conjunto de datos utilizados para evaluar el enfoque propuesto, y los resultados obtenidos. Con estos resultados se puede ver cuál es la estrategia de clasificación que mejor se adapta a las necesidades de este trabajo.

6. Descripción del experimento

En este capítulo se detalla el conjunto de datos utilizados para realizar la evaluación experimenta y el procedimiento llevado a cabo para efectuar la misma. Finalmente, se muestran los resultados obtenidos, un análisis de los resultados y sus implicancias.

6.1. Conjunto de datos

Para realizar los experimentos, se recolectó un conjunto de interacciones realizadas por estudiantes que realizan una tarea de colaboración con Google Docs. Los estudiantes pertenecen a un curso de ingeniería de sistemas de la Universidad UNICEN (Argentina). Los 82 participantes del experimento se dividieron en 17 grupos, con 5 o 6 miembros cada uno. Cada grupo recibió instrucciones de escribir, de forma colaborativa, tres documentos con diferentes consignas.

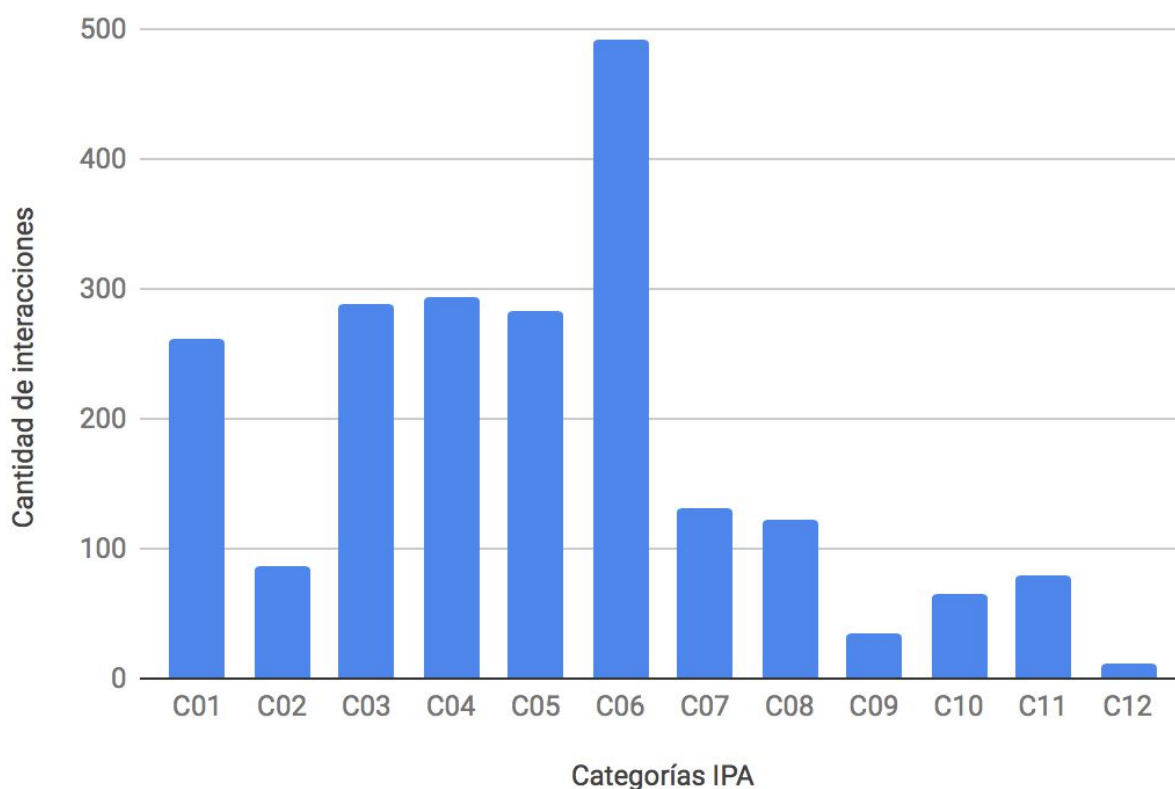


Figura 6.1: Distribución de las interacciones de las categorías IPA.

Los datos de la interacción entre los estudiantes se recopilaron y se analizaron manualmente para asignar la categoría IPA más relacionada a cada interacción. El número total de registros en el conjunto de datos es de 2149 interacciones. La Figura 6.1 muestra la distribución de las interacciones de acuerdo con las 12 categorías de IPA.

6.2. Proceso del experimento

El objetivo es construir un modelo capaz de clasificar automáticamente las interacciones entre los miembros de un grupo en las 12 categorías de IPA con el fin de reducir la gran cantidad de recursos humano-temporales necesarios para aplicar manualmente este método. Se evaluaron 3 enfoques distintos para clasificar los datasets que serán explicados a continuación.

6.2.1 Enfoque directo

Con este enfoque se toma como entrada el texto obtenido del chat grupal y el clasificador WEKA seleccionado. Se entrena el modelo de clasificación seleccionado utilizando el conjunto de interacciones mencionado anteriormente y el mismo se utiliza para clasificar directamente el texto libre con el objetivo de identificar una de las 12 categorías de IPA. La Figura 6.2 muestra el enfoque de clasificación directa.



Figura 6.2: Enfoque directo de clasificación.

6.2.2 Enfoque en fases

También se proponen dos enfoques distintos de clasificación (único clasificador y múltiples clasificadores), divididos en tres fases. Para explicar bien estas dos diferencias se debe separar la explicación en dos partes: el entrenamiento y la clasificación.

El entrenamiento se realiza de igual manera para los dos enfoques de clasificación en fases. Primero, se entrena un modelo de clasificación directamente del texto libre para identificar el **área** de IPA (socio-emocional u orientado a tareas). En la segunda fase, se entrena un modelo con el texto y el **área** detectada en la fase previa para detectar el tipo de **reacción** que se expresa (positiva, negativa, pregunta o respuesta). Finalmente, un tercer clasificador se entrena con el texto libre, el **área** y el tipo de **reacción** detectada por los clasificadores anteriores para asignar una de las **12 categorías** de IPA.

La etapa de clasificación es distinta para los dos enfoques propuestos. Uno de estos enfoques utiliza el mismo clasificador en las tres fases y el otro enfoque utiliza múltiples clasificadores en las distintas etapas. Este último enfoque utiliza un clasificador para la

primera fase, dos para la segunda y 4 para la tercera fase. En lo que sí coinciden estos dos enfoques es que la salida de una fase sirve como entrada de la siguiente.



Figura 6.3: Enfoque en fases de clasificación (único clasificador).

La Figura 6.3 muestra el enfoque en fases utilizando un único clasificador y la Figura 6.4 muestra el enfoque de clasificación en fases utilizando múltiples clasificadores. Para este último enfoque, el proceso de clasificación es más complejo. El modelo de la fase 1 se utiliza para clasificar las interacciones dentro del área de IPA (socio-emocional u orientado a tareas). En la fase 2 se cuentan con 2 modelos, el primero se utiliza para clasificar las interacciones clasificadas como *socio-emocional* en la fase 1 y clasifica las interacciones como *positiva* o *negativa* y el otro modelo se utiliza para las interacciones clasificadas como *tarea* en la fase 1 y clasifica las mismas como *pregunta* o *respuesta*. La fase 3 sigue la misma lógica utilizando 4 modelos para clasificar. Un modelo clasifica las interacciones clasificadas en la fase 2 como *positiva*, otro clasifica las interacciones clasificadas como

negativa en la fase anterior y lo mismo sucede para lo otros dos modelos para las interacciones clasificadas como *respuesta* y *pregunta*.

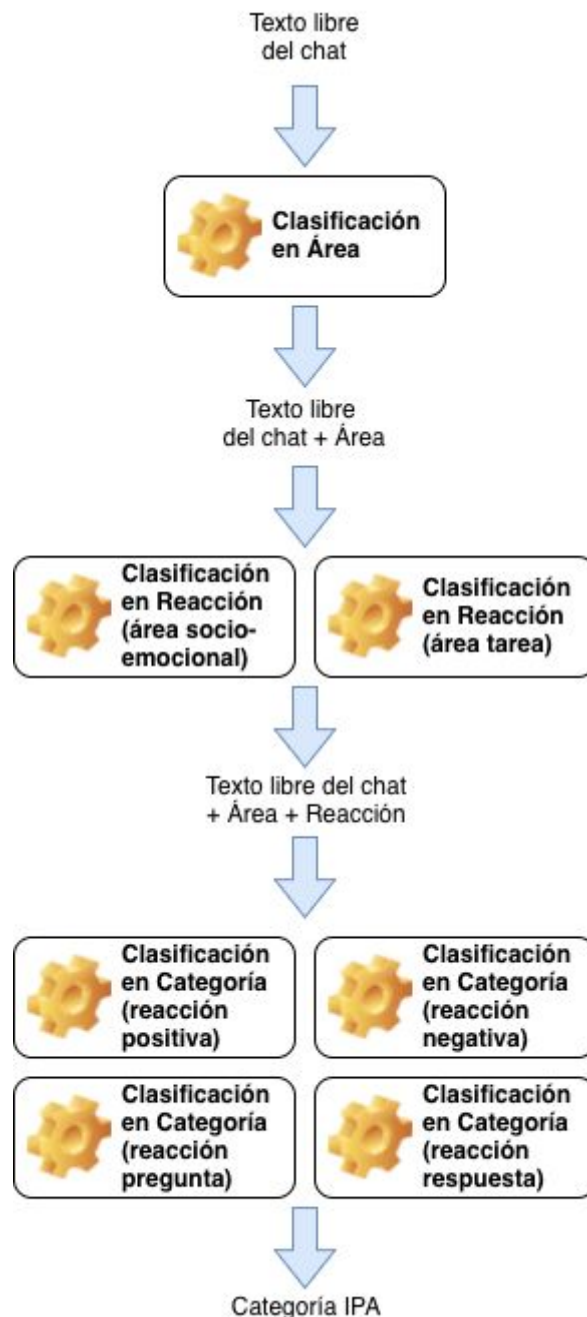


Figura 6.4: Enfoque en fases de clasificación (múltiples clasificadores).

Se tiene como primera hipótesis que al realizar el proceso de clasificación de tres fases se pueden mejorar los resultados obtenidos al clasificar de manera directa, que asigna una de las 12 categorías de IPA directamente al texto libre de cada interacción.

La segunda hipótesis que se planteó es que al utilizar múltiples clasificadores se puede elegir el mejor clasificador para cada una de las fases y con eso mejorar la clasificación.

Se probó el rendimiento de diferentes algoritmos de clasificación utilizando los enfoques de clasificación en fases propuestos contrastando con un modelo de clasificación

directa. Los clasificadores usados son los siguientes: *Naive Bayes*, *Naive Bayes Multinomial Updateable*, *SMO*, *IBk*, *KStar*, *Logit Boost*, *Decision Table*, *JRip*, *PART*, *J48*, *REPTree*. De esta forma, se busca demostrar que el enfoque en fases es capaz de obtener mejoras significativas en comparación con la aplicación directa de los modelos de clasificación simple.

6.3 Resultados

Se comparó el rendimiento de los 11 algoritmos de clasificación tanto para el enfoque propuesto de tres fases como para el enfoque monofásico o directo. Las tablas 6.1, 6.2 y 6.3 muestra los resultados obtenidos para la clasificación directa, clasificación en fases (único clasificador) y clasificación en fases (múltiples clasificadores) respectivamente.

Para la Tabla 6.3, los resultados mostrados en las columnas “Fase 2” y “Fase 3” son un promedio de la clasificación hecha durante cada fase. Para la fase 2 se entrenaron dos clasificadores distintos, cada uno utilizando los mensajes del chat originales junto con la clasificación de la primera fase para una de las dos áreas (socio-emocional u orientado a tareas). Para la fase 3 se entrenaron cuatro clasificadores distintos, cada uno utilizando los mensajes originales junto con la clasificación de la segunda fase para cada una de las reacciones (positiva, negativa, pregunta o respuesta).

Se mostrarán en más detalle los diferentes resultados de los entrenamientos para cada una de las fases cuando se muestren las matrices de confusión de los clasificadores que obtuvieron mejores resultados.

Clasificador	Clasificación directa
Naive Bayes	38,12%
Naive Bayes Multinomial Updateable	32,59%
SMO	43,56%
IBk	33,56%
KStar	N/A
Logit Boost	40,77%
Decision Table	32,63%
JRip	32,12%
PART	38,77%
J48	38,35%
REPTree	32,72%

Tabla 6.1: Precisión (%) para los modelos de la clasificación directa.

Clasificación	Fase 1	Fase 2	Fase 3	Promedio
Naive Bayes	68,71%	67,04%	57,27%	64,34%
Naive Bayes Multinomial Updateable	73,36%	60,99%	42,63%	58,99%
SMO	78,01%	81,26%	63,09%	74,12%
IBk	74,80%	74,10%	53,09%	67,33%
KStar	32,26%	N/A	N/A	N/A
Logit Boost	76,47%	80,70%	62,48%	73,22%
Decision Table	76,05%	80,52%	54,16%	70,24%
JRip	74,43%	80,05%	56,57%	70,35%
PART	74,05%	77,82%	60,81%	70,89%
J48	76,20%	80,19%	61,32%	72,57%
REPTree	76,66%	80,75%	59,18%	72,20%

Tabla 6.2: Precisión (%) para los modelos de la clasificación en fases (único clasificador).

Clasificación	Fase 1	Fase 2	Fase 3	Promedio
Naive Bayes	68,71%	68,75%	59,92%	65,79%
Naive Bayes Multinomial Updateable	73,40%	80,57%	60,30%	71,42%
SMO	78,06%	80,89%	63,60%	74,18%
IBk	74,80%	79,31%	56,35%	70,15%
KStar	77,98%	79,73%	64,63%	58,87%
Logit Boost	76,48%	80,29%	63,13%	73,30%
Decision Table	73,06%	80,38%	59,13%	70,86%
JRip	74,34%	80,24%	57,37%	70,65%
PART	73,13%	79,08%	58,90%	70,37%
J48	75,96%	80,80%	61,23%	72,66%
REPTree	76,75%	80,61%	59,18%	72,18%

Tabla 6.3: Precisión (%) para los modelos de la clasificación en fases (múltiples clasificadores).

Una matriz de confusión (Tabla 6.4) es una herramienta útil para analizar qué tan bien un clasificador puede reconocer tuplas de diferentes clases [Han et al., 2012]. En una matriz de confusión, TP y TN indican que el clasificador clasificando correctamente, mientras que FP y FN indican cuando el clasificador está clasificando incorrectamente. Más formalmente, dadas m clases (donde $m \geq 2$), una matriz de confusión es una tabla de tamaño m por m . Una entrada, $MC_{i,j}$ en las primeras m filas y m columnas indica el número de tuplas de clase i que fueron clasificadas por el clasificador como clase j . Para que un clasificador tenga una buena precisión, lo ideal sería que la mayoría de las tuplas estuvieran representadas a lo largo de la diagonal de la matriz de confusión, desde la entrada $MC_{1,1}$ hasta la entrada $MC_{m,m}$, con el resto de las entradas siendo cero o cercanas a cero. Es decir, idealmente, FP y FN son cercanas a cero.

		Clase predicha		
		Sí	No	Total
Clase real	Sí	TP	FN	P
	No	FP	TN	N
	Total	P'	N'	P + N

Tabla 6.4: Matriz de confusión, mostrada con totales para tuplas positivas y negativas.

Comparando las tablas 6.1, 6.2 y 6.3 se puede observar que el enfoque de tres fases (tanto utilizando el enfoque con un único clasificador como el enfoque de múltiples clasificadores) obtuvo mejores resultados para todos los algoritmos de clasificación en comparación con el enfoque de clasificación directa. La mejor precisión de clasificación en fases se obtuvo mediante el enfoque de múltiples clasificadores para el modelo **SMO** con un porcentaje promedio de 74,18%. Mientras que la mejor precisión obtenida de la clasificación directa se obtuvo también mediante el modelo **SMO** con un porcentaje de 43,56%. El modelo en fases obtuvo una mejora de 30,62% con respecto a los mejores resultados de clasificación del enfoque de clasificación directa.

La Tabla 6.5 muestra la matriz de confusión para el clasificador **SMO** utilizando el enfoque directo y de la Tabla 6.6 a la Tabla 6.12 se muestran las matrices de confusión para el clasificador **SMO** utilizando el enfoque en fases. La principal observación de estas matrices de confusión es que, para el enfoque en fases, las instancias mal clasificadas siempre están dentro de la Reacción de IPA correspondiente. Por ejemplo, se puede ver que hay instancias para las categorías C1, C2 y C3 que no están correctamente clasificadas, pero las categorías asignadas por nuestro enfoque son siempre C1, C2 y C3, que corresponden a la reacción IPA "Positiva" (R1). En otras palabras, no hay clasificaciones erróneas de Reacciones cruzadas. Este hecho no ocurre para el enfoque de clasificación directo, donde se puede observar que las instancias que no están clasificadas correctamente por el modelo se distribuyen en todas las categorías.

Ninguno de los enfoques es capaz de identificar instancias con la clase C12 (muestra antagonismo, rebaja el estado de otros, defiende o afirma el propio). Este comportamiento se puede atribuir al hecho de que en el dataset utilizado solo hay 0.5% de ejemplos de esta conducta (11 instancias de 2149).

Los experimentos revelaron que el enfoque propuesto de clasificación de tres fases logra niveles muy buenos de rendimiento que mejoran significativamente los métodos de clasificación utilizados en los enfoques directos.

Clasificado como →	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
C1	102	10	67	10	7	56	3	3	0	2	1	0
C2	3	59	15	0	0	7	0	1	0	0	1	0
C3	19	3	204	8	15	36	1	3	0	0	0	0
C4	21	5	39	81	21	113	6	7	1	1	0	0
C5	9	2	49	18	85	107	4	4	0	5	1	0
C6	23	3	48	24	31	344	4	7	0	4	4	0
C7	11	1	32	9	3	60	10	6	0	0	0	0
C8	12	1	24	13	4	37	0	30	1	0	0	0
C9	8	0	0	3	0	14	0	4	4	0	1	0
C10	1	3	18	1	7	20	1	0	0	14	0	0
C11	6	15	11	4	3	35	0	1	0	1	4	0
C12	0	0	1	1	2	6	0	0	0	1	0	0
Precisión (avg=0.441)	.474	.578	.402	.471	.478	.412	.345	.455	.667	.500	.333	0
Recall (avg=0.436)	.391	.686	.706	.275	.299	.699	.76	.246	.118	.215	.50	0
F-measure (avg=0.402)	.429	.628	.512	.347	.368	.518	.124	.319	.200	.301	.87	0
ROC area (avg=0.741)	.773	.910	.807	.704	.735	.733	.677	.714	.662	.780	.620	.500

Tabla 6.5: Matriz de confusión, clasificador SMO con enfoque directo.

Clasificado como →	Socio-emocional	Tarea
Socio-emocional	461	331
Tarea	141	1218
Precisión (avg=0.779)	0.766	0.786
Recall (avg=0.781)	0.582	0.896
F-measure (avg=0.773)	0.661	0.838
ROC area (avg=0.739)	0.739	0.739

Tabla 6.6: Matriz de confusión, clasificador SMO con enfoque en fases (fase 1).

Clasificado como →	Positiva	Negativa	Pregunta	Respuesta
Positiva	609	27	0	0
Negativa	106	50	0	0
Pregunta	0	0	0	0
Respuesta	0	0	0	0
Precisión (avg=0.812)	0.852	0.649	0	0
Recall (avg=0.832)	0.958	0.321	0	0
F-measure (avg=0.809)	0.902	0.429	0	0
ROC area (avg=0.639)	0.639	0.639	?	?

Tabla 6.7: Matriz de confusión, clasificador SMO con enfoque en fases (fase 2), entidades clasificadas como Socio-emocional en la fase 1.

Clasificado como →	Positiva	Negativa	Pregunta	Respuesta
Positiva	0	0	0	0
Negativa	0	0	0	0
Pregunta	0	0	42	246
Respuesta	0	0	32	1039
Precisión (avg=0.757)	0	0	0.568	0.809
Recall (avg=0.795)	0	0	0.146	0.970
F-measure (avg=0.744)	0	0	0.232	0.882
ROC area (avg=0.558)	?	?	0.558	0.558

Tabla 6.8: Matriz de confusión, clasificador SMO con enfoque en fases (fase 2), entidades clasificadas como Orientado a la tarea en la fase 1.

Clasificado como →	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
C1	164	10	87	0	0	0	0	0	0	0	0	0
C2	8	63	15	0	0	0	0	0	0	0	0	0
C3	44	3	242	0	0	0	0	0	0	0	0	0
C4	0	0	0	0	0	0	0	0	0	0	0	0
C5	0	0	0	0	0	0	0	0	0	0	0	0
C6	0	0	0	0	0	0	0	0	0	0	0	0
C7	0	0	0	0	0	0	0	0	0	0	0	0
C8	0	0	0	0	0	0	0	0	0	0	0	0
C9	0	0	0	0	0	0	0	0	0	0	0	0
C10	0	0	0	0	0	0	0	0	0	0	0	0
C11	0	0	0	0	0	0	0	0	0	0	0	0
C12	0	0	0	0	0	0	0	0	0	0	0	0
Precisión (avg=0.743)	.759	.829	.703	0	0	0	0	0	0	0	0	0
Recall (avg=0.737)	.628	.733	.837	0	0	0	0	0	0	0	0	0
F-measure (avg=0.735)	.688	.778	.765	0	0	0	0	0	0	0	0	0
ROC area (avg=0.781)	.763	.851	.776	?	?	?	?	?	?	?	?	?

Tabla 6.9: Matriz de confusión, clasificador SMO con enfoque en fases (fase 3), entidades clasificadas como Positiva en la fase 2.

Clasificado como →	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
C1	0	0	0	0	0	0	0	0	0	0	0	0
C2	0	0	0	0	0	0	0	0	0	0	0	0
C3	0	0	0	0	0	0	0	0	0	0	0	0
C4	0	0	0	95	21	179	0	0	0	0	0	0
C5	0	0	0	25	95	164	0	0	0	0	0	0
C6	0	0	0	26	33	433	0	0	0	0	0	0
C7	0	0	0	0	0	0	0	0	0	0	0	0
C8	0	0	0	0	0	0	0	0	0	0	0	0
C9	0	0	0	0	0	0	0	0	0	0	0	0
C10	0	0	0	0	0	0	0	0	0	0	0	0
C11	0	0	0	0	0	0	0	0	0	0	0	0
C12	0	0	0	0	0	0	0	0	0	0	0	0
Precisión (avg=0.605)	0	0	0	.651	.638	.558	0	0	0	0	0	0
Recall (avg=0.582)	0	0	0	.322	.335	.880	0	0	0	0	0	0
F-measure (avg=0.549)	0	0	0	.431	.439	.683	0	0	0	0	0	0
ROC area (avg=0.669)	?	?	?	.688	.690	.645	?	?	?	?	?	?

Tabla 6.10: Matriz de confusión, clasificador SMO con enfoque en fases (fase 3), entidades clasificadas como Negativa en la fase 2.

Clasificado como →	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
C1	0	0	0	0	0	0	0	0	0	0	0	0
C2	0	0	0	0	0	0	0	0	0	0	0	0
C3	0	0	0	0	0	0	0	0	0	0	0	0
C4	0	0	0	0	0	0	0	0	0	0	0	0
C5	0	0	0	0	0	0	0	0	0	0	0	0
C6	0	0	0	0	0	0	0	0	0	0	0	0
C7	0	0	0	0	0	0	103	28	1	0	0	0
C8	0	0	0	0	0	0	48	70	4	0	0	0
C9	0	0	0	0	0	0	14	12	8	0	0	0
C10	0	0	0	0	0	0	0	0	0	0	0	0
C11	0	0	0	0	0	0	0	0	0	0	0	0
C12	0	0	0	0	0	0	0	0	0	0	0	0
Precisión (avg=0.628)	0	0	0	0	0	0	.624	.636	.615	0	0	0
Recall (avg=0.628)	0	0	0	0	0	0	.780	.574	.235	0	0	0
F-measure(avg=0.614)	0	0	0	0	0	0	.694	.603	.340	0	0	0
ROC area (avg=0.677)	?	?	?	?	?	?	.701	.669	.614	?	?	?

Tabla 6.11: Matriz de confusión, clasificador SMO con enfoque en fases (fase 3), entidades clasificadas como Pregunta en la fase 2.

Clasificado como →	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
C1	0	0	0	0	0	0	0	0	0	0	0	0
C2	0	0	0	0	0	0	0	0	0	0	0	0
C3	0	0	0	0	0	0	0	0	0	0	0	0
C4	0	0	0	0	0	0	0	0	0	0	0	0
C5	0	0	0	0	0	0	0	0	0	0	0	0
C6	0	0	0	0	0	0	0	0	0	0	0	0
C7	0	0	0	0	0	0	0	0	0	0	0	0
C8	0	0	0	0	0	0	0	0	0	0	0	0
C9	0	0	0	0	0	0	0	0	0	0	0	0
C10	0	0	0	0	0	0	0	0	0	49	16	0
C11	0	0	0	0	0	0	0	0	0	34	46	0
C12	0	0	0	0	0	0	0	0	0	5	6	0
Precisión (avg=0.579)	0	0	0	0	0	0	0	0	0	.557	.676	0
Recall (avg=0.609)	0	0	0	0	0	0	0	0	0	.754	.575	0
F-measure (avg=0.586)	0	0	0	0	0	0	0	0	0	.641	.622	0
ROC area (avg=0.641)	?	?	?	?	?	?	?	?	?	.663	.643	.500

Tabla 6.12: Matriz de confusión, clasificador SMO con enfoque en fases (fase 3), entidades clasificadas como Respuesta en la fase 2.

6.4 Tiempos de ejecución

Al trabajar con set de datos con cientos de miles de interacciones para clasificar, los tiempos de espera pueden llegar a ser largos, tal vez de varios minutos o hasta horas. Por esto, es importante tener en cuenta estos tiempos a la hora de elegir un clasificador para entrenarlo y luego usarlo como clasificador.

A continuación se verá una comparativa de los tiempos de entrenamiento y clasificación para todos los clasificadores utilizados para nuestra aplicación para los enfoques *Directo* y *en Fases*. Todas las pruebas se realizaron utilizando la misma computadora (MacBook Pro mid 2012, procesador 2,5 GHz Intel Core i5, memoria 4 GB 1600 MHz DDR3) y el mismo dataset.

6.4.1 Tiempo de entrenamiento

Los tiempos de entrenamiento son importantes y tienen relevancia a la hora de elegir el mejor clasificador, pero si se piensa que este entrenamiento se puede realizar una sola vez y luego el modelo entrenado se puede utilizar las veces que se desee, se puede optar por elegir un clasificador que demore mucho tiempo para entrenarse, pero a la hora de clasificar sea rápido y tenga buena precisión.

En la Tabla 6.13 se ven los tiempos de entrenamiento que se necesitaron para generar los modelos de datos utilizando un dataset de entrenamiento con 2149 interacciones. Como se puede ver, los tiempos de entrenamiento varían mucho de un clasificador a otro.

El clasificador más rápido es el *Naive Bayes Multinomial Updateable*, que demora 2 segundos, 20 segundos y 10 segundos para los enfoques directo y en fases respectivamente. Junto con este clasificador, se encuentran otros que necesitan poco tiempo para ser entrenados, estos clasificadores son IBk, SMO y Naive Bayes, que necesitan unos segundos (o unos pocos minutos) para entrenar.

Por otro lado, el clasificador más lento es el *KStar* con 14 horas para el enfoque directo y 21 horas y 4 horas para los enfoques en fases. Junto con este clasificador está KStar, Logit Boost, Decision Table y JRip que se necesitan algunas horas para ser entrenados.

Clasificador	Clasificación directa	Clasificación en fases (un clasificador)	Clasificación en fases (multi-clasificador)
Naive Bayes	230s	340s	162s
Naive Bayes Multinomial Updateable	2s	20s	10s
SMO	63s	113s	67s
IBk	10s	24s	14s
KStar	51759s (+14 horas)	78196s (+21 horas)	16493s (+4 horas)
Logit Boost	16088s (+4 horas)	17122s (+4 horas)	7303s (+2 horas)
Decision Table	14432s (+4 horas)	28274s (+7 horas)	21566s (6 horas)
JRip	13147s (+3 horas)	11841s (+3 horas)	2343s (+39 minutos)
PART	17460s (+4 horas)	34856s (+9 horas)	14974s (+4 horas)
J48	4290s (+71 minutos)	3838 (+63 minutos)	2248s (+37 minutos)

REPTree	486s (+8 minutos)	1000s (+16 minutos)	581s (+9 minutos)
---------	-------------------	---------------------	-------------------

Tabla 6.13: tiempos de entrenamiento por clasificador y por enfoque de entrenamiento.

6.4.2 Tiempo de clasificación

Se notó que a la hora de clasificar, los tiempos eran muy similares entre todos los clasificadores, por lo tanto no se deben tener en cuenta los tiempos para la clasificación. A excepción del clasificador KStar que se demoró hasta cientos de veces más que el resto de los clasificadores. Debido a este extremado tiempo extra que el clasificador KStar se demora, se puede decir que no es apto para la tarea que se desea realizar.

6.5 Conclusión

En este capítulo se presentaron los resultados de clasificación, utilizando los diferentes enfoques, para identificar automáticamente las categorías de colaboración del Análisis del Proceso de Interacción (IPA) de las interacciones de texto libre. Esta es una tarea extremadamente difícil por la gran cantidad de clases en que se puede clasificar una interacción. Aunque una precisión de clasificación del 74,18%, para la clasificación en fases, puede parecer baja a primera vista, es un 30,62% superior a la precisión obtenida por clasificación directa y un 65,85% superior a un clasificador aleatorio (que obtiene un 8,33%). Además, dado que las instancias incorrectamente clasificadas con el enfoque en fases siempre se clasifican dentro de la reacción de IPA correcta, se puede construir un sistema de recomendación para ayudar a quien use el sistema. Este sistema de recomendación puede sugerirle al usuario la lista de posibles categorías en las que se puede reclasificar la interacción teniendo en cuenta la reacción identificada. De esta manera se puede acelerar el tiempo requerido para aplicar el método IPA. Finalmente, el enfoque en fases es útil para detectar problemas de colaboración surgidos en un entorno de trabajo colaborativo soportado por computadoras.

Para finalizar este capítulo se puede concluir que los mejores resultados obtenidos fueron con el clasificador SMO, que es el clasificador que mejor precisión obtuvo y para entrenar y clasificar necesita realmente poco tiempo, en comparación con otros clasificadores. El clasificador IBk también obtuvo muy buenos resultados, un poco más bajos que el SMO.

7. Conclusiones

Al comienzo de esta tesis se hizo la introducción a los estudios de Bales [Bales, 1950], y al modelo IPA, que llevan casi 70 años de ser utilizados y validados. Estos estudios, hoy en día, se continúan utilizando por la comunidad para buscar conflictos en grupos de trabajo colaborativos. También se observó que existen estudios actuales que aseguran que los estudios de Bales también pueden ser aplicados para las conversaciones tenidas a través de un canal de chat grupal utilizando internet [Fahy, 2006]. Estos dos estudios son los que sentaron las bases de esta tesis.

En el presente trabajo también se pudo ver cómo a través de una aplicación se puede agilizar el trabajo de una persona a la hora de detectar conflictos en grupos de trabajo colaborativos, así como también encontrar buenas combinaciones de personas para formar nuevos grupos y que estos conflictos sean minimizados; todo esto con el uso de una aplicación de escritorio creada especialmente para esta tarea y la utilización de los estudios nombrados en el párrafo anterior y varias herramientas, entre las más importantes se encuentran las de análisis de texto en lenguaje natural y las de Machine Learning, FreeLing y WEKA respectivamente. Se detalló un experimento llevado a cabo utilizando la aplicación desarrollada para esta tesis utilizando como datos de entrada los chats de varios grupos de trabajo colaborativo formados por alumnos de una cátedra. Estos alumnos debían resolver cierta tarea y para hacerlo, se les pidió que utilicen una herramienta de chat grupal. Luego de realizar los experimentos, se analizaron los grupos para detectar la posibilidad de que se presente algún conflicto. También se generaron los perfiles de cada persona, lo cual permite la generación de nuevos grupos de trabajo con la finalidad de que se reduzcan estos conflictos.

La primera conclusión que se obtuvo al finalizar este trabajo es que la complejidad de construir una herramienta que sea capaz de clasificar texto natural dentro de las 12 categorías IPA es realmente alta. Si se analiza la eficiencia de clasificadores para dos o tres clases, es común encontrar una precisión superior al 90%. En los experimentos presentados, se observó que el mejor clasificador, utilizando la clasificación en fases, obtuvo resultados cerca del 74%, lo cual mejora significativamente la clasificación directa, que como máximo obtuvo cerca de un 43%. Aún así, son resultados bajos para poder confiar en esta clasificación. Esta complejidad para obtener buenos resultados no es solo por la cantidad de categorías sino que también se da porque se quiere clasificar texto natural que es muy variado, al ser un chat muchas veces se encuentran errores de ortografía y hasta dependiendo el contexto, se puede interpretar una frase como perteneciente a una categoría o a otra completamente opuesta.

Como conclusión general, en base a lo anteriormente expresado, se puede decir que se ha comprobado la utilidad de implementar un algoritmo basado en una clasificación en fases por sobre la clasificación directa. Utilizando una clasificación directa se vuelve muy difícil obtener una clasificación con resultados confiables. Sin embargo, al utilizar la clasificación en fases, se puede pensar que con un poco más de pre-procesamiento de los datos para normalizarlos y un mejor dataset de entrenamiento, se puede mejorar el porcentaje de clasificación para estar cerca del 90% lo cual ya es mucho más alentador.

Habiendo presentado las conclusiones y todos los resultados del trabajo hasta este punto, se torna la atención hacia las limitaciones del mismo y los futuros desarrollos que pueden llevarse a cabo sobre lo producido.

7.1 Limitaciones

En líneas generales, las condiciones de desarrollo del trabajo y las propuestas básicas de las cuales se partió permitieron llevar a cabo un desarrollo exitoso de la funcionalidad a la que se apuntó en los objetivos del mismo. Sin embargo, cabe mencionar en este punto algunas limitaciones encontradas:

- La sección Análisis de Datos de la aplicación es la que más trabajo en potencia tiene. Se podrían buscar mejores formas, más complejas, para crear los perfiles de las personas así como también los perfiles de los grupos (no incluido en este trabajo). También la creación de los grupos se puede mejorar significativamente; ya que ésta, con el algoritmo presentado en el capítulo 5 solo busca nivelar las conductas con más desviación para cada grupo y es muy probable que no se llegue a la mejor solución que se podría encontrar.
- Durante el experimento, se pudo observar que al ejecutar la aplicación con datasets muy grandes (ya sea para entrenar o para clasificar), la aplicación falla por falta de memoria y suspende la ejecución.
- El uso de la herramienta FreeLing presentó muchas dificultades al querer utilizarla en el desarrollo de la aplicación con el lenguaje de programación Java con los sistemas operativos MacOS y Windows. FreeLing fue desarrollada para ser utilizada en sistemas con Linux y con el lenguaje de programación C/C++ y para ser utilizado en este desarrollo fue necesario generar las librerías para cada sistema operativo y un “wrapper” a su API para que se pueda utilizar desde Java. También fue necesario hacer una instalación de la herramienta en la computadora en la que se la quiere utilizar. Si alguien quisiera continuar los estudios de este trabajo, se recomienda eliminar el uso de esta herramienta y reemplazarlo por alguna otra. Por ejemplo, WEKA contiene mucha de la funcionalidad que brinda FreeLing, con la diferencia de que FreeLing está hecha para la lengua española y WEKA no.
- Si se utilizara un dataset de entrenamiento con más interacciones y mejor balanceado (similar cantidad de interacciones para cada categoría), los modelos estarían mejor entrenados y por lo tanto, serían más precisos a la hora de clasificar las interacciones.

7.2 Trabajos futuros

Partiendo del estado actual del proyecto, es posible proponer algunas optimizaciones y extensiones que pueden hacerse sobre él:

- Como se mencionó anteriormente, se sugiere reemplazar la herramienta FreeLing para que sea más simple la utilización de la aplicación, se puedan aprovechar los beneficios de una aplicación escrita en Java y no se tenga que depender de la generación de las librerías para el sistema operativo que se desee utilizar.
- Hoy en día, los modelos entrenados son estáticos, es decir, no se actualizan al obtener nueva evidencia de interacciones de cada clase. Para mejorar en este

aspecto, es necesario cambiar la forma en la que se entrenan los modelos y proveer alguna forma en la aplicación para que el usuario pueda re-clasificar las interacciones mal clasificadas y el modelo aprenda a partir de este feedback. WEKA permite entrenar incrementalmente ciertos clasificadores (para un grupo reducido de clasificadores). Esta forma de entrenar los modelos permite procesar gran cantidad de información utilizando una cantidad de memoria reducida, ya que el conjunto de datos de entrenamiento no necesita entrar en memoria. Para hacer este proceso, primero se debe inicializar el modelo y luego actualizarlo interacción por interacción.

- Es aconsejable ampliar el dataset de entrenamiento para que contenga una buena cantidad de interacciones para cada una de las 12 categorías IPA. De esta forma los modelos estarán mejor entrenados para identificar cualquiera de las categorías.
- Agregar una nueva sección a la aplicación donde se muestre en forma de gráficos los perfiles por persona y por grupo.
- Mejorar la generación de los nuevos grupos de trabajo, con un mejor algoritmo y basado en otros estudios sociológicos. Para esto, es necesario hacer un estudio exhaustivo sobre cómo utilizar los perfiles de personas para poder generar grupos que no tengan conflictos o al menos los minimicen.

Apéndice I: Definición de categorías de Bales (resumen)

1. Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa

Cualquier acto que muestra hospitalidad, ser amistoso/a, manifestar apoyo o similitud de los sentimientos; indicaciones de sentirse atraído/a, demostraciones de afecto; instando a la unidad o armonía; expresando el deseo de cooperación o la solidaridad; mostrando una actitud de protección o querer educar; alabar, premiar, aprobar o alentar a otros; sostener o tranquilizar a una persona que tenga dificultad; elogiar o felicitar; el intercambio de objetos, el comercio, o de préstamo (por ejemplo, una lapicera); confiando en otro; expresar gratitud o aprecio, entregar o ceder la palabra a otro (por ejemplo, cuando se interrumpe); presentación amigable para que otro pueda continuar hablando; confesiones de ignorancia; actos de disculpa, mueca de placer, sonriendo directamente a la otra persona. Que se ofrece para llevar a cabo un trabajo en nombre de los demás, porque tiene el poder o los recursos para hacerlo, y, de esta manera paternalista/maternalista, ofrece servicios, asistencia, tiempo, energía, dinero o algún otro recurso cae en esta categoría.

2. Muestra liberación de tensión, bromea, ríe, muestra satisfacción

Cualquier acto que hace énfasis en el significado o implicaciones emocionales ocultas o es especialmente auto-reveladora acerca de una persona. Con mayor frecuencia, estos son chistes o historias con un doble sentido. Pueden tomar la forma de una anécdota sobre una persona en particular en la que se expresan los sentimientos emocionales, o pueden ser acciones simbólicas (se encoge de hombros) o corporales y expresiones faciales que retratan gran asombro, sorpresa, miedo o ira. Más de un significado es casi siempre implícito en dramatizaciones como aquí se definen; por ejemplo, una postura, la expresión facial, la observación, o los tres juntos pueden implicar: (a) "Desde luego, se cree ser algo" y (b) "No estoy de acuerdo". El tono personal de esta parte dramática, junto con los matices de los sentimientos emocionales parcialmente ocultos, son típicos de los actos anotados en esta categoría. Según Bales, "La broma es una forma muy común de la dramatización en la interacción del grupo. El bromista tiene la expectativa, aunque tal vez no siempre de la manera más clara, de producir un shock de reconocimiento del significado oculto, de provocar una risa, un alivio repentino o muestra de tensión". Bales llegó a sugerir que el concepto que el bromista ofrece, es que cuando alguien se ríe es porque admite "la verdad oculta". La cualidad esencial de esta categoría es la que cuando un significado especial, personal, parcialmente oculto, es sutilmente expuesto de una manera que es emocionalmente liberado, aunque de forma arriesgada; por lo tanto, una característica común de tales actos es que parece que tienen dos significados, uno peligroso por exponer y el otro un poco de diversión en la superficie. Comportamiento como este puede parecer ser peligroso o mejor evitado; sin embargo, Bales hace este comentario evaluativo: "En cuanto a los servicios psicológicos realizados, y la importancia general en el grupo, así como la vida individual, estas actividades no están orientadas a la tarea, pero no dejan de ser un asunto serio psicológico."

La actitud puede, por supuesto, ser por alivio. Expresiones de sentirse mejor después de un período de tensión, cualquier manifestación de alegría, liviandad, satisfacción, placer, entusiasmo o felicidad; cualquier indicio de que el miembro se ha emocionado, se puso eufórico, o similares, puede actuar como un liberador de tensión para los demás y, por lo tanto, están adecuadamente clasificados en esta categoría.

3. Concuerda, muestra aceptación pasiva, comprende, coincide, obedece

Cualquier acto que muestra armonía, asentimiento sobre los hechos, inferencias e hipótesis.

Las formas menores incluyen dar signos específicos de atención a lo que el otro está diciendo, como un medio de animar a la otra persona a que diga lo que desea, con un movimiento de la cabeza, diciendo "Ya veo", "Sí", o "Ajá "; completando lo que el otro está diciendo al decir la palabra que la otra persona busca o no se atreve a decir, o de otra manera ayudar y facilitar la comunicación. Otras formas menores incluyen mostrando comprensión o entendimiento, después de un período de no comprender (por ejemplo, "Oh!", "Ya veo" y "Sí").

Las formas más importantes tienen una implicación más vinculante o contractual, que se dan como queriendo comprometer al miembro que está de acuerdo con el contenido sustancial de lo que se ha dicho, y como si fueran a ser invocados después. El acuerdo puede ser por información, opinión o sugerencia. Puede expresar confirmación, convicción, armonía, asentimiento, sobre los factores de asentimiento, inferencias e hipótesis. Los ejemplos incluyen: "Esa es la forma en la que lo veo también"; "Creo que tiene razón en eso"; "Sí, eso es verdad"; y "Precisamente". Del mismo modo, la categoría incluye la aprobación o respaldo de expresiones de valor, sensación o un sentimiento; por ejemplo, "Me siento de la misma manera que tu", "Yo también lo espero", "Esos son mis sentimientos exactamente," y "Eso es correcto". Acuerdos no verbales pueden implicar asintiendo con la cabeza, mostrando especial interés, o mostrar atención significativamente visibles a lo que se dice.

4. Da sugerencia, indicación, da autonomía al otro

Cualquier acto que va en la dirección la tarea. Esta categoría incluye el control de la rutina de la comunicación y dirigir la atención del grupo a los problemas de la tarea cuando han sido acordados por el grupo. Por lo tanto, mencionar un problema a tratar, señalando la relevancia de una observación, llamar a una reunión para ordenarse, hacer referencia a la agenda y la apertura de una nueva fase de actividad, todos estos entran en esta categoría si son rutina, se pusieron de acuerdo y si son mencionados de una manera que implica la aceptación del disenso si alguien así lo desea.

Por ejemplo, "Mira atentamente ahora" y "A continuación, quiero que se fijen en..." Llamar a una reunión al orden, refiriéndose de nuevo a una agenda, la entrega de cualquier señal de rutina si uno empieza una nueva fase de actividad o un nuevo foco de esfuerzo, que significa el final de una fase de actividad o en la pronunciación de la junta aplazada; estos son claros ejemplos de sugerencias de procedimiento.

En general, los intentos directos para orientar o aconsejar, o preparar otro miembro para alguna actividad, para prevalecer sobre él o ella, para persuadirlo/a, exhortar a la persona, incitar, imponer, o inspirarlo/a para alguna acción, por la dependencia en la autoridad o ascendencia en lugar de por inferencias lógicas son ejemplos de esta categoría.

En el presente sistema, sin embargo, las sugerencias deben ser neutrales; es decir, ni negativas ni positivas en sentimiento para ser clasificadas en esta categoría. Una sugerencia debe ser clasificada en la categoría "*12. Muestra antagonismo, rebaja el estado de los demás, defiende o afirma el suyo*" si cualquier sentimiento negativo es detectable. De la misma manera, hay que anotarlo en la categoría "*1. Muestra solidaridad, eleva el estado de los demás, da ayuda, recompensa*" si algún sentimiento positivo apreciable está presente. Esta clasificación está destinada a hacer las categorías 1 y 12 más sensibles, y dejar en la categoría "*4. Da sugerencias, indicaciones, implicando autonomía al otro*" sólo aquellas sugerencias que mantienen una sensación estrictamente neutral.

5. Da opinión, evaluación, análisis, expresa sentimiento, desea

Esta categoría es la más común entre las categorías de interacción. Es la categoría más frecuentemente usada en muchas situaciones de observación e incluye muchos de esos tipos de actos por los cuales el grupo consigue hacer su "trabajo" (su solución al problema, toma de decisiones y trabajo legislativo y administrativo). En esta categoría se busca incluir sólo actos que tienen implicaciones estrictamente neutrales, serias y objetivas. El tipo de acciones que parecen ser más congruentes con esta categoría son frases de obligación moral, afirmaciones de convicciones o valores, frases de política, intención, guía de principios o leyes. Referencia a una perspectiva futura lejana e indefinida, aún no implementada. Por ejemplo: "Deseo que lo podamos arreglar así podemos...", "Creo que debemos ser justos sobre esto...", "Espero que podamos hacer algo al respecto...", "Yo lo veo bien...", "Por supuesto, siempre hemos adherido a una política de...", "Siempre ha sido mi creencia que..." y "En este grupo, nosotros...".

Esta categoría también incluye cualquier acto que involucra una obligación moral, ofrece una creencia superior o valor, o indica adherencia a una política o guía de principios. Tales actos deben ser serios pero no personales, sincero pero objetivo. Esta categoría incluye expresiones de entendimiento o percepción aparte de esos juicios de valores, como "Creo que entiendo tu punto", "Creo que debemos reconocer nuestra obligación de..." o "Me parece que vamos por buen camino". Si un acto no es serio o sincero, debe ser clasificado en la categoría 2. La categoría 5 debe ser distinguida de la categoría 6 principalmente por la base que su uso es por inferencia/conclusiones y juicio de valores.

6. Da orientación, información, repetición y confirmación

Cualquier acto de reportar hechos (no necesariamente ciertos) u observaciones o experiencias potencialmente verificables (probables). Probablemente los casos más claros de dar información son frases que son reconocidas como generalmente establecidas o fáciles de confirmar por observación. Casos comunes de esta categoría son reportes sobre situaciones problemáticas enfrentando al grupo, como "La legislatura no ha actuado todavía sobre esa cuenta", "Ellos, por lo general, preparan café para nosotros en la sala", "El teléfono no funciona", "El número es 567-445", "Va a llevar 3 días que le llegue la carta", "Ellos recogen el correo a las 5:30" y "Solo nos quedan dos días". Estas sentencias esencialmente son objetivas, sin deducción o interpretación, sin tonos emocionales, no vagas, y son, en principio, probables.

La información por lo general es dada gratuitamente, en longitud, como en una lectura, o en el proceso de transmitir conocimiento en un contexto donde la implicación es que la otra persona quiere conocer algo que el orador le puede decir. La expectativa es que la información dada será aceptada, si es entendida. No es ofrecida si fuera problemática, o un tema de decisión, a pesar de que, de hecho, muchos actos de dar información provocan desacuerdo. Al dar opiniones (categoría 5), la expectativa usualmente es menos optimista

(una cantidad mayor de desacuerdos son esperados al dar opiniones que en el caso de dar información, y la expectativa usualmente es justificada). Cualquier sentencia demasiado vaga en principio a ser probada no es clasificada en esta categoría sino, usualmente es clasificada en la categoría 5.

7. Pide orientación, información, repetición y confirmación

Preguntas que solicitan hechos, descriptivas, tipos de respuestas objetivas, una respuesta basada en experiencia, la observación o investigación empírica. Si tal tipo de respuesta es imposible, requiere adivinar, suponer, esperar el tiempo a que sucedan eventos que todavía no han ocurrido, o similares, la pregunta debe ser clasificada en la categoría 8 y no en esta categoría. Las preguntas pueden ser sobre situaciones o tareas que el grupo enfrenta, sobre el grupo en sí mismo (por ejemplo su estructura u organización), sobre otra persona, sobre uno mismo, sobre lo que se ha sido dicho o hecho el proceso de comunicación actualmente sucediendo. La pregunta puede ser directa (por ejemplo “¿Qué día del mes es?” o “¿Qué Dijiste?”) o indirecta (Por ejemplo: si, en el curso de una descripción, una persona dice “No estoy seguro sobre la fecha del examen” esta puede ser considerada una pregunta indirecta o implícita; “Me olvide a quien anotamos”; o “No te escuche”). Cualquier pedido de rutina de repetición apropiadamente es clasificado en esta categoría.

Esta categoría tiene la intención de incluir primariamente aquellas preguntas que requieren una respuesta de hechos simple más que una evaluación, una inferencia o la expresión del sentimiento. Preguntar por información es preguntar por una respuesta que puede ser juzgada como verdadera o falsa con la simple observación (aunque no necesariamente posible en el momento), o una respuesta que es considerada generalmente un hecho empírico es probablemente aceptada como no problemática una vez dada y entendida.

8. Pide opinión, evaluación, análisis, expresión de sentimientos

Cualquier tipo de acto que busca una interpretación, la sentencia que involucra creencias o actitudes, juzgar un valor, o el reporte de entendimiento propio. Puede incluir el pedido de diagnóstico de una situación o una reacción a una idea.

Este tipo de categoría incluye cualquier tipo de preguntas que intentan animar una sentencia de reacción sobre la parte de otra persona sin limitar la naturaleza de la respuesta, excepto en un caso muy general, con la implicación de que la otra persona tiene libertad de expresar sus intereses y desintereses, donde no es puesto bajo presión para estar de acuerdo o en desacuerdo, para decir cualquier frase predeterminada, tipo de respuesta o actitud.

Aquí hay que tener cuidado con preguntas como “¿Entiendes lo que digo?” y “¿Lo ves?”. Estos son ejemplos de intentos de búsqueda de acuerdo y debe ser identificado como esfuerzo persuasivo, debe ser clasificado en la categoría 4.

Otro problema ocurre cuando un líder elegido maneja el grupo en formas que otros miembros del grupo le encargan algo y el líder lucha por entender y cumplir con sus deseos. En tal caso, el líder debe preguntar por una opinión de esta manera “¿Te gustaría tener un comité para trabajar sobre eso?”. Sí, por otro lado, el líder pregunta “¿Qué deberíamos hacer para incrementar nuestra membresía?”, el líder está preguntando por sugerencias con respecto a la forma de resolver un problema del grupo, consecuentemente, este tipo de preguntas deben ser identificadas la categoría 9.

9. Pide sugerencia, indicación, posibles caminos de acción

Cualquier acto donde se pida una guía en el proceso de resolución de problemas, es neutral en tono emocional e intenta volcar iniciativa sobre el otro. A menudo, tales actos indican sentimiento de confusión o inseguridad sobre la posición en el grupo con respecto a sus objetivos, el camino de la discusión para presentar el punto, o sobre lo que ha sido dicho o está sucediendo (ej. "¿Dónde estamos?", "¿Dónde nos paramos ahora?", "No sé qué hacer"). Para encajar en esta categoría correctamente, el pedido debe ser "de final abierto", sin la implicación de una respuesta específica (ej. "¿Qué creen que deberíamos hacer ahora?"). Sí, sin embargo, la pregunta es hecha de cierta manera que la respuesta específica es implicada, debería ser clasificada en la categoría 4 (un ejemplo de la sugerencia encubierta es "Me pregunto si hay alguna otra manera de conseguir la información de la legislatura" porque parece que implica que hay otras maneras y sugiere que sean consideradas).

10. Discrepa (está en desacuerdo), muestra rechazo pasivo, formalidad, retiene recursos

Cualquier acto en secuencia que rechaza una frase de información de otra persona, opinión o sugerencia. Es una reacción a la acción de otra persona, definida por Bales como: "El sentimiento negativo comunicado es adjuntado al contenido de lo que ha dicho otra persona, no a él o ella como persona. El sentimiento negativo no debe ser muy fuerte, o el acto sería antagonista." (en tal caso sería calificado en la categoría 12).

Frases que siguen el rechazo inicial de otra persona, como discusiones, refutaciones y preguntas (ej. "No lo creo" y "No creo que eso esté bien"), no son clasificadas en esta categoría, sino que son clasificadas en otras categorías.

11. Muestra tensión, pide ayuda, deja de prestar atención

Cualquier acto que muestra conflicto entre sumisión e inconformidad, y al mismo tiempo no muestran claramente sentimientos negativos hacia la otra persona. Mucha variedad de actos son clasificados en esta categoría, no todos esos parecen ser similares en un nivel superficial. Risas, en particular, parece ser muy diferentes de actos de ansiedad emocional.

Actos de ansiedad emocional indican un conflicto entre actuar y no actuar. Pequeños brotes de ansiedad reactiva primero deben ser mencionados, como una aparición sorpresiva, desconcertado, nervioso, confundido, temblando, apretando una lapicera, poniéndose colorado, balbuceando, transpirando, tragando, o humedeciéndose los labios persistentemente también puede ser incluida.

Hay varios tipos de conflicto. A veces miedos a conflictos de desaprobación con deseos instintivos o impulsivos. A veces la tendencia a conformar a un grupo de valores o exigencias autoritarias entra en conflicto con otro grupo de valores o exigencias autoritarias. La persona en conflicto puede contener, ocultar o fallar a mencionar algo que es considerado desacreditable, como pueden ser inaptitudes, ignorancia, un defecto, un crimen, un trato o evento humillante. Indicios de retención como estos son clasificados en esta categoría.

Las risas son muy importantes en esta categoría. En la superficie, la risa puede parecer indicar reducción de tensión, y puede, en parte, servir para ese propósito. La emoción puede ser ansiedad, agresión, afección, o cualquier otra. La risa sirve

momentáneamente para romper la tensión, y si es continuada, puede ayudar a reducir el estado de tensión general, pero igualmente es apropiado tratarlo como un signo de tensión.

Un comportamiento adicional para ser incluido en esta categoría es cualquier reacción embarazosa para desaprobación, como en la apariencia de ser mortificado o castigado. Mostrar tensión es, más generalmente, cualquier expresión verbal o corporal de miedo, aprehensión, preocupación, pavor, susto, terror o pánico.

12. Muestra antagonismo/oposición, rebaja el estado de los demás, defiende o afirma el suyo

Cualquier acto que es personalmente negativo, no está orientado por el contenido, que sería clasificado como "desacuerdo" cuando es negativo, pero está orientado hacia otra persona.

Muchos intentos para tapar al otro en la conversación, interrumpir al otro, interferir cuando habla, gratuitamente finalizar sus oraciones cuando no necesita ayuda, insistir en que termine, evitar interrupciones son todos ejemplos de esta categoría.

Hay un gran número de variedades de ataques para rebajar el estado de los demás, yendo de suave a violento. Cualquier implicación de inferioridad o incompetencia de una parte sobre la otra es incluida. Evaluar a la otra persona con desprecio, menosprecio, de forma despectiva, ridiculizando, o minimizando al otro, reduciendo sus observaciones a lo absurdo, y burlarse de la otra persona son incluidas. Actos más extremos pueden ser descritos como sarcástico, satíricos o irónicos, en la cual la persona caricaturiza, se burla de la otra persona o la insulta están también incluidos. Burlas, abucheos, regodeo, desprecio, acoso, incitar, provocar la otra persona para decir algo indiscreto o perjudicial están incluidos. Maldecir al otro, buscar errores, quejarse, criticar; cualquier acto que sea abusivo, acusatorio está incluido. Hacer acusaciones contra el otro, culpar a la persona, denunciar, reprender, malos tratos, intimidar al otro todos están, por supuesto, incluidos. La categoría incluye cualquier acto de chisme, difamación, calumnia; marcar características indeseables, degradar a la otra persona, exponer, menoscabar su posición; difamar o desacreditar a la persona y colocarla en desventaja. Otros actos similares incluyen engañar, embaucar, humillar al otro.

Un buen número de actos en los que la persona que trabaja contra la autoridad o las personas con autoridad se incluyen: mostrando falta de sumisión ante la autoridad; no conformidad o la independencia excesiva; actos que desde el punto de vista de la persona con autoridad son vistos como rebeldes, insubordinados, irresponsable, intencionada, turbulenta, sin restricciones, quejarse, regañar, acosar, molestar, perturbar. En la misma categoría entran los actos que parecen irrespetuosos, descorteses, descarados, atrevidos, impertinentes y actitudes que parecen sin vergüenza o arrepentimiento.

Esta categoría también incluye los actos en los que hay un intento de controlar, regular, gobernar, dirigir o supervisar de una manera que parece arbitrario y en el que la libertad de elección o el consentimiento de la otra persona, ya sea en gran medida, limitada o inexistente, con la implicación de que la otra persona no tiene derecho a protestar o modificar la demanda, pero se espera que siga la directiva de inmediato y sin argumento. La categoría incluye de forma arbitraria la asignación de un papel, la localización o la reubicación de la otra, definir o restringir sus poderes, exigir o dar órdenes, como "¡Ven aquí!", "¡Deja de hacer eso!" "Date prisa" y "¡Vete!". Todo acto en el que la persona da ultimátums, apunta, presiona o de otro modo controla directamente la actividad de la otra persona puede ser incluido (a menos que, por supuesto, es de rutina y, por lo tanto, neutro).

La intervención en una discusión o conflicto con los intentos arbitrarios para juzgar o resolver o para tomar una decisión se incluye en esta categoría. Los aspectos legalistas moralistas de la conducta relevante también se muestran en la desaprobación, incluidos los

actos que van desde las formas leves, tales como reprender a la otra persona, culpar, o regañando, amonestando o recordando a la persona de su deber; a formas más extremas, como las indicaciones que uno se sorprende, indigna, horroriza, escandaliza en algo que el otro ha hecho y muestra el error o disgusto. Los indicios de que uno está indignado, ofendido, insultado o sobre una ofensa personal se incluyen. Indicios de una indignación moral más generalizada, como una expresión sombría, apareciendo indignados, o furiosos por alguna desviación de la otra persona están incluidos. Prestar atención a algo que no sea la actividad que constituye el objetivo del grupo, cuando existe la expectativa de que todos los miembros presten atención o participen de forma activa, está incluido. Hablar o prestar atención a personas fuera del grupo de trabajo, como observadores, cuando el grupo en su conjunto está trabajando en otro problema se incluye. Todo contacto indeterminado entre miembros del grupo; es decir, que se vayan a otro lado, que susurren, hacerse guiños y así sucesivamente, mientras la discusión principal está pasando entre otros pueden indicar rechazos por parte de ambos participantes al resto del grupo.

Actos de rechazos pasivos que frustren al otro pueden ser incluidos; actos en los que uno frustra, obstruye o pone obstáculos en el camino de los otros, confina, obliga, o se interpone en el camino del otro, hace que sus esfuerzos sean en vano, trastorna sus planes, se anticipa, contraviene, o pone entre la espada y la pared a la otra persona. Esta categoría también incluye los actos de retener recursos de otros miembros, la manifestación de posesivo, retracción o actitudes secretas. Todo acto en el que una persona defrauda a otra, se niega a dejar que la otra persona participe, o tener acceso a algún recurso puede ser incluido aquí.

Acciones o mostrar actitudes que indican que la persona no está atenta, aburrida o no presta atención a los demás y del problema en cuestión están incluidos, como encorvarse, bostezar, cerrar los ojos, soñar despierto, mirar a otro lado y no donde están los otros en el grupo, mirar para otro lado y no donde el trabajo sucede y dejar que los ojos se pierdan. Actitudes donde la persona parece apática, lánguida, desconcertada, absorbida o ajena a los demás se califican en esta categoría. Se incluyen otros actos definidos y manifiestos de ausencia, como retirarse, renunciar, abandonar o alejarse también son incluidas. Las formas más extremas de comportamiento autista, subjetivo, o socialmente irrelevante que indican una falta de contacto con lo que está pasando se pueden incluir, por ejemplo, hablar con uno mismo o entre dientes. Negarse a hablar lo suficientemente fuerte para ser escuchado también se anota en esta categoría.

Apéndice II: Etiquetas EAGLE usadas por FreeLing

A continuación se presentan, en formato de tablas, las etiquetas EAGLE de la forma en que FreeLing las utiliza.

Adjetivos

Posición	Atributo	Valor	Código
1	Categoría	Adjetivo	A
2	Tipo	Calificativo	Q
		Ordinal	O
3	Grado	Aumentativo	A
		Diminutivo	D
		Comparativo	C
		Superlativo	S
4	Género	Masculino	M
		Femenino	F
		Común	C
5	Número	Singular	S
		Plural	P
		Invariable	N
6	Función	-	0
		Participio	P

Tabla A2.1: Etiqueta EAGLE/FreeLing para adjetivos

Adverbios

Posición	Atributo	Valor	Código
1	Categoría	Adverbio	R
2	Tipo	General	G
		Negativo	N

Tabla A2.2: Etiqueta EAGLE/FreeLing para adverbios

Determinantes

Posición	Atributo	Valor	Código
1	Categoría	Determinante	D
2	Tipo	Demostrativo	D
		Posesivo	P
		Interrogativo	T
		Exclamativo	E
		Indefinido	I
		Artículo	A
3	Persona	Primera	1
		Segunda	2
		Tercera	3
4	Género	Masculino	M
		Femenino	F
		Común	C
		Neutro	N
5	Número	Singula	S
		Plural	P
		Invariable	N
6	Poseedor	Singular	S
		Plural	P

Tabla A2.3: Etiqueta EAGLE/FreeLing para determinantes

Nombres

Posición	Atributo	Valor	Código
1	Categoría	Nombre	N
2	Tipo	Común	C
		Propio	P
3	Género	Masculino	M
		Femenino	F
		Común	C
4	Número	Singula	S
		Plural	P
		Invariable	N
5-6	Clasificación semántica	Persona	SP
		Lugar	G0
		Organización	O0
		Otros	V0
7	Grado	Aumentativo	A
		Diminutivo	D

Tabla A2.4: Etiqueta EAGLE/FreeLing para nombres

Verbos

Posición	Atributo	Valor	Código
1	Categoría	Verbo	V
2	Tipo	Principal	M
		Auxiliar	A
		Semiauxiliar	S
3	Modo	Indicativo	I
		Subjuntivo	S
		Imperativo	M
		Infinitivo	N
		Gerundio	G
		Participio	P
4	Tiempo	Presente	P
		Imperfecto	I
		Futuro	F
		Pasado	S
		Condicional	C
		-	0
5	Persona	Primera	1
		Segunda	2
		Tercera	3
6	Número	Singular	S
		Plural	P
7	Género	Masculino	M
		Femenino	F

Tabla A2.5: Etiqueta EAGLE/FreeLing para verbos

Pronombres

Posición	Atributo	Valor	Código
1	Categoría	Pronombre	P
2	Tipo	Personal	P
		Demostrativo	D
		Posesivo	X
		Indefinido	I
		Interrogativo	T
		Relativo	R
		Exclamativo	E
3	Persona	Primera	1
		Segunda	2
		Tercera	3
4	Género	Masculino	M
		Femenino	F
		Común	C
		Neutro	N
5	Número	Singular	S
		Plural	P
		Impersonal/Invariable	N
6	Caso	Nominativo	N
		Acusativo	A
		Dativo	D
		Oblicuo	O
7	Poseedor	Singular	S
		Plural	P
8	Cortesía	Cortés	P

Tabla A2.6: Etiqueta EAGLE/FreeLing para pronombres

Conjunciones

Posición	Atributo	Valor	Código
1	Categoría	Conjunción	C
2	Tipo	Coordinada	C
		Subordinada	S

Tabla A2.7: Etiqueta EAGLE/FreeLing para conjunciones

Interjecciones

Posición	Atributo	Valor	Código
1	Categoría	Interjección	I

Tabla A2.8: Etiqueta EAGLE/FreeLing para interjecciones

Preposiciones

Posición	Atributo	Valor	Código
1	Categoría	Adposición	S
2	Tipo	Preposición	P
3	Forma	Simple	S
		Contraída	C
4	Género	Masculino	M
5	Número	Singular	S

Tabla A2.9: Etiqueta EAGLE/FreeLing para preposiciones

Puntuación

Posición	Atributo	Valor	Código
1	Categoría	Puntuación	F

Tabla A2.10: Etiqueta EAGLE/FreeLing para puntuación

Numerales

Posición	Atributo	Valor	Código
1	Categoría	Cifra	Z
2	Tipo	Partitivo	d
		Monera	m
		Porcentaje	p
		Unidad	u

Tabla A2.11: Etiqueta EAGLE/FreeLing para numerales

Fechas y horas

Posición	Atributo	Valor	Código
1	Categoría	Fecha/Hora	W

Tabla A2.12: Etiqueta EAGLE/FreeLing para fechas y horas

Apéndice III: Formato ARFF utilizado por WEKA

ARFF (Attribute-Relation File Format)

WEKA define un formato especial de archivos que serán los archivos de entrada y salida de esta herramienta. Un archivo ARFF (formato de archivo de atributo-relación) es un archivo de texto ASCII que describe una lista de instancias que comparten un conjunto de atributos.

Los archivos ARFF tienen dos secciones distintas. La primera sección es la información del **encabezado**, que es seguida por la información de **datos**.

El encabezado del archivo ARFF contiene el nombre de la relación, una lista de los atributos (las columnas en los datos) y sus tipos. Un encabezado de ejemplo en el conjunto de datos IRIS estándar se parece a lo mostrado en la Tabla A3.1.

```
% 1. Title: Iris Plants Database
%
% 2. Sources:
%   (a) Creator: R.A. Fisher
%   (b) Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
%   (c) Date: July, 1988
%
@RELATION iris

@ATTRIBUTE sepallength NUMERIC
@ATTRIBUTE sepalwidth NUMERIC
@ATTRIBUTE petallength NUMERIC
@ATTRIBUTE petalwidth NUMERIC
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
```

Tabla A3.1: ejemplo de encabezado de un archivo ARFF.

La sección de datos del archivo ARFF se parece a lo que se puede ver en la Tabla A3.2.

```
@DATA
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
5.4,3.9,1.7,0.4,Iris-setosa
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
```

Tabla A3.2: ejemplo de la sección de datos de un archivo ARFF.

Las líneas que comienzan con un % son comentarios. Las declaraciones @RELATION, @ATTRIBUTE y @DATA no distinguen entre mayúsculas y minúsculas.

Sección de encabezado ARFF

La sección de encabezado ARFF del archivo contiene la declaración de relación y las declaraciones de atributo.

Declaración de @relation

El nombre de la relación se define como la primera línea en el archivo ARFF. El formato es:

```
@relation <relation-name>
```

Tabla A3.3: formato de @relation en un archivo ARFF.

donde <relation-name> es un texto. El texto debe estar entre comillas si el nombre incluye espacios.

Declaraciones @attribute

Las declaraciones de atributos toman la forma de una secuencia ordenada de declaraciones @attribute. Cada atributo en el conjunto de datos tiene su propia declaración de @attribute que define de manera única el nombre de ese atributo y su tipo de datos. El orden en que se declaran los atributos indica la posición de la columna en la sección de datos del archivo. Por ejemplo, si un atributo es el tercero declarado, WEKA espera que todos los valores de los atributos se encuentren en la tercera columna delimitada por comas.

El formato para la declaración @attribute es:

```
@attribute <attribute-name> <datatype>
```

Tabla A3.4: formato de @attribute en un archivo ARFF.

donde el <attribute-name> debe comenzar con un caracter alfabético. Si se quieren incluir espacios en el nombre, el nombre completo debe estar entre comillas.

El <datatype> puede ser cualquiera de los cuatro tipos soportados por WEKA:

- *numeric*
 - *integer* se trata como *numeric*
 - *real* se trata como *numeric*
- <nominal-specification>
- *string*
- *date* [<date-format>]

donde <nominal-specification> y <date-format> se definen a continuación. Las palabras clave **numeric**, **real**, **integer**, **string** y **date** no distinguen entre mayúsculas y minúsculas.

Atributos numéricos

Los atributos numéricos pueden ser números reales o enteros.

Atributos nominales

Los valores nominales se definen proporcionando una *<nominal-specification>* que enumera los valores posibles: {<nominal-name1>, <nominal-name2>, <nominal-name3>, ...}

Por ejemplo, el valor de clase del conjunto de datos *Iris* se puede definir de la siguiente manera (Tabla A3.5):

```
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
```

Tabla A3.5: formato de @attribute para atributos nominales en un archivo ARFF.

Los valores que contienen espacios deben estar entre comillas.

Atributos de cadena de texto

Los atributos de cadena de texto permiten crear atributos que contienen valores textuales arbitrarios. Esto es muy útil en aplicaciones de minería de textos, ya que se pueden crear conjuntos de datos con atributos de cadena y luego escribir filtros WEKA para manipular cadenas (como `StringToWordVectorFilter`). Los atributos de cadena se declaran de la siguiente manera (Tabla A3.6):

```
@ATTRIBUTE LCC string
```

Tabla A3.6: ejemplo del formato de @attribute para atributos de cadenas de texto en un archivo ARFF.

Atributos de fecha

Las declaraciones de atributos de fecha toman la forma (Tabla A3.7):

```
@attribute <name> date [<date-format>]
```

Tabla A3.7: formato de @attribute para atributos de tipo fecha en un archivo ARFF.

donde <name> es el nombre del atributo y <date-format> es una cadena opcional que especifica cómo deben analizarse e imprimirse los valores de fecha (este es el mismo formato utilizado por `SimpleDateFormat`). La cadena de formato predeterminada acepta el formato combinado de fecha y hora ISO-8601: `yyyy-MM-dd HH:mm:ss`.

Sección de datos ARFF

La sección de datos ARFF del archivo contiene la línea de declaración de datos y las líneas de instancia reales.

Declaración @data

La declaración @data es una sola línea que indica el inicio del segmento de datos en el archivo. El formato es (Tabla A3.8):

```
@data
```

Tabla A3.8: comienzo de la sección de datos en un archivo ARFF.

Datos de instancia

Cada instancia se representa en una sola línea. Los valores de atributo para cada instancia se pueden delimitar mediante comas o tabulaciones. Una coma/tabulación puede ir seguida de cero o más espacios. Los valores de atributo deben aparecer en el orden en que se declararon en la sección de encabezado.

Un valor faltante se representa con un solo signo de interrogación, como en (Tabla A3.9):

```
@data
4.4,?,1.5,?,Iris-setosa
```

Tabla A3.9: ejemplo de formato una instancia en un archivo ARFF.

Los valores de la cadena y los atributos nominales distinguen entre mayúsculas y minúsculas, y cualquiera que contenga espacio debe estar entre comillas de la siguiente manera (Tabla A3.10):

```
@relation LCCvsLCSH

@attribute LCC string
@attribute LCSH string

@data
AG5, 'Encyclopedias and dictionaries.;Twentieth century.'
AS262, 'Science -- Soviet Union -- History.'
AE5, 'Encyclopedias and dictionaries.'
AS281, 'Astronomy, Assyro-Babylonian.;Moon -- Phases.'
AS281, 'Astronomy, Assyro-Babylonian.;Moon -- Tables.'
```

Tabla A3.10: ejemplo de un archivo ARFF.

Las fechas deben especificarse en la sección de datos utilizando la representación de cadena de texto especificada en la declaración de atributo. Por ejemplo (Tabla A3.11):

```
@RELATION Timestamps

@ATTRIBUTE timestamp DATE "yyyy-MM-dd HH:mm:ss"

@DATA
"2001-04-03 12:12:12"
"2001-05-03 12:59:55"
```

Tabla A3.11: formato de las fechas en un archivo ARFF.

Apéndice IV: Manual de usuario

En el este apéndice se verá cómo se presenta la aplicación al usuario y se explicará cómo utilizarla. También se mostrarán las dos formas de procesar los datos, en la primera el usuario tiene menos control sobre las opciones que puede elegir pero se ejecuta de manera más rápida; por otro lado, la segunda presenta más opciones al usuario para que elija y configure los clasificadores, pero la aplicación se demora más tiempo en clasificar. Al final del apéndice se hablará sobre la sección que le permite al usuario ver los resultados del análisis de los datos y crear nuevos grupos de trabajo colaborativo.

Secciones de la aplicación

La aplicación cuenta con 3 secciones principales presentadas en 3 pestañas distintas, estas son:

- Procesar datos (simple)
- Procesar datos (avanzado)
- Análisis de datos

Cada una de estas secciones le permite al usuario hacer operaciones distintas. Las 2 primeras, *Procesar datos (simple)* y *Procesar datos (avanzado)*, son muy similares en apariencia y funcionamiento, son las que le permiten al usuario clasificar un dataset, la diferencia está en que la primera sección el usuario tiene menos opciones para elegir que la segunda. A continuación se describe cada una de las secciones de la aplicación y cómo usarlas.

Procesar datos (simple)

Esta es la primera sección que se le presenta al usuario al iniciar la aplicación, vea la Figura A4.1. En esta sección se van a utilizar unos modelos de datos generados previamente, explicado en el capítulo 4.4.3 *Caso especial: Entrenamiento Automático*.

Si se compara esta sección con *Procesar datos (avanzado)* (que será presentada más adelante), en esta sección se verán menos elementos con los que el usuario puede interactuar. Es por esto que, en esta sección, el usuario solo podrá seleccionar el archivo que desea clasificar, el enfoque de clasificación (directa o en fases) y el clasificador que desea usar. Al ya tener generados los modelos de los clasificadores, clasificar utilizando esta sección es mucho más rápido, pero menos flexible que la sección *Procesar datos (avanzado)*.

Los archivos admitidos por la aplicación son archivos con extensión:

- ARFF
- JSON
- XLS o XLSX

Luego, se debe seleccionar el tipo de clasificación que se va a realizar, aquí se pueden ver 5 pestañas que permiten elegir entre 3 enfoques de clasificación:

- Directo

- Fases
- Fase 1, Fase 2 y Fase 3

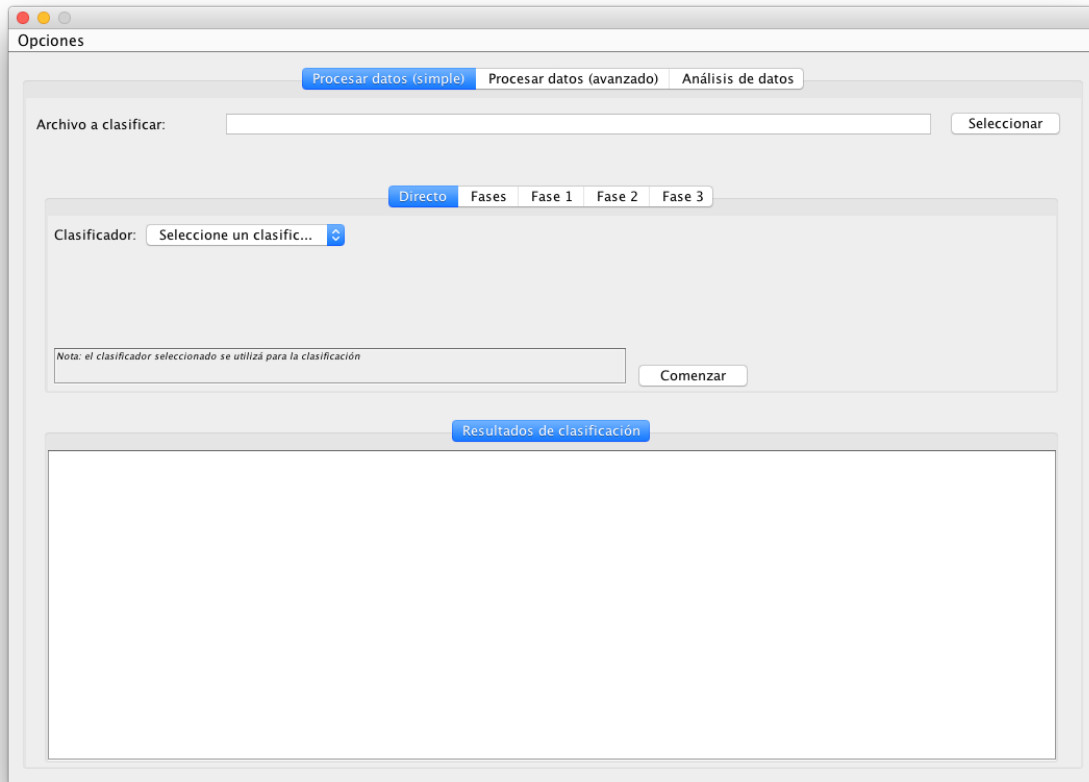


Figura A4.1: sección *Procesar datos (simple)* de la aplicación, procesamiento *Directo*.

Directo

Clasificando de esta manera el usuario solo tiene la opción de elegir un clasificador dentro de una amplia lista de clasificadores y el dataset a clasificar, vea la Figura A4.1. Por último solo queda presionar el botón “Comenzar” para que la aplicación comience a clasificar el archivo seleccionado; al finalizar, presentará los resultados por pantalla y también generará un archivo de salida con los resultados.

El archivo de salida será un ARFF y tendrá el siguiente formato mostrado en la Tabla A4.1:

```
@relation [relación]
```

```
...
```

```
@data
```

```
[Número de categoría de Bales], [Nombre], [Mensaje]
```

```
[Número de categoría de Bales], [Nombre], [Mensaje]
```

```
[Número de categoría de Bales], [Nombre], [Mensaje]
```

Número de categoría de Bales: 1-12

Nombre: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla A4.1: Formato de salida de la clasificación directa.

Fases

Utilizando esta forma de clasificar, al igual que lo explicado en la subsección anterior, el usuario solo puede elegir un único clasificador y el dataset a clasificar. Luego de presionar “Comenzar” se hará la clasificación, se presentarán los resultados y se generará un archivo con los resultados. Visualmente esta subsección es casi idéntica a la subsección explicada anteriormente, vea la Figura A4.2. La diferencia se encuentra en la forma en que se procesan los datos, como se explicó anteriormente. Eligiendo esta opción, el archivo a clasificar será clasificado 3 veces (por medio de archivos temporales intermedios) utilizando siempre el mismo clasificador.

- La primera fase generará un archivo temporal que será utilizado por la siguiente fase.

El primer archivo intermedio tendrá el siguiente formato, Tabla A4.2:

```
@relation [relación]
```

```
...
```

```
@data
```

```
[Área], [Nombre], [Mensaje]
```

```
[Área], [Nombre], [Mensaje]
```

```
[Área], [Nombre], [Mensaje]
```

Área: Socio-emocional/Tarea

Nombre: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla A4.2: Formato de salida de la 1ª fase de clasificación

Como se puede ver en la Tabla A4.2, al dataset a clasificar se le agregó el primer atributo: *[Área]*, que es resultado de la ejecución de la primera fase de clasificación.

- Luego, nuevamente se generará un archivo temporal a partir de los datos obtenidos en la primera fase y la ejecución de la segunda fase.

El segundo archivo intermedio tendrá el siguiente formato, vea la Tabla A4.3:

```
@relation [relación]
...
@data
[Reacción],[Área],[Nombre],[Mensaje]
[Reacción],[Área],[Nombre],[Mensaje]
[Reacción],[Área],[Nombre],[Mensaje]
```

Reacción: Positiva/Negativa/Pregunta/Respuesta

Área: Socio-emocional/Tarea

Nombre: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla A4.3: Formato de salida de la 2º fase de clasificación

Como se puede ver en la Tabla A4.3, al dataset a clasificar (el generado en la primera fase) se le agregó el primer atributo: *[Reacción]*, que es resultado de la ejecución de la segunda fase de clasificación.

- Finalmente, en la tercera fase se tomará como entrada el archivo temporal de la segunda fase y se utilizará para clasificar las interacciones dentro de las 12 categorías de Bales.

El archivo tendrá el siguiente formato, Tabla A4.4:

```
@relation [relación]
...
@data
[Núm. de cat. IPA],[Reacción],[Área],[Nombre],[Mensaje]
[Núm. de cat. IPA],[Reacción],[Área],[Nombre],[Mensaje]
[Núm. de cat. IPA],[Reacción],[Área],[Nombre],[Mensaje]
```

Número de categoría de IPA: 1-12

Reacción: Positiva/Negativa/Pregunta/Respuesta

Área: Socio-emocional/Tarea

Nombre: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla A4.4: Formato de salida de la 3º fase de clasificación

Nuevamente, luego de esta fase se ve que se agregó un atributo al dataset: *[Núm. de cat. Bales]*, que es el archivo final de la clasificación.

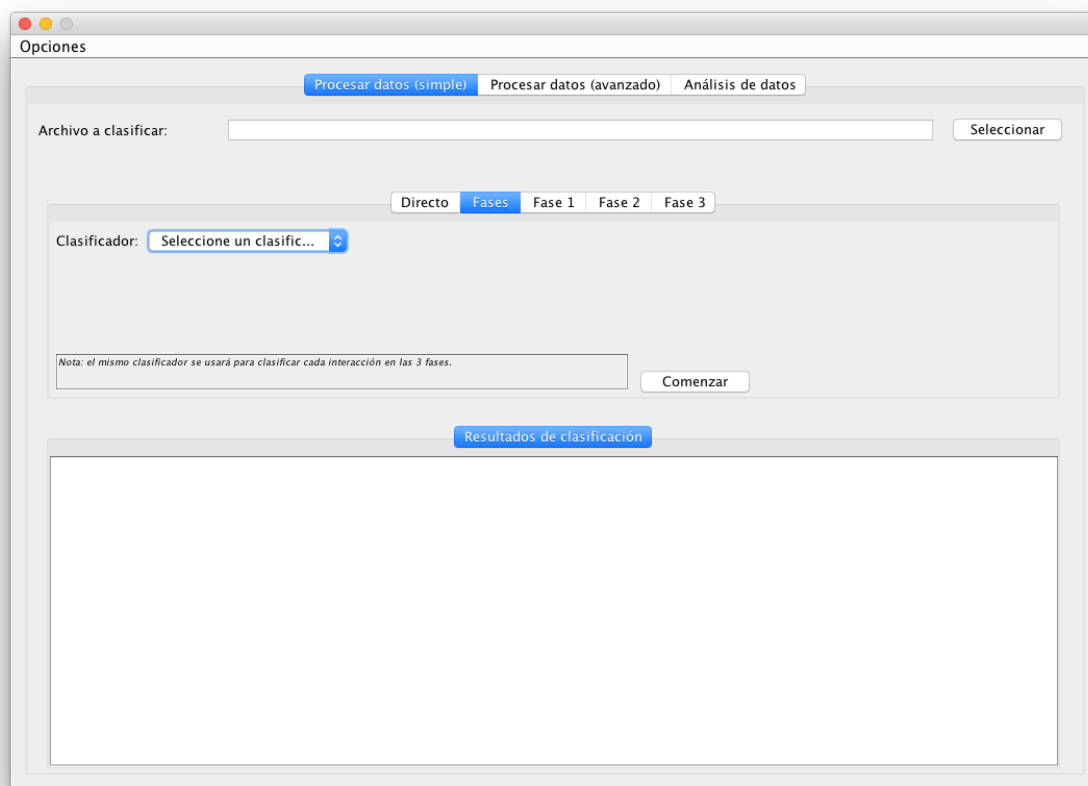


Figura A4.2: sección *Procesar datos (Simple)* de la aplicación, procesamiento *Fases*.

Fase 1, Fase 2 y Fase 3

A diferencia de lo explicado en las 2 subsecciones anteriores, donde el usuario solo podía elegir 1 clasificador, en esta sección el usuario deberá elegir 7 clasificadores, vea la Figura A4.3. El resto del funcionamiento de esta forma de clasificar un dataset es igual al explicado en la sección anterior.

- Primero se clasificará al dataset dentro del *área: Socio-emocional o Tarea*. Esta primera clasificación generará un archivo temporal que será utilizado por la siguiente fase. El archivo tendrá el formato mostrado en la Tabla A4.2.
- Luego, utilizando la salida de la primera fase, se clasificará este archivo dentro de las *reacciones* enumeradas por Bales (*positiva, negativa, pregunta y respuesta*). Solo que se generarán 2 archivos temporales, uno por cada clasificador elegido. Se utilizará uno de ellos para clasificar las interacciones clasificadas en la *Fase 1* como *Socio-emocional* y el otro para las clasificadas como *Tarea*. Los archivos tendrán el formato presentado en la Tabla A4.3.
- Finalmente, en la tercera fase se tomarán como entrada los archivos temporales de la segunda fase y se utilizarán para clasificar las interacciones dentro de las 12 categorías de Bales. Uno de los clasificadores se usará para clasificar las interacciones *Positiva*, otro de ellos se usará para las interacciones *Negativa*, el tercero para las interacciones *Pregunta* y el último para las interacciones *Respuesta*. El archivo tendrá el formato correspondiente a la Tabla A4.4.

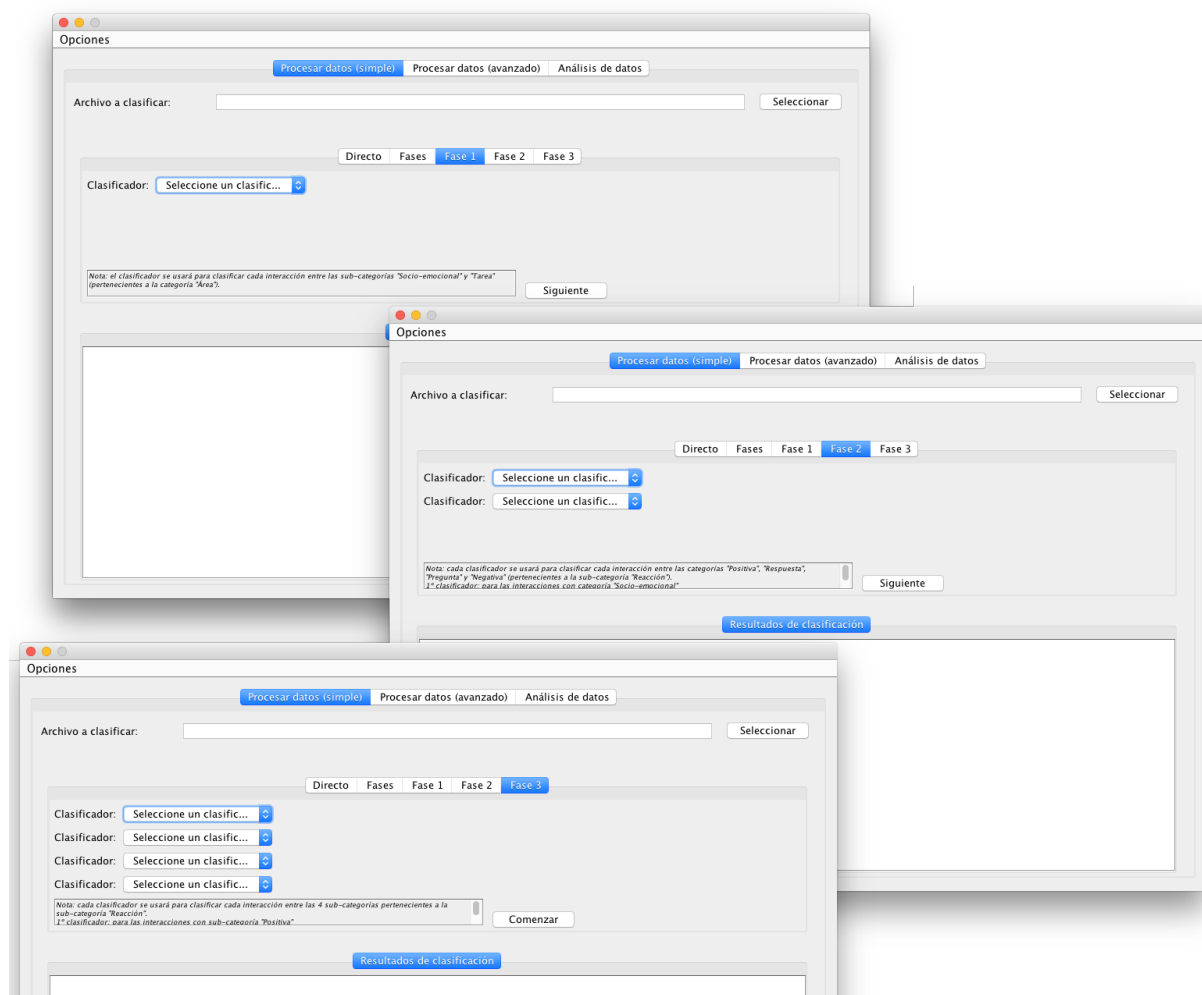


Figura A4.3: sección *Procesar datos (Simple)* de la aplicación, procesamiento *Fase 1*, *Fase 2* y *Fase 3*.

Procesar datos (avanzado)

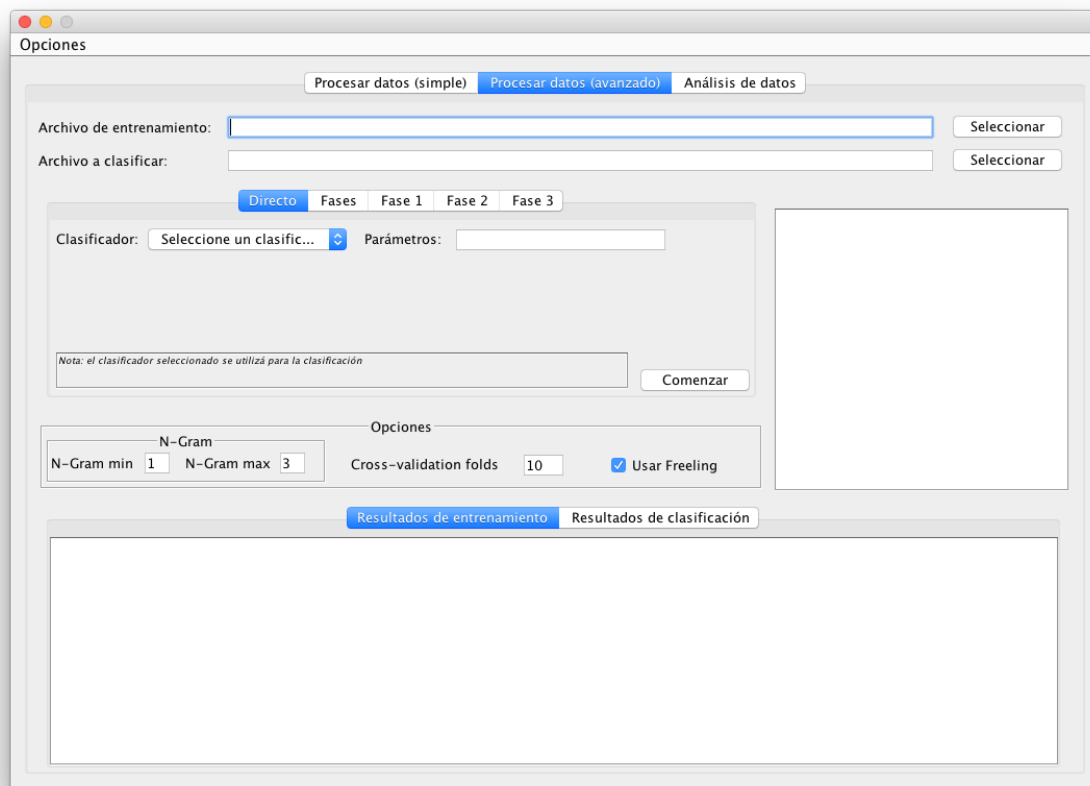


Figura A4.4: sección *Procesar datos (avanzado)* de la aplicación, procesamiento *Directo*.

Esta sección es muy similar a la anterior (*Procesar datos (simple)*), con la diferencia que el usuario tiene más opciones para elegir y configurar la clasificación a la hora de procesar sus datos. En esta sección también es posible elegir entre procesado directo o en fases, al igual que en *Procesar datos (simple)*, solo que aquí el usuario también puede ingresar distintos parámetros con los que se quiere configurar cada clasificador elegido, a la derecha se puede ver un cuadro de texto donde se detallan los parámetros que se pueden utilizar para el último clasificador seleccionado. En la documentación de WEKA se puede leer más sobre los parámetros que acepta cada clasificador y cómo usarlos. También es posible elegir la cantidad de pliegues (cross-validation folds) que se quieren utilizar, los N-gramas mínimos y máximos y si se desea utilizar FreeLing o no para el preprocesado de los datos. De la misma manera, el usuario deberá proveer a la aplicación un dataset de entrenamiento para que la aplicación “entrene”. La salida de esta sección será la misma que en la sección anterior, solo que el resultado va a demorar mucho más tiempo en mostrarse, debido a que al elegir esta sección, no se cuenta con los modelos de datos previamente generados y estos se generan en el momento. Se muestran las Figuras A4.4 y A4.5 a modo de referencia, al utilizar la aplicación se podrán ver todas las pantallas.

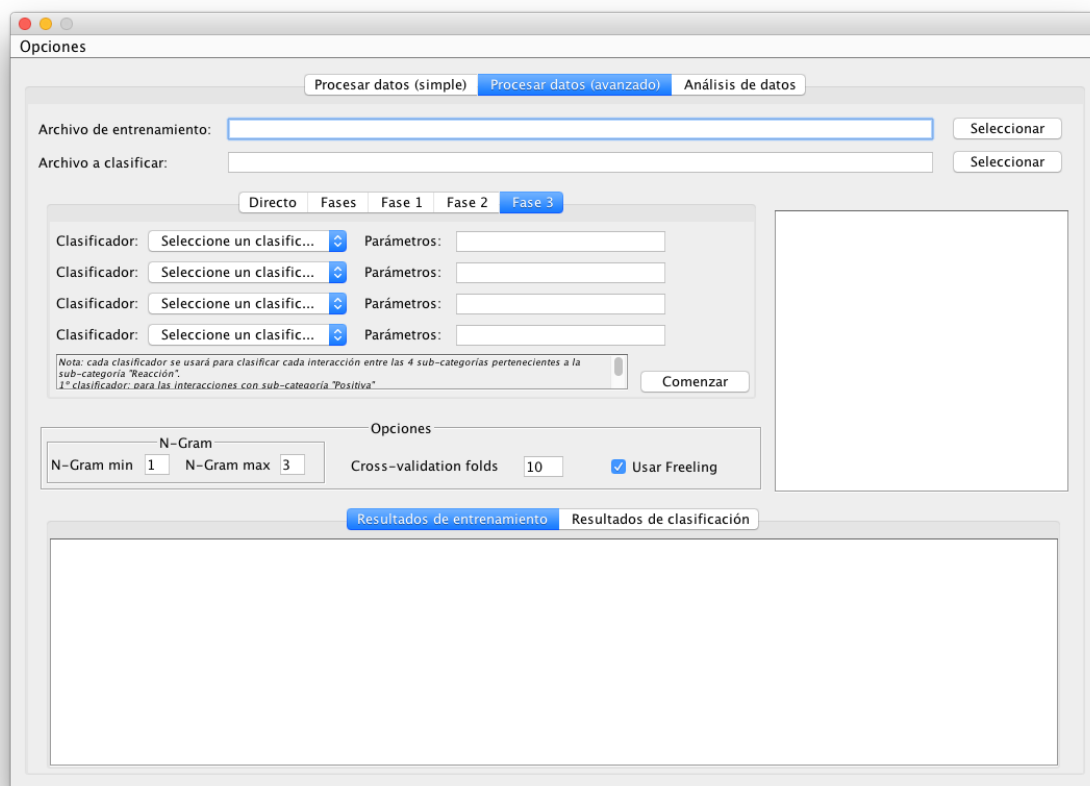


Figura A4.5: sección *Procesar datos (Avanzado)* de la aplicación, procesamiento *Fase 1*, *Fase 2* y *Fase 3*.

Análisis de datos

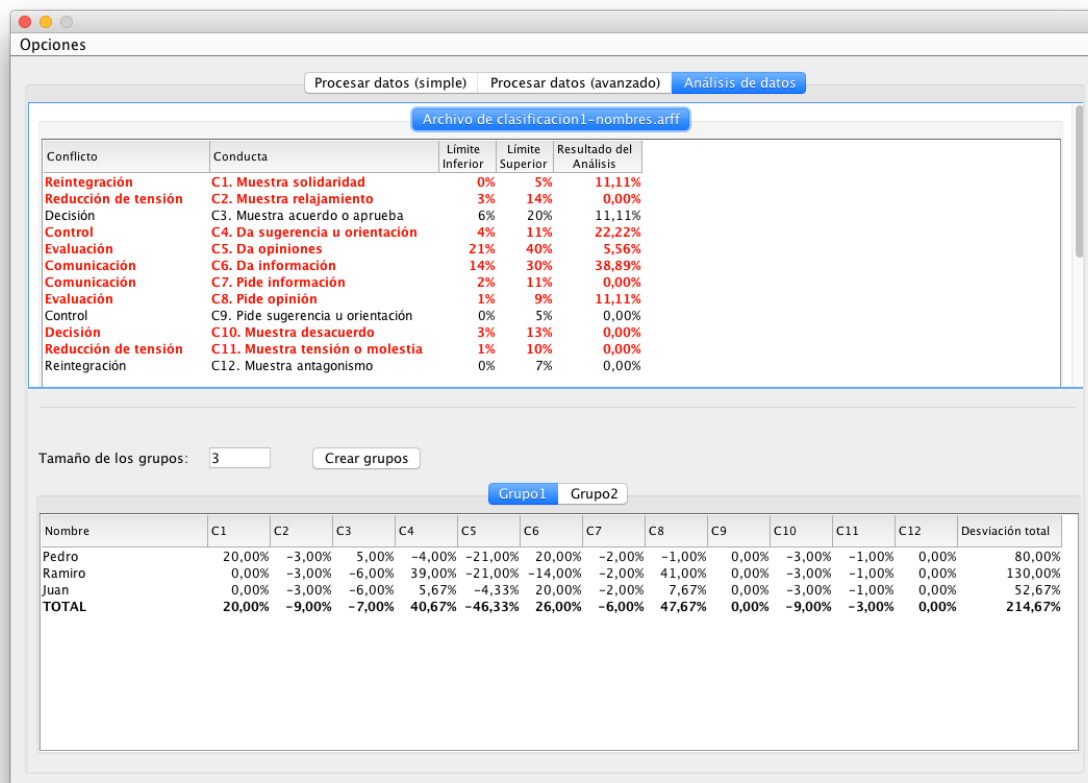


Figura A4.6: sección *Análisis de datos* de la aplicación

En esta sección se analizan y presentan los datos obtenidos en la sección *Procesar datos* (tanto la versión *simple* como la *avanzada*) así como también los resultados de la creación de nuevos grupos de trabajo colaborativo buscando reducir los conflictos, vea la figura A4.6. Lo que se hace es analizar los datos que quedaron registrados durante la clasificación en el/los archivo/s ARFF y a partir de estos datos se presentan los porcentajes de la cantidad de interacciones de cada *Conducta* IPA que tuvo cada grupo. Si un grupo en particular tuvo, para alguna de las *Conductas*, un porcentaje fuera de los valores límites identificador por Bales (véase la Tabla 2.2), estos datos serán presentados en color rojo para que resalten del resto. En esta sección, también se crea el perfil de cada persona que participó en los grupos de trabajo. Esta información será utilizada a la hora de crear nuevos grupos de trabajo buscando minimizar los conflictos. Para esto, el usuario puede elegir la cantidad de miembros que quiere que tenga cada grupo, así como también los miembros del grupo que desea que participen seleccionandolos de una lista y la aplicación le presentará los resultados de los grupos donde, según el perfil de cada usuario, se minimizan los conflictos.

Glosario

A continuación se presenta un glosario para referencia del lector en el cual se definen las palabras más específicas.

- **Dataset de entrenamiento**

El dataset de entrenamiento debe ser un archivo ARFF el cual deberá contar con los datos necesario para que la aplicación “entrene” y pueda crear el modelo de datos para luego poder clasificar. Este dataset deberá tener los chats de ejemplo y su clasificación dentro de alguna de las categorías IPA (en este caso no es necesario tener el nombre de quien escribió este mensaje).

Se puede ver un ejemplo de un dataset de entrada en la tabla G.1

<pre>@relation chat @attribute Conducta {1,2,3,4,5,6,7,8,9,10,11,12} @attribute message string @data [Núm. de cat. Bales],[Mensaje] [Núm. de cat. Bales],[Mensaje] [Núm. de cat. Bales],[Mensaje]</pre>
Número de categoría de Bales: 1-12 Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla G.1: ejemplo de un dataset de entrenamiento

- **Dataset a clasificar**

Es un conjunto de datos que la aplicación utilizará como entrada para luego clasificar. Este dataset debe tener la información que necesita la aplicación para funcionar: interacción que hace cada persona en el chat y el nombre de quién lo dijo. Estos datasets pueden ser archivos con extensión ARFF, JSON, XLS o XLSX. Lo que hará la aplicación al comenzar, es convertir cualquiera de estos archivos (a excepción de los archivos ARFF) de entrada en uno o varios archivos ARFF, el/los cuales serán utilizados por WEKA. La diferencia entre estos datasets a clasificar y los datasets de entrenamiento es que los atributos son diferentes y que para los datasets de entrenamiento no se tiene definida la conducta con un número, en este caso se reemplaza por un “?”.

Se puede ver un ejemplo de un dataset de entrada en la tabla 3.4

```
@relation chat
```

```
@attribute Conducta {1,2,3,4,5,6,7,8,9,10,11,12}
```

```
@attribute nombre string
```

```
@attribute message string
```

```
@data
```

```
?, [Nombre], [Mensaje]
```

```
?, [Nombre], [Mensaje]
```

```
?, [Nombre], [Mensaje]
```

Nombre: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Mensaje: debe ser una cadena de caracteres. Rodeado de comillas si contiene un espacio

Tabla G.2: ejemplo de un dataset de entrada

- **Modelo de datos**

Luego de que se hace el entrenamiento con el dataset de entrada, la aplicación generará el archivo del modelado de los datos con extensión DAT, este será el archivo que después se utilizará para clasificar el dataset de entrada.

- **Clasificador WEKA**

La aplicación le permite al usuario elegir con qué clasificador WEKA desea clasificar el dataset de entrada. WEKA tiene muchos clasificadores disponibles y a lo largo del desarrollo de la aplicación se fueron agregando más (siempre eligiendo clasificadores que sirvan para la clasificación de cadenas de caracteres).

- **N-gram**

En nuestra aplicación, el usuario tiene la posibilidad de elegir los *N-gramas* mínimos y máximos con los que desea que la aplicación trabaje. Por defecto, el n-grama mínimo está seteado en 1 y el máximo en 3.

- **Cross-validation folds**

Es posible elegir la cantidad de pliegues de validación cruzada que se desean utilizar para las pruebas y luego para el entrenamiento. Por defecto, el número es 10 pliegues.

- **Usar FreeLing**

El usuario tiene la posibilidad de elegir entre utilizar o no la herramienta FreeLing para el entrenamiento y clasificación de los datos.

Referencias

- [Aguirre, Soroa, 2009] Eneko Agirre and Aitor Soroa. 2009. Personalizing pagerank for word sense disambiguation. In Proceedings of the 12th conference of the European chapter of the Association for Computational Linguistics (EACL-2009), Athens, Greece.
- [Apple et al., 2011] Apple K, Reis-Bergan M, Adams A, Saunders G. Online tools to promote student collaboration. In: Getting connected: Best practices for technology enhanced teaching and learning in high education; 2011. p. 239–52.
- [Bales, 1950] Bales, R. A set of categories for the analysis of small group interaction. American Sociological Review, Vol. 151. 1950.
- [Birnholtz et al., 2013] Birnholtz J, Steinhardt S, Pavese A. Write here, write now!: an experimental study of group maintenance in collaborative writing. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, CHI 2013. New York, NY, USA: ACM; 2013. p. 961–70 . ISBN 978-1-4503-1899-0
- [Briscoe et al., 1996] G. Briscoe, and T. Caelli, “A Compendium of Machine Learning.Vol. 1: Symbolic Machine Learning.” Ablex Publishing Corporation, New Jersey, 1996.
- [Brodahl et al., 2011] Brodahl C, Hadjerrouit S, Hansen NK. Collaborative writing with web 2.0 technologies: education students’ perceptions. J Inf Technol Educ 2011;10:73–103.
- [Buschmann et al., 2001] Buschmann F, Meunier R, Rohnert H, Sommerlad P, Stal M - Pattern-Oriented Software Architecture, A System Of Patterns, Volume 1
- [Cavnar et al., 1994] Cavnar, W. B., & Trenkle, J. M. (1994). N-Gram-based text categorization. In Proceedings of the Third Symposium on Document Analysis and Information Retrieval. Las Vegas: UNLV Publications/Reprographics.
- [Chacon et al., 2014] Scott Chacon, Ben Straub. Pro Git, Second Edition, 2014
- [Cincunegui et al., 2015] Cincunegui M, Berdun F, Armentano MG, Amandi A. Clasificación de conductas colaborativas a partir de interacciones textuales. Argentine Symposium on Artificial Intelligence (ASAI 2015)-JAIIO 44 (Rosario, 2015); 2015.
- [Clark et al., 1989] P. Clark and T. Niblett. “The CN2 induction algorithm. Machine Learning”. 3:261-283, 1989.

- [Cooper et al., 1997] Cooper, G., et al. (1997). An evaluation of machine-learning methods for predicting pneumonia mortality. *Artificial Intelligence in Medicine*.
- [Dan et al., 2013] L. Dan, L. Lihua, and Z. Zhaoxin, "Research of Text Categorization on WEKA," in *Proceedings of the 2013 Third International Conference on Intelligent System Design and Engineering Applications*, 2013, pp. 1129-1131.
- [Delic et al., 2016] Delic A, Neidhardt J, Nguyen TN, Ricci F. Research methods for group recommender systems. *Proc RecTour 2016*.
- [Duda et al., 1973] R. Duda and P. Hart. "Pattern Classification and Scene Analysis". New York: John Wiley & Sons, 1973.
- [D'Silva, 2013] D'Silva, R. (2013). Buzzle. Retrieved July 24, 2013, from Benefits of Corporate Business Instant Messaging: <http://www.buzzle.com/articles/benefits-of-corporate-business-instant-messaging.html>
- [EAGLES, 1996a]. Recommendations for the morphosyntactic annotation of corpora, Eag-tcwg-mac/r, ILC-CNR, Pisa.
- [EAGLES, 1996b]. Synopsis and comparison of morphosyntactic phenomena encoded in lexicons and corpora, Eag-clwg-morphsyn/r, ILC-CNR, Pisa. A common proposal and applications to european languages.
- [ECMA, 2017] Standard ECMA-404, The JSON Data Interchange Syntax, 2nd edition (2017).
- [Fahy, 2006] Fahy, P. Online and face-to-face group interaction process compared using Bales' Interaction Process Analysis (IPA). *European Journal of Open, Distance and E-Learning*. 2006. http://www.eurodl.org/materials/contrib/2006/Patrick_J_Fahy.htm
- [Fischer, 2000] Fischer, G. User Modeling in Human - Computer Interaction. Contribution to the 10th Anniversary Issue of the Journal "User Modeling and User Adapted Interaction (UMUAI)", 2000.
- [Frank et al., 2016] Eibe Frank, Mark A. Hall, and Ian H. Witten (2016). The WEKA Workbench. Online Appendix for "Data Mining: Practical Machine Learning Tools and Techniques", Morgan Kaufmann, Fourth Edition, 2016.
- [Gamma et al., 1994] Erich Gamma, John Vlissides, Ralph Johnson y Richard Helm (1995) *Design Patterns, Elements of Reusable Object-Oriented Software*, 1995.

- [Goodman et al., 2005] Goodman BA, Linton FN, Gaimari RD, Hitzeman JM, Ross HJ, Zarrella G. Using dialogue features to predict trouble during collaborative learning. *User Model User-Adapt Interact* 2005;15(1-2):85–134.
- [Hall et al., 2009] Mark Hall, Eibe Frank, Geoffrey Holmes, Bernhard Pfahringer, Peter Reutemann, Ian H. Witten (2009); *The WEKA Data Mining Software: An Update*; SIGKDD Explorations, Volume 11, Issue 1.
- [Han et al., 2012] Han J., Kamber M., Pei J (2012); *Data Mining, Concepts and Techniques*.
- [Hong et al., 1986] J. Hong, I. Mozetic, and R. S. Michalski. "AQ15: Incremental learning of attribute-based descriptions from examples, the method and user's guide". In Report ISG 85-5, UIUCDCS-F-86-949, Department of Computer Science, University of Illinois at Urbana-Champaign, 1986.
- [Israel et al., 2007] Israel J, Aiken R. Supporting collaborative learning with an intelligent web-based system. *Int J Artif Intell Educ* 2007;17(1):3–40.
- [Jeff et al., 2001] Jeff, T., & Cooper, A. (2001, 28 28). HowStuffWorks. Retrieved July 12, 2013, from How Instant Messaging Works: <http://computer.howstuffworks.com/e-mail-messaging/instant-messaging.htm>
- [John, 1997] G. H. John. "Enhancements to the Data Mining Process". Ph.D. Thesis, Computer Science Dept., Stanford University, 1997.
- [Jurafsky, 2009] Jurafsky D., Martin J. (2009) *Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (2nd edition).
- [Kass, Finin, 1988] Kass, R., Finin, T.: Modeling the user in natural language systems. *Computational Linguistics* 14 (1988) 5–22
- [Kim et al., 2007] Kim MS, Kim YS, Kim TH. Analysis of team interaction and team creativity of student design teams base on personal creativity modes. In *ASME 2007 International Design Engineering Technical Conferences and Computers Information in Engineering COnference*. American Society of Mechanical Engineers; 2007. p. 55-67
- [Langley, 1996] P. Langley. "Elements of Machine Learning". San Francisco: Morgan Kaufmann, 1996.
- [Lee, 1989] Lee, K. (1989). *Automatic speech recognition: The development of the Sphinx system*. Boston: Kluwer Academic Publishers.

- [Löfstrand et al., 2014] Löfstrand P, Zakrisson I. Competitive versus non-competitive goals in group decision-making. *Small Group Res* 2014;45(4):451–64.
- [Major et al., 1995] J. Major and J. Mangano. "Selecting among rules induced from a hurricane database. *Journal of Intelligent Information Systems*",4:39-52, 1995.
- [Mansilla et al., 2014] Mansilla PS, Costaguta R, Schiaffino S. Multi agent model for skills training of CSCL e-tutors: modelo multi agente para el entrenamiento de habilidades de e-tutores de ACSC. In: *Proceedings of the 7th Euro American Conference on Telematics and Information Systems*. ACM; 2014. p. 30.
- [McAlister et al., 2004] McAlister S, Ravenscroft A, Scanlon E. Combining interaction and context design to support collaborative argumentation using a tool for synchronous CMC. *J Comput Assist Learn* 2004;20(3):194–204. ISSN:1365-2729
- [Mitchell, 1997] T. Mitchell, "Machine Learning," McGraw Hill, NY, 1997.
- [Molina López et al., 2006] José Manuel Molina López, Jesús García Herrero (2006). *Técnicas de análisis de datos - Aplicaciones prácticas utilizando Microsoft Excel y WEKA*.
- [Oishi, 2007] Oishi, L. Working Together: Google Apps Goes to School. *Technology & Learning*, 27(9), 46 (2007).
- [Olson et al., 2017] Olson JS, Wang D, Olson GM, Zhang J. How people write together now: beginning the investigation with advanced undergraduates in a project course. *ACM Trans Comput-Hum Interact* 2017;24(1) 4:1–4:40. ISSN 1073-0516.
- [Padró, Stanilovsky, 2012] Lluís Padró and Evgeny Stanilovsky. FreeLing 3.0: Towards Wider Multilinguality *Proceedings of the Language Resources and Evaluation Conference (LREC 2012) ELRA*. Istanbul, Turkey. May, 2012.
- [Pomerleau, 1989] Pomerleau, D. A. (1989). ALVINN: An autonomous land vehicle in a neural network. (Technical Report CMU-CS-89-107). Pittsburgh, PA: Carnegie Mellon University.
- [Quinlan, 1987] Quinlan, J.R.: "Simplifying decision trees", *International Journal of Man-Machine Studies*, núm. 27, pp. 221-234. 1987
- [Quinlan, 1993] Quinlan, J.R., "C4.5: Programs for Machine Learning," Morgan Kaufmann, CA, 1993.

- [Rich, 1989] Rich, E. (1989) "Stereotypes and User Modeling." In A. Kobsa & W. Wahlster (Eds.), *User Models in Dialog Systems*, Springer-Verlag, New York.
- [Rouse, 2003] Rouse, M. (2003). Learn IT: Instant Messaging in the Workplace. Retrieved July 12, 2013, from techtarget: <http://searchunifiedcommunications.techtarget.com/definition/Learn-IT-Instant-Messaging-in-the-Workplace>
- [Rumelhart et al., 1986] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. "Learning internal representations by error propagation". In D. E. Rumelhart and J. L. McClelland, editors, *Parallel Distributed Processing*. Cambridge, MA: MIT Press, 1986.
- [Samuel, 1959] Samuel, Arthur L. (1959). "Some Studies in Machine Learning Using the Game of Checkers". *IBM Journal of Research and Development*.
- [Sanchez-Marco et al., 2011] Cristina Sánchez-Marco, Gemma Boleda, and Lluís Padró. 2011. Extending the tool, or how to annotate historical language varieties. In *Proceedings of the 5th ACL-HLT Workshop on Language Technology for Cultural Heritage, Social Sciences, and Humanities*, pages 1–9, Portland, OR, USA, June. Association for Computational Linguistics.
- [Savolainen, 2011] Savolainen R . Asking and sharing information in the blogosphere: the case of slimming blogs. *Libr Inf Sci Res* 2011;33(1):73-9. ISBN 0740-8188
- [Tesauro, 1992] Tesauro, G. (1992). Practical issues in temporal difference learning. *Machine Learning*, 8, 257.
- [Tesauro, 1995] Tesauro, G. (1995). Temporal difference learning and TD-gammon. *Communications of the ACM*, 38(3), 5848.
- [Turner et al., 2010] Turner, T., Qvarfordt, P., Biehl, J. T., Golovchinsky, G., & Back, M. (2010). *Exploring the Workplace Communication Ecology*. FX Palo Alto Laboratory, Inc, Atlanta, Georgia, USA.
- [Uddin et al., 2017] Khaja Viqar Uddin, MD Daniyal Raza, Faheem Uddin, Abdul Rais (2017). Online File-Hosting Service, Lords Institute of Engineering and Technology, Hyderabad, India.
- [Utgoff, 1988] Utgoff, P.E., "An Incremental ID3." In *Proc. Fifth Int. Conf. Machine Learning*, pages 107-120, San Mateo, CA, 1988
- [Wahlster, Kobsa, 1986] Wahlster, W., Kobsa, A.: Dialogue-based user models. In: *Proceedings of the IEEE*. (1986) 948–960 Published as *Proceedings of the IEEE*, volume 74, number 7.

- [Waibel et al., 1989] Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., & Lang, K. (1989). Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech and Signal Processing*, 37(3), 328-339.
- [Weiss et al., 1991] S.M. Weiss and C. A. Kulikowski. "Computer Systems That Learn: Classification and Prediction Methods from Statistics, Neural Nets, Machine Learning, and Expert Systems". San Mateo, CA: Morgan Kaufmann, 1991.
- [Weiss et al., 1998] S.M Weiss, and Indurkha. "Predictive Data Mining". San Francisco: Morgan Kaufmann 1998
- [Woodard, 2011] Woodard, R. (2011, January 24). Pros and Cons of Instant Messaging. Retrieved July 23, 2013, from Bright Hub: <http://www.brighthub.com/office/collaboration/articles/104224.aspx>
- [Zhang et al., 2005] Zhang C, Zhang C. Discovering users' participant roles in virtual communities with the help of social interaction theories. *PACIS 2005 Proc* 2005:65.
- [Zhou et al., 2012] Zhou W, Simpson E, Domizi DP. Google docs in an out-of-class collaborative writing activity. *Int J Teaching Learn Higher Educ* 2012;24(3):359–75.